



Escola Politécnica de Enxeñaría de Ferrol

UNIVERSIDADE DA CORUÑA



Trabajo Fin de Grado CURSO 2023/2024

*Diseño de equipo de adquisición de datos inalámbrico
para plantas de control de temperatura*

Grado en Ingeniería Electrónica Industrial y Automática

AUTOR: IVÁN HERMIDA MELLA

TUTORES: FRANCISCO ZAYAS GATO
PAULA PATRICIA ARCANO BEA

TFG N°: 2324_GEIA_7

FECHA: SEPTIEMBRE 2024

Título Diseño de equipo de adquisición de datos inalámbrico
para plantas de control de temperatura

Índice general

Peticionario Escuela Politécnica de Ingeniería de Ferrol
Rúa Mendizábal, s/n, Campus de Esteiro,
15403, Ferrol

Fecha Septiembre 2024

Autor El Alumno

Fdo. Iván Hermida Mella

Índice general

1	Índice general	1
2	Memoria	5
2.1	Objeto	7
2.2	Alcance	7
2.3	Antecedentes	7
2.4	Normas y referencias	9
2.4.1	Disposiciones legales y normas aplicadas	9
2.4.2	Bibliografía	9
2.4.3	Programas de cálculo	10
2.5	Definiciones y abreviaturas	10
2.6	Requisitos de diseño	11
2.7	Análisis de las soluciones	11
2.7.1	Selección del microcontrolador	11
2.7.2	Implementación del regulador	12
2.8	Resultados finales	12
2.8.1	Descripción general	12
2.8.2	Planta	16
2.8.3	Actuador	16
2.8.3.1	Circuito detector de paso por cero	18
2.8.3.2	Circuito de disparo	20
2.8.3.3	Circuito de acondicionamiento de la señal de control	22
2.8.3.4	Circuito de alimentaciones	24
2.8.4	Sensor	29
2.8.4.1	Circuito de acondicionamiento	32
2.8.5	Software	33
2.8.5.1	Adquisición de datos y control de potencia con Wemos D1	33
2.8.5.2	Servidor MQTT con Raspberry Pi	36
2.8.5.3	Regulador PID con Node-RED	40
2.8.5.4	Base de datos de series temporales con InfluxDB	46
2.9	Orden de prioridad entre los documentos	50
3	Anexos	53
3.1	Contenido	55
3.2	Documentación de partida	55
3.3	Cálculos	58
3.3.1	Circuito detector de paso por cero	58
3.3.2	Circuito de disparo	63
3.3.3	Circuito de acondicionamiento de la señal de control	64

3.3.4	Temporización del ESP8266	65
3.3.5	Ajuste del regulador PID	70
3.4	Anexos en función del ámbito de aplicación del trabajo	74
3.4.1	Código de Wemos D1	75
3.4.2	Código controlador PID en Node-Red	78
3.4.3	Código controlador Relay-Feedback en Node-Red	89
4	Estudios con entidad propia	101
4.1	Contenido	103
5	Planos	105
	Circuito detector de paso por cero	107
	Circuito de disparo	108
	Circuito de acondicionamiento de la señal de control	109
	Circuito de alimentaciones	110
	Circuito de acondicionamiento del sensor	111
	Conexiones con Wemos D1	112
6	Pliego de condiciones	113
6.1	Contenido	115
7	Mediciones	117
7.1	Lista de materiales	119
7.1.0.1	Equipo electrónico	119
7.1.0.2	Conexión del equipo electrónico	120
7.1.0.3	Equipo de control	120
8	Presupuesto	121
8.1	Cuadros de precios	123
8.1.1	Equipo electrónico	123
8.1.2	Conexión del equipo electrónico	124
8.1.3	Equipo de control	124
8.2	Mano de obra	124
8.3	Coste total	125

Título Diseño de equipo de adquisición de datos inalámbrico
para plantas de control de temperatura

Memoria

Peticionario Escuela Politécnica de Ingeniería de Ferrol
Rúa Mendizábal, s/n, Campus de Esteiro,
15403, Ferrol

Fecha Septiembre 2024

Autor El Alumno

Fdo. Iván Hermida Mella

Índice general

2.1	Objeto	7
2.2	Alcance	7
2.3	Antecedentes	7
2.4	Normas y referencias	9
2.4.1	Disposiciones legales y normas aplicadas	9
2.4.2	Bibliografía	9
2.4.3	Programas de cálculo	10
2.5	Definiciones y abreviaturas	10
2.6	Requisitos de diseño	11
2.7	Análisis de las soluciones	11
2.7.1	Selección del microcontrolador	11
2.7.2	Implementación del regulador	12
2.8	Resultados finales	12
2.8.1	Descripción general	12
2.8.2	Planta	16
2.8.3	Actuador	16
2.8.3.1	Circuito detector de paso por cero	18
2.8.3.2	Circuito de disparo	20
2.8.3.3	Circuito de acondicionamiento de la señal de control	22
2.8.3.4	Circuito de alimentaciones	24
2.8.4	Sensor	29
2.8.4.1	Circuito de acondicionamiento	32
2.8.5	Software	33
2.8.5.1	Adquisición de datos y control de potencia con Wemos D1	33
2.8.5.2	Servidor MQTT con Raspberry Pi	36
2.8.5.3	Regulador PID con Node-RED	40
2.8.5.4	Base de datos de series temporales con InfluxDB	46
2.9	Orden de prioridad entre los documentos	50

2.1. Objeto

El objeto del presente proyecto es el diseño de un nuevo equipo electrónico de adquisición de datos y de interfaz para el control de potencia entregada a las plantas de control de temperatura que se encuentran en el Laboratorio de Optimización y Control de la EPEF, que incorpore nuevas funcionalidades de control y configuración de forma inalámbrica. Además, el nuevo equipo deberá ser compatible con el actual en cuanto al intercambio de señales.

2.2. Alcance

- Estudio de opciones para la actualización del microcontrolador que gestiona el equipo.
- Diseño del nuevo circuito con las nuevas especificaciones.
- Diseño electrónico del nuevo equipo.
- Implementación del prototipo.
- Software o app para test y/o configuración de forma inalámbrica.

2.3. Antecedentes

El Laboratorio de Optimización y Control de la EPEF cuenta con diversas plantas industriales a escala, que representan una herramienta importante para la formación del alumnado. Existen tres tipos de plantas diferentes:

- Las plantas de nivel (figura 2.1) están formadas por dos depósitos. En esta planta, el elemento actuador es una bomba, de forma que cuando se alimenta la bomba se bombea agua desde el depósito inferior al superior, por lo que la señal de control es la potencia aplicada a la bomba, mientras que la variable controlada es el nivel de agua en el depósito superior. Además, el depósito superior cuenta con una válvula de alivio cuya apertura se puede modificar, haciendo que el nivel baje a mayor o menor velocidad.

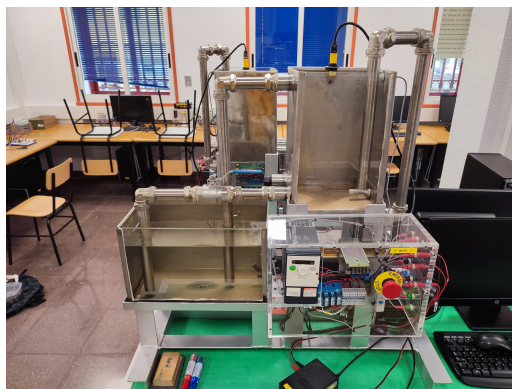


Figura 2.1: Planta de nivel

- Las plantas de temperatura (figura 2.2) consisten principalmente en una caja prismática de metacrilato en cuyo interior se encuentran dos bombillas incandescentes, de forma que la planta funciona como un horno en miniatura. Al encenderse las bombillas, se calienta el interior de la caja y se mide esta temperatura a través de un sensor. La señal de control es la potencia aplicada a las bombillas, mientras que la señal que se pretende manipular es la temperatura en la planta.

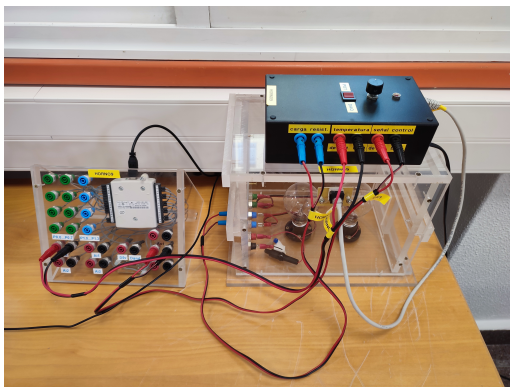


Figura 2.2: Planta de temperatura

- Los aeropéndulos (figura 2.3) consisten en un péndulo unido a un soporte en cuyo extremo libre se sitúa una aspa accionada por un pequeño motor de CC. Cuando se alimenta este motor, el aspa comienza a girar a mayor o menor frecuencia haciendo levantarse el péndulo hasta un ángulo determinado sobre la vertical.



Figura 2.3: Aeropéndulo

En concreto, las plantas de control de temperatura cuentan con diversos elementos hardware para formar un conjunto o bloque y así desplegar un sistema de control completo. Entre ellos, se dispone de una interfaz de control de potencia totalmente funcional pero que carece de algunas funcionalidades como capacidad de test, control de forma inalámbrica, más opciones para alimentación o mayor número de interfaces de entrada y salida, entre otros.

El equipo de control y adquisición de datos empleado actualmente está basado en el micro-controlador PIC18F252 que controla la potencia que entrega el circuito de control de potencia a las bombillas incandescentes del horno. Si bien se emplea un controlador diferente para el control de potencia con nuevas funcionalidades, se parte del circuito de control ya existente.

2.4. Normas y referencias

En esta sección se indica la normativa aplicada a lo largo de la elaboración del proyecto, así como la bibliografía utilizada y cualquier otro tipo de fuente de información consultada.

2.4.1. Disposiciones legales y normas aplicadas

- UNE157001 - Criterios generales para la elaboración formal de los documentos que constituyen un proyecto técnico
- UNE-EN 60617:2017 - Símbolos gráficos para esquemas de equipos y sistemas eléctricos y electrónicos
- UNE-EN 62368-1:2021 - Equipos de tecnología de la información y de telecomunicaciones. Requisitos de seguridad
- ISO/IEC 20922:2016 - Information technology - Message Queuing Telemetry Transport (MQTT) v3.1.1
- UNE-ISO/IEC 27001:2017 - Tecnología de la información. Técnicas de seguridad. Sistemas de gestión de la seguridad de la información
- UNE-EN 301 489-17 V3.2.4:2021 - Compatibilidad electromagnética y espectro radioeléctrico (ERM). Norma para equipos de radio de banda ancha y sistemas de transmisión de datos

2.4.2. Bibliografía

- **Libros:**

- M.A. Pérez García, J. C. Álvarez Antón, J. C. Campo Rodríguez, F. J. Ferrero Martín, G. J. Grillo Ortega, «Instrumentación Electrónica», Thomson
- D. W. Hart, «Electrónica de potencia», Prentice Hall

- **Artículos:**

- J. L. Calvo Rolle, «Ajuste empírico de reguladores PID»

- **Bibliografía web:**

- <https://nodered.org/docs/>
- <https://flows.nodered.org/node/node-red-contrib-easy-pid-controller>
- <https://flows.nodered.org/node/node-red-dashboard>

- <https://flows.nodered.org/node/node-red-contrib-influxdb>
- <https://www.arduino.cc/reference/en/libraries/pubsubclient/>
- <https://github.com/wemos>
- <https://www.raspberrypi.com/documentation/computers/os.html>

2.4.3. Programas de cálculo

Los programas de diseño y cálculo utilizados para el desarrollo del presente proyecto son:

- **OrCAD Capture CIS 17.2.** Software para el diseño, cálculo y simulación de los circuitos electrónicos que integran el sistema.
- **Gecko Circuits.** Software para el diseño, cálculo y simulación de los circuitos electrónicos que integran el sistema.
- **Arduino IDE 2.2.** Entorno de desarrollo y depuración empleado para la programación de la Wemos D1.
- **AutoCAD 2024.** Software de diseño para la obtención de planos de los circuitos electrónicos según los cajetines estandarizados de la Escuela Politécnica de Ingeniería de Ferrol.
- **Node-RED.** Software de programación visual para el desarrollo del algoritmo del regulador PID, visualización del sistema en dashboard y comunicación con Mosquitto e InfluxDB.
- **InfluxDB 2.7.** Base de datos de series temporales para registrar los mensajes enviados en el servidor MQTT.

2.5. Definiciones y abreviaturas

- CI: Circuito impreso
- A/D: Analógico/digital
- ADC: Analog-to-Digital Converter (convertidor analógico a digital)
- DAQ: Data Acquisition System (sistema de adquisición de datos)
- TAD: Tarjeta de Adquisición de Datos
- GPIO: General Purpose Input/Output (entrada/salida de propósito general)
- MQTT: Message Queuing Telemetry Transport

2.6. Requisitos de diseño

Para la correcta consecución de este proyecto deben cumplirse los siguientes requisitos de diseño:

- Compatibilidad con el sistema de control existente en la planta, basado en la tarjeta de adquisición de datos NI PCI-6221. El equipo electrónico debe poder ser conectado con el circuito de control que se utiliza normalmente en el laboratorio sin la necesidad de elementos hardware adicionales.
- Implementación de un sistema de control que permita:
 - Controlar de forma eficaz la temperatura en la planta en tiempo real.
 - Actuar sobre la planta de forma remota, vía WiFi. Existirá conexión cableada entre el circuito de control y la planta, pero la señal de control se enviará desde un equipo externo y no a través de una señal eléctrica.
 - Almacenar y visualizar los datos referentes a la planta en una base de datos de series temporales.

2.7. Análisis de las soluciones

Se exponen en este apartado las diferentes soluciones que se han tomado en consideración para cumplir con los citados requisitos de diseño.

2.7.1. Selección del microcontrolador

El principal objetivo del proyecto es que el nuevo equipo de control funcione de forma inalámbrica. Esto obliga a sustituir el microcontrolador que se emplea en el sistema de control actual, el PIC18F252, por uno compatible con protocolos de comunicación inalámbricos, tales como WiFi o Bluetooth.

Los sistemas más empleados en proyectos relacionados con IoT son aquellos basados en ESP32 y ESP8266. Ambas opciones ofrecen compatibilidad WiFi, pero ESP32 permite además conexión con Bluetooth. Esto podría suponer un factor importante a la hora de decantarse por la elección de ESP32, pero como se estableció que el protocolo empleado para la comunicación del sistema de control con la planta sería MQTT, el uso de Bluetooth no va a ser necesario. Teniendo lo anterior en cuenta, existen dos factores principales a la hora de decantarse por el uso de un sistema basado en ESP8266:

- En general, el coste de las placas de desarrollo basadas en ESP8266 es inferior al de las basadas en ESP32, si bien el coste de ambas suele ser moderado, pudiendo catalogarse las dos dentro del hardware de bajo coste.
- El microcontrolador ESP8266 presenta un consumo energético menor que ESP32. ESP32 presenta consumos superiores a 100 mA cuando se emplea la conexión WiFi, mientras que el consumo de ESP8266 ronda los 70 mA. Si bien estas cifras no son exactas y presentan variaciones en función del modo de funcionamiento del microcontrolador, en general se puede decir que el consumo de ESP32 es superior.

A pesar de esto, ESP32 también cuenta con una importante ventaja: cuenta con hasta 18 entradas analógicas, mientras que ESP8266 dispone de una única entrada a ADC. Este mayor número de entradas analógicas permitiría tener conectados ambos sistemas de control a la vez al actuador, eligiendo el método de control a través de software o de un interruptor hardware. Se ha decidido finalmente el empleo de una placa de desarrollo basada en ESP8266, concretamente de Wemos D1, cuya similitud a la hora de ser programado y en la disposición de los pines con un Arduino Uno simplifica enormemente su uso, frente a otro tipo de placas de mayor tamaño cuya conexión con el prototipo sería más costosa.

2.7.2. Implementación del regulador

A la hora de elegir el lenguaje o entorno de programación a utilizar para implementar el regulador de la planta existen múltiples alternativas muy variadas: Node-RED, Python, MATLAB o Simulink, etc.

Se descarta inicialmente el uso de MATLAB y/o Simulink por el hecho de que es el sistema que se venía utilizando normalmente en el laboratorio junto con la tarjeta de adquisición de datos NI PCI-6221 y se buscaba una renovación completa de este sistema, tanto a nivel de hardware como de software.

Se decide la utilización de Node-RED frente a otros lenguajes de texto estructurado más convencionales como Python debido a la sencillez e intuitividad de su entorno gráfico, así como al elevado número de librerías disponibles que permiten integrar de forma sencilla con nodos ya existentes el regulador PID, la comunicación con MQTT o el envío de datos a InfluxDB. Además, se considera interesante su uso al diferenciarse claramente de la inmensa mayoría de lenguajes que se han venido viendo durante el estudio del Grado.

2.8. Resultados finales

En este apartado se lleva a cabo una exposición en detalle de los resultados finales obtenidos en el presente proyecto.

2.8.1. Descripción general

La función principal del equipo diseñado es el control de la temperatura en el interior de un horno de metacrilato con dos bombillas incandescentes de 100W de potencia. Para ello, se emplea una estructura de control realimentado con un controlador. Un sistema de control con realimentación responde, de forma genérica, a la estructura representada en la figura 2.4.

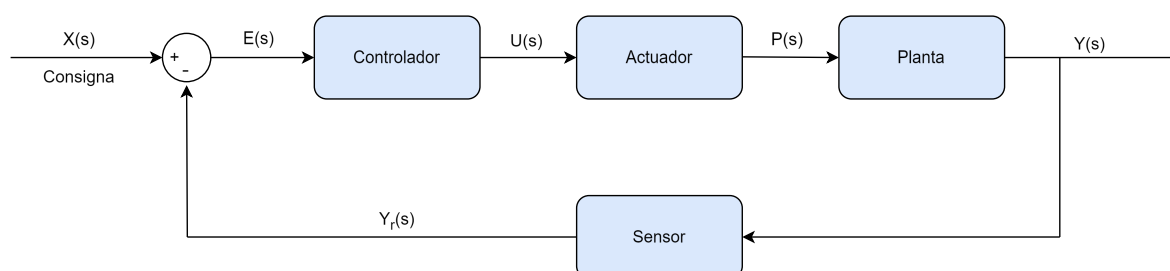


Figura 2.4: Sistema de control con realimentación

El lazo de control consta de 4 bloques: controlador, actuador, planta y sensor. La interacción entre los diferentes bloques con unas condiciones concretas permitirá el correcto control de la temperatura de la planta. Cada uno de los bloques trabaja con unas señales de entrada y genera unas señales de salida que han de adaptarse a las condiciones requeridas.

Se parte de una señal de consigna (o set-point), $X(s)$, que marca el punto de trabajo deseado para la planta, en este caso, la temperatura deseada en el horno. Este valor será establecido vía software por el usuario en el rango 0-100°C, que es para el que está diseñado el sistema. En la práctica será complicado que la planta alcance temperaturas muy elevadas debido a la potencia ofrecida por las bombillas y a la falta de aislamiento del horno, pues se trata de equipo de uso didáctico y no de un horno industrial.

Esta señal de consigna se compara con la señal de salida (temperatura) medida por el sensor, $Y_r(s)$, y se obtiene la señal de error $E(s)$ como la diferencia entre ambas. La señal $Y_r(s)$ es resultado del acondicionamiento de la señal de salida $Y(s)$, puesto que para poder compararla con la consigna ambas señales deben ser del mismo tipo.

La señal de error $E(s)$ resulta de la diferencia entre la consigna y la realimentación. Esta señal es llevada como entrada al controlador, que generará una señal de control $U(s)$ para el actuador en función del valor del error.

Esta señal $U(s)$ de salida del controlador es de vital importancia, pues la respuesta del sistema presentará una fuerte dependencia de ella, por ser la que ataca directamente al elemento actuador del sistema que provoca los cambios de temperatura.

Finalmente, a partir de la señal de control adaptada, el actuador proporcionará una potencia $P(s)$ en cada instante de tiempo para atacar a la carga situada en el interior de la planta (en este caso, el horno).

La variable que buscamos manipular es la temperatura en el interior del horno $Y(s)$, que se mide a través de un sensor para obtener una realimentación y cerrar el lazo de control, repitiéndose el proceso aquí explicado.

Existirán dos variantes de este lazo de control pues, como se explica en secciones posteriores del documento, se contemplan dos sistemas de control de la temperatura en lazo cerrado:

- El sistema previamente utilizado en la planta, donde la temperatura leída por el sensor es acondicionada para ser adquirida mediante una TAD. Esta TAD es la encargada de generar la señal de control enviada al circuito de disparo, que regula la potencia entregada a la planta (figura 2.5).
- El nuevo sistema de control remoto a través de WiFi, implementado a través del hardware Raspberry Pi y el software Node-Red mediante protocolo MQTT, que permite realizar el control de la planta sin necesidad de una conexión cableada entre el equipo (PC) y el conjunto planta/circuito actuador y registrar señales en bases de datos de series temporales con InfluxDB. (figura 2.6).

En ambos sistemas de control existen ciertas diferencias en cuanto a los medios físicos empleados para la transmisión de señales, el algoritmo de control y ciertos elementos del circuito actuador, pero ambos responden al esquema representado en la figura 2.4.

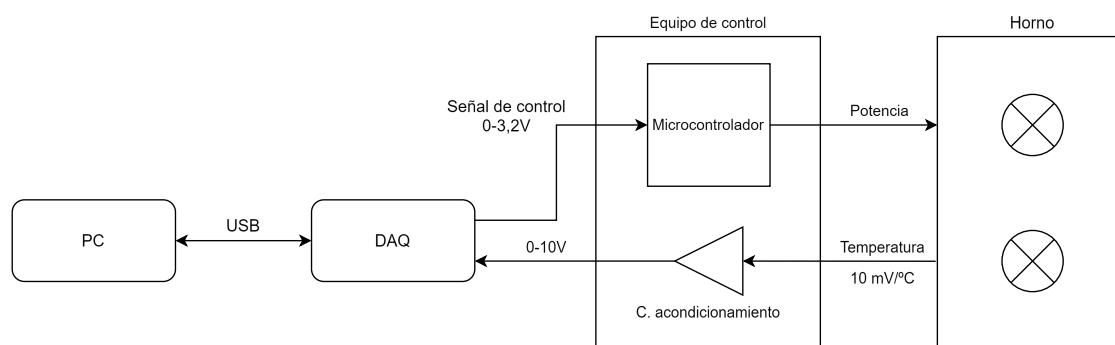


Figura 2.5: Esquema del sistema de control de la planta a través de TAD

Cuándo la adquisición se realiza mediante la tarjeta de adquisición de datos, el sistema funciona de la siguiente forma:

1. Desde el PC el usuario fija un nivel de consigna de temperatura entre 0°C y 100°C , que se transmite a la TAD.
2. La temperatura de la planta es medida mediante un sensor de temperatura LM35 que presenta una sensibilidad de $10\text{mV}/^{\circ}\text{C}$, lo que nos devuelve una tensión de 0-1V para el rango de temperaturas a medir ($0\text{-}100^{\circ}\text{C}$). Esta señal se hace pasar por un circuito de acondicionamiento para adaptarla al rango 0-10V, que es el rango de entrada de la TAD.
3. El PC recibe la lectura de la temperatura como un valor (0-1023) entero tras ser adquirido por la TAD, y se calcula la señal de error como la diferencia entre la consigna y la lectura de temperatura.
4. El algoritmo de control implementado en el PC (MATLAB, Simulink, etc.) utiliza esta señal de error para obtener una señal de control entre 0 y 10 voltios que se envía al microcontrolador a través de la TAD.
5. Tras ser acondicionada para adaptarla al rango 0-3,2V, esta señal llega a la entrada del convertidor A/D del microcontrolador, que obtiene a partir de esta un valor en el rango 0-100, que representa el % de potencia a entregar a la planta.

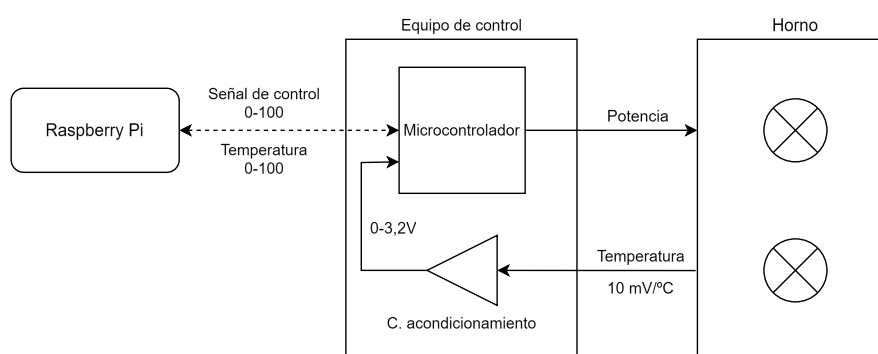


Figura 2.6: Esquema del sistema de control de la planta a través de Raspberry Pi

Cuándo el control se realiza a través del programa de Node-Red implementado en Raspberry Pi, el funcionamiento del sistema es ligeramente diferente.

La temperatura se mide igualmente con un sensor LM35, que genera una señal entre 0 y 1 voltio. En este caso, el circuito de acondicionamiento de la señal de temperatura es diferente, pues esta señal se va a mandar directamente a la entrada del convertidor A/D del microcontrolador, que tiene un rango de entrada de 0-3,2V, frente al rango 0-10V que presentaba la TAD.

EL microcontrolador adapta esta señal analógica a un valor en coma flotante entre 0 y 100, que representa el valor real de la temperatura en la planta, y envía el valor a la Raspberry Pi mediante WiFi empleando el protocolo MQTT. En la Raspberry Pi se implementa un programa (flujo) de Node-Red, que es el encargado de obtener la señal de control (0-100) que se envía al micro también a través de MQTT.

El sistema integra también una tercera opción de control, utilizada principalmente para probar el sistema. En este último modo de funcionamiento no existe una realimentación del lazo de control, si no que el usuario actúa directamente sobre el circuito actuador mediante un potenciómetro para fijar manualmente la señal de control, de forma que es el propio usuario el que gestiona la potencia entregada a la planta, no existiendo como tal un regulador o controlador que calcule esta señal de control. Se trata por tanto de un sistema de control en lazo abierto en el que no existe una regulación automática, por lo que su finalidad es la de realizar pruebas sobre la respuesta de la planta y comprobar el correcto funcionamiento del circuito, más que servir como una forma de control alternativa a las anteriores.

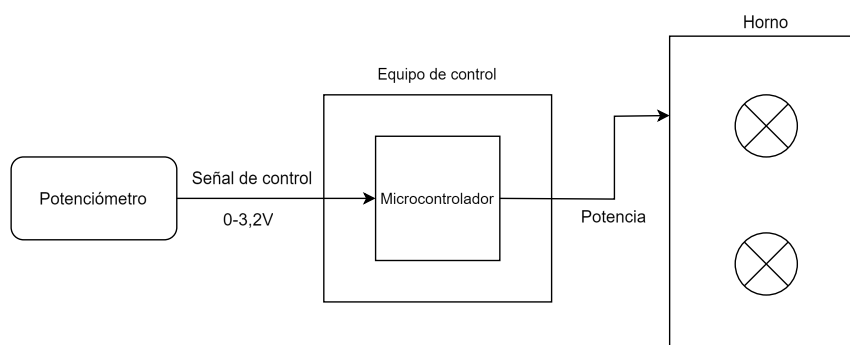


Figura 2.7: Esquema del sistema de control de la planta en lazo abierto

El proyecto comprende, por tanto, el diseño de la electrónica del sistema y la implementación del nuevo sistema de intercambio de señales para permitir los modos de funcionamiento anteriormente comentados. El orden seguido en el desarrollo del proyecto es el siguiente:

- **Actuador.** Se diseña el circuito encargado de entregar la potencia necesaria a la planta en función del valor de la señal de control, que ha de seguir una proporción lineal.
- **Sensor.** La planta presente en el laboratorio incluye el propio sensor ya montado sobre una placa de circuito impreso que permite su conexionado con el circuito de acondicionamiento. El circuito de acondicionamiento se integra en la misma placa de prototipo que el circuito de potencia.

- **Montaje y conexionado.** Se especifican los tipos de conexiones que se han de realizar para la implementación del controlador y la puesta en marcha del sistema.
- **Controlador.** Se ofrece la posibilidad de implementar el algoritmo de control a través de MATLAB/Simulink o Node-Red, cada uno en caso de empleo del sistema correspondiente, como se ha expuesto previamente. Esto es así para mantener la compatibilidad con el sistema de control actualmente existente en el laboratorio basado en TAD de National Instruments. En este proyecto, el análisis del software empleado y del controlador se centrarán en el método de control vía MQTT a través de Node-RED.
- **Base de datos.** Con la herramienta InfluxDB se implementa una base de datos para el registro y la visualización de históricos. Las señales que se van a registrar son la señal de control ($U(s)$ en el esquema 2.4) y la señal de salida ($Y(s)$ en el esquema 2.4), que en este caso es la temperatura.

2.8.2. Planta

La planta para la que se va a diseñar el sistema de control consiste en un horno de metacrilato, cuyo elemento calefactor son dos bombillas incandescentes de 100W. Algunas de estas plantas presentes en el laboratorio cuentan también con un ventilador para acelerar el proceso de refrigeración, pero no se va a utilizar en el transcurso de este proyecto, donde únicamente se controlará la potencia suministrada a las bombillas. Esto hace que el sistema sea más lento, pues tardará más tiempo en reducirse la temperatura, pero permite trabajar con una única señal de control. En la figura 2.8 se puede observar la propia planta de temperatura en funcionamiento.

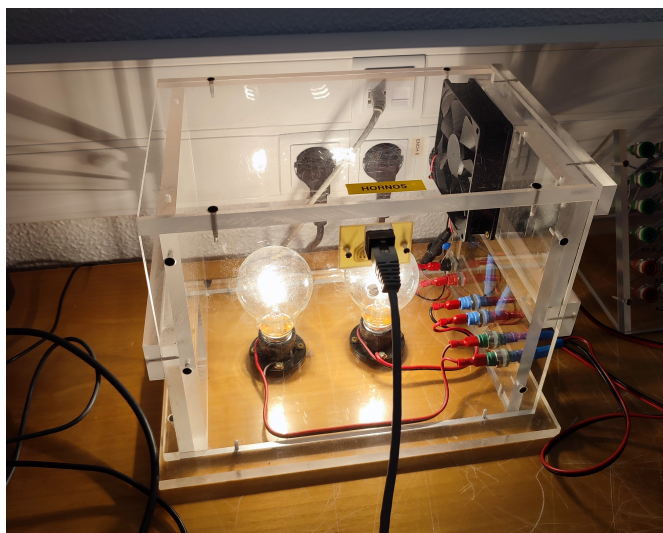


Figura 2.8: Planta de temperatura en funcionamiento

2.8.3. Actuador

El actuador es el bloque encargado de entregar una potencia $P(s)$ a la planta directamente proporcional al valor de la señal de control recibida. Esta señal de control se transmite de forma diferente en función del modo de control empleado:

- Empleando la DAQ, esta señal de control es una señal de tensión de 0-10V generada por la tarjeta (Controlador 1 en la figura 2.9).
- Empleando el control a través de Node-Red, el controlador envía directamente una señal de control en forma de valor entero en un rango de 0 a 100 a través de MQTT que refleja el % de potencia que se ha de entregar a la planta y que se transmite al circuito actuador (Controlador 2 en la figura 2.9).

Realmente, en ambos casos la señal de control es en última instancia un valor entero en el rango de 0-100, pues aunque estemos realizando la adquisición de datos a través de la TAD, la señal de 10V que esta genera es acondicionada y adquirida por el convertidor A/D que obtiene, a partir de esta tensión, un valor entero en este rango.

Este número entero es el índice que se va a usar para tomar de un array de 101 valores (desde índice 0 hasta 100) el valor del conteo a ejecutar desde el paso por cero de la señal de tensión de red hasta el disparo de un triac, que cierra el circuito permitiendo que las bombillas incandescentes de la planta sean alimentadas a 220V directamente desde la red.

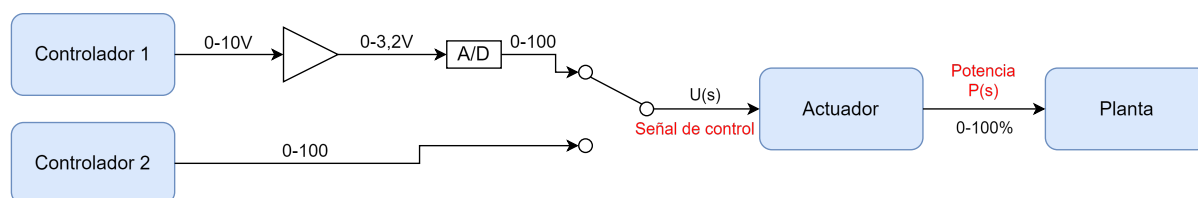


Figura 2.9: Bloque actuador

De esta forma, se manipula la potencia entregada a las plantas controlando durante cuánto tiempo reciben alimentación las bombillas y durante cuánto tiempo se abre el circuito dejando sin alimentación las bombillas. Tomando los 10 milisegundos que dura un semiciclo de la señal de red, si se alimenta la carga durante los 10 milisegundos se entrega una potencia del 100 %, mientras que, en el caso opuesto, si se alimenta la carga durante 0 de esos 10 milisegundos, la carga no recibe potencia. Por ejemplo, si queremos entregar una potencia del 50 % a la planta se ha de alimentar la carga durante 5 ms (el 50 % del tiempo total). Hay que tener en cuenta que esta relación entre tiempo y potencia no siempre es lineal, de forma que, en caso de que queramos entregar a la planta un 30 % de la potencia total, el tiempo durante el que debemos alimentar la carga no es necesariamente el 30 % del tiempo total (3 ms de los 10 milisegundos que dura el semiciclo).

Esto se explica detalladamente en la sección 2.8.5.3, relativa al regulador.

Físicamente, el equipo electrónico que funciona como actuador cuenta con los siguientes bloques diferenciados:

- **Circuito detector de paso por cero.** Genera un pulso a nivel alto (3,3V) durante un semiciclo de la señal de la red de 50 Hz, y un pulso a nivel bajo (0V) durante el otro semiciclo.
- **Circuito de disparo.** A través de un optoacoplador con salida a triac se abre y se cierra el circuito de alimentación de la carga situada en el horno.

- **Circuito de acondicionamiento de la señal de control.** Adapta la señal de control generada por TAD al rango de entrada del microcontrolador. Este circuito únicamente se emplea si se utiliza el control mediante DAQ, pues si se emplea el control mediante Node-Red la señal de control se envía directamente al micro como un valor entero, no como una señal eléctrica.
- **Circuito de alimentaciones.** Genera las tensiones continuas necesarias para alimentar los componentes que conforman el resto de los circuitos.

2.8.3.1. Circuito detector de paso por cero

Para definir el instante en el que se ha de disparar el triac para entregar a la planta la potencia deseada se ha de tomar una referencia de tiempos (o de ángulos). Esta referencia será el paso por cero de la señal senoidal de la red. Para poder detectar este instante, se implementa el circuito detector de paso por cero.

El funcionamiento de este circuito se basa en la comparación de la señal de red con la masa del circuito (0V) a través de un comparador LM311.

En primer lugar, se utiliza un transformador con toma intermedia en el secundario para obtener, a partir de la señal de red, una señal senoidal de la misma frecuencia y menor amplitud. El transformador empleado es de 220V en el primario y 12+12V en el secundario. De esta forma, a la salida del transformador tenemos una señal senoidal de 50 Hz en fase con la red y con un valor de tensión eficaz de 12V (aproximadamente 17V de pico).

Como no es necesario trabajar con amplitudes tan grandes se coloca un divisor de tensión antes de la entrada no inversora para atenuar esta señal y trabajar con tensiones inferiores a las de la alimentación del comparador. Con los valores de resistencias que se observan en la figura 2.10 la amplitud de la señal es de aproximadamente 2,2V

El LM311 se alimenta entre +12V y 0V, de forma que debemos asegurar que la tensión de entrada al comparador está siempre comprendida entre estos dos valores. Tras el divisor de tensión se emplea un diodo rectificador para evitar la entrada de tensiones negativas al comparador, de forma que durante los semiciclos negativos la entrada no inversora del comparador verá una tensión constante de -0,7V.

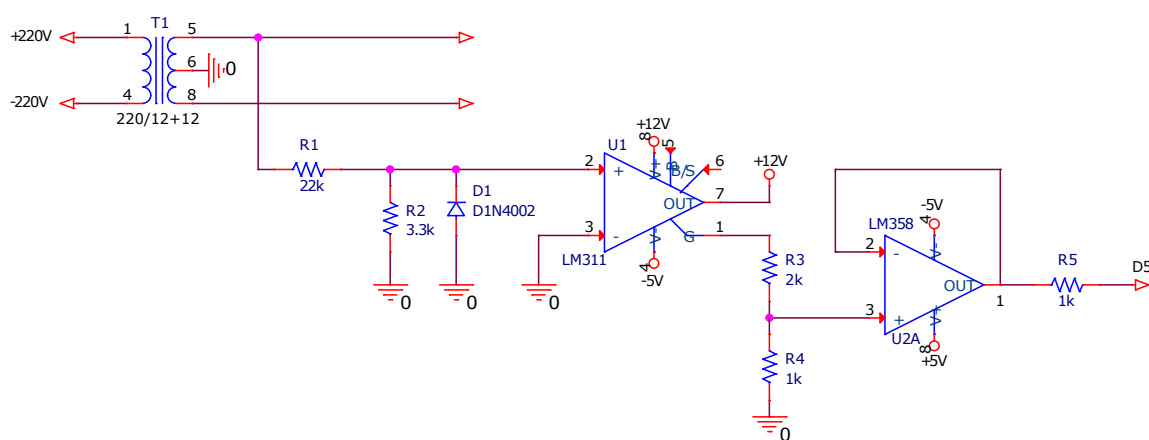


Figura 2.10: Circuito detector de paso por cero

El comparador LM311 posee una salida a transistor, de forma que cuando se aplique una

tensión negativa a la base del transistor, es decir, cuando la tensión de entrada a la patilla inversora sea mayor que la tensión en la patilla no inversora (0V), existe conducción entre el colector (pin 7) y el emisor (pin 1). Es así que durante los semiciclos negativos de la señal de red aparece en el emisor una tensión de 10V (los 12V que se aplican al colector con una caída de tensión de 2V entre colector y emisor) como consecuencia de que el comparador satura a negativo al haber mayor tensión en la patilla inversora que en la no inversora. Esta señal se hace pasar por un divisor de tensión de ganancia 1/3 para obtener una señal cuadrada de 3,3V de amplitud. Tras el divisor de tensión se coloca un seguidor para asegurar que la impedancia de entrada al microcontrolador no afecte al funcionamiento del circuito y una resistencia R_5 para limitar la corriente.

De esta forma, el circuito entrega al microcontrolador una señal a nivel bajo (0V) durante los semiciclos positivos de la tensión de red y una señal a nivel alto (3,3V) durante los semiciclos negativos de la tensión de red. Esta señal se lleva a la entrada digital D5 del controlador y se utilizará la detección de estos flancos para tomar una referencia para el conteo del tiempo a transcurrir hasta el disparo del triac.

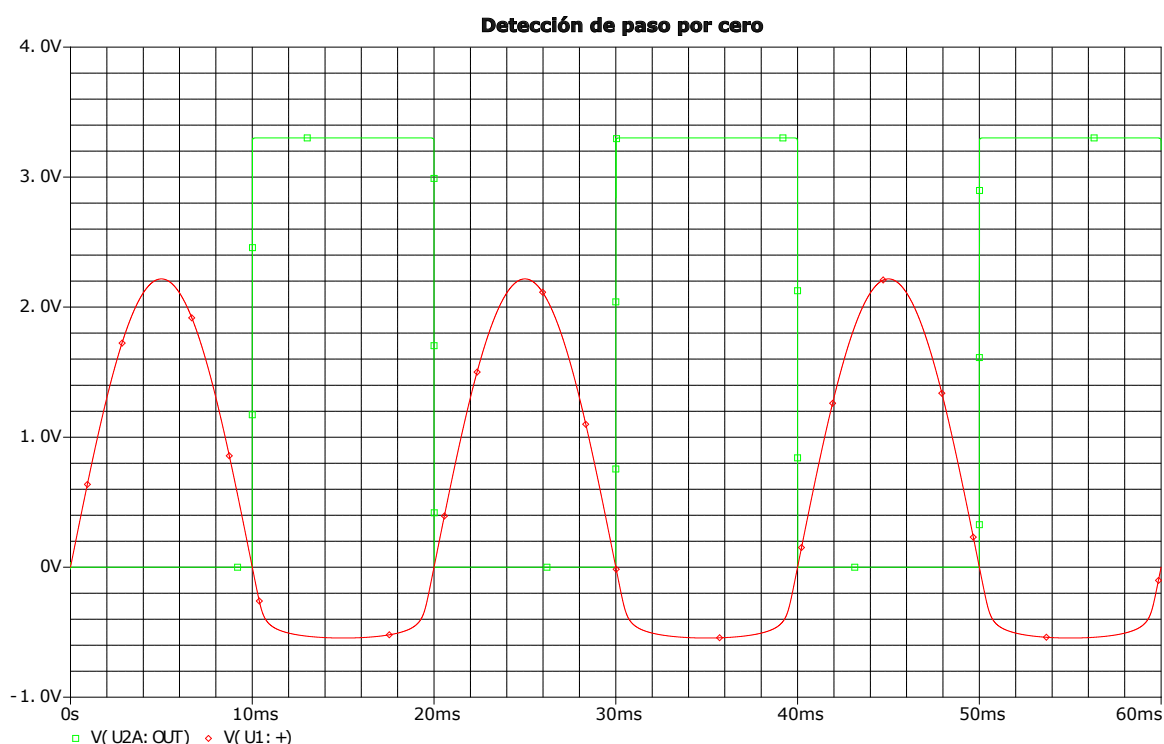


Figura 2.11: **Verde:** Señal de entrada no inversora del LM311

Rojo: Señal de salida del circuito

Simulación del circuito detector de paso por cero

En la figura 2.11 podemos observar la señal de salida del circuito detector de paso por cero frente a la señal de entrada a la patilla no inversora del LM311, en fase con la tensión de red. En este caso la señal cuadrada está a nivel alto en los semiciclos negativos, pero podríamos hacer que se activara a nivel alto en los semiciclos positivos, pues se van a usar tanto los flancos ascendentes como los descendentes para iniciar un conteo cada 10 milisegundos, de forma que estos flancos siempre coinciden en el mismo instante, si bien varía el tipo de flanco

(ascendente/descendente).

2.8.3.2. Circuito de disparo

El circuito de disparo es el encargado de abrir y cerrar el circuito de potencia que alimenta la planta. Esto se consigue mediante el disparo de un triac en un instante concreto calculado a partir del paso por cero de la tensión de red.

En la sección 2.8.5.1 se expone detalladamente la forma en la que el microcontrolador realiza el conteo para determinar en que instante se ha de disparar el triac, pues aquí únicamente se expone el diseño del circuito de disparo, no los criterios para determinar el instante temporal de dicho disparo.

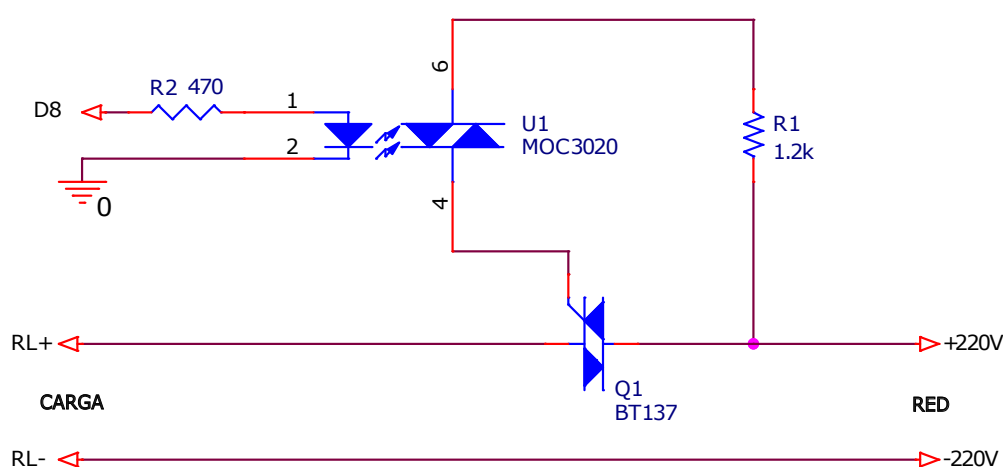


Figura 2.12: Circuito de disparo

En la figura 2.12 podemos ver el esquemático del circuito. Se utiliza la salida digital D8 del controlador para cerrar el circuito una vez que el micro haya finalizado el conteo del tiempo a esperar tras el paso por cero. En este momento, la salida D8 se activa a nivel alto durante 200 microsegundos, de forma que aparece una tensión positiva en el pin 1 del MOC3020.

El MOC3020 es un optoacoplador con salida a triac, que permite la conducción entre sus terminales 4 y 6 cuando la tensión en el ánodo del emisor de infrarrojos (terminal 1) es mayor que en el cátodo (terminal 2).

Como el cátodo está conectado a masa, cuando apliquemos cualquier tensión positiva a través de la salida D8 se disparará el circuito. Esta idealización no se cumple en realidad debido a que existe una pequeña caída de tensión en la resistencia interna del diodo cuyo valor típico es de 1,2V. Como la tensión de salida a nivel alto que ofrecen las salidas digitales del microcontrolador es de 3,3V esta caída de tensión no provoca variaciones en el funcionamiento del circuito. Para limitar la corriente por el optoacoplador se emplea la resistencia R_2 (figura 2.12).

El terminal 4 del MOC3020 está conectado directamente a la base de un triac BT137, de forma que cuando dispare el MOC3020 disparará también el BT137, alimentando la carga a $\pm 220V$.

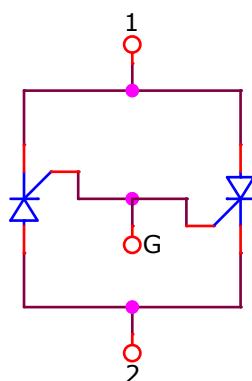


Figura 2.13: Modelo equivalente del triac BT137

El modelo equivalente del triac BT137 se muestra en la figura 2.13. Se puede representar como dos tiristores en paralelo con la polaridad invertida, cuyas puertas están conectadas, de forma que ambas partes disparan en el mismo instante, una conduce en los semiciclos negativos de la tensión de red y otra en los positivos.



Figura 2.14: **Verde:** Tensión suministrada a la carga
Rojo: Tensión en la base del optotriac (pin D8 del micro)
 Simulación del circuito de disparo

La figura 2.14 muestra una simulación de la respuesta del circuito de disparo al activarse la salida digital D8 del microcontrolador. Para realizar esta simulación se ha supuesto que el microcontrolador dispara el triac a los 3 milisegundos tras el paso cero, de forma que se estaría entregando un 85 % de la potencia máxima, como se explica en la sección "" (??).

2.8.3.3. Circuito de acondicionamiento de la señal de control

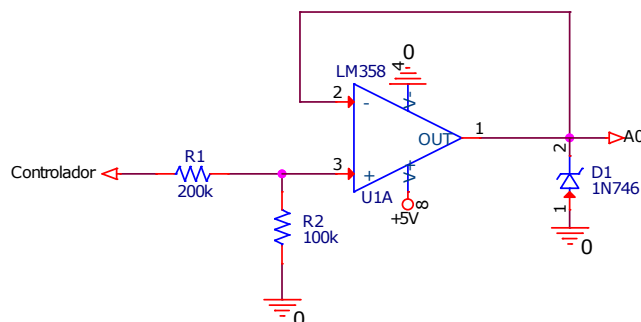


Figura 2.15: Circuito de acondicionamiento de la señal de control

En la figura 2.15 se muestra el esquemático del circuito de acondicionamiento de la señal de control.

La señal de control generada por el equipo de adquisición de datos, en un rango de 0 a 10 voltios, se hace pasar por un divisor de tensión resistivo para reducir su amplitud $\frac{1}{3}$, hasta un máximo de 3,3V. Se utiliza un amplificador operacional LM358 para implementar un seguidor de tensión que haga que esta señal de control de 0 a 3,3 voltios no se vea alterada por impedancias posteriores. A la salida del LM358 se coloca un diodo zener 1N746 ($V_z = 3,3V$), de forma que se limita la tensión a la salida del operacional al rango deseado para evitar que lleguen a la entrada A0 del microcontrolador tensiones demasiado elevadas o negativas que el micro no pueda reconocer. En este caso no se limita la corriente de salida del amplificador con una resistencia como se hace en la mayoría de circuitos anteriores ya que entre la entrada A0 del microcontrolador y la entrada al convertidor A/D a la que ataca esta entrada analógica existe un divisor resistivo con la finalidad de adaptar la tensión al rango de entrada del A/D, y cuya ganancia se vería modificada en caso de colocar una resistencia antes de la entrada para limitar la corriente, reduciendo la precisión del sistema de forma considerable.

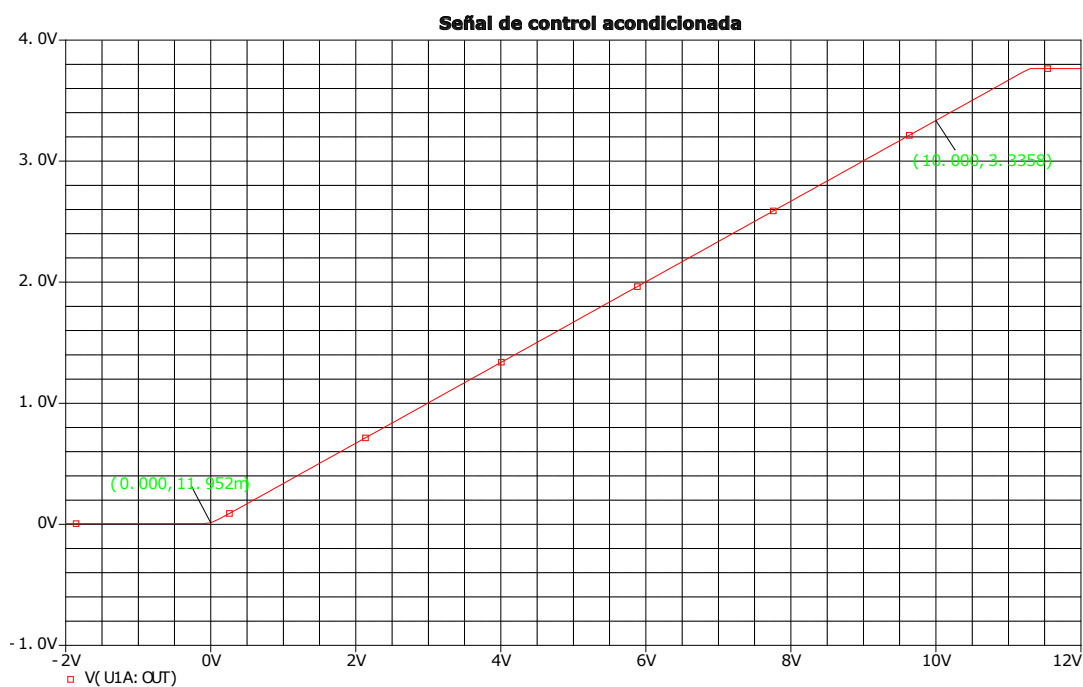


Figura 2.16: Tensión de salida del circuito de acondicionamiento frente a tensión de entrada

En la figura 2.16 podemos ver como la tensión de salida del circuito de acondicionamiento varía linealmente con la tensión de entrada, de forma que se cumple que:

$$V_o = \frac{1}{3} \cdot V_i$$

siendo V_i la tensión de entrada al circuito de acondicionamiento y V_o la señal de salida. Cabe recordar que este circuito de acondicionamiento se emplea únicamente el modo de control que en "Descripción general" se definió como el previamente existente en la planta, donde la adquisición de datos se realiza a través de una tarjeta de adquisición de datos del fabricante National Instruments.

2.8.3.4. Circuito de alimentaciones

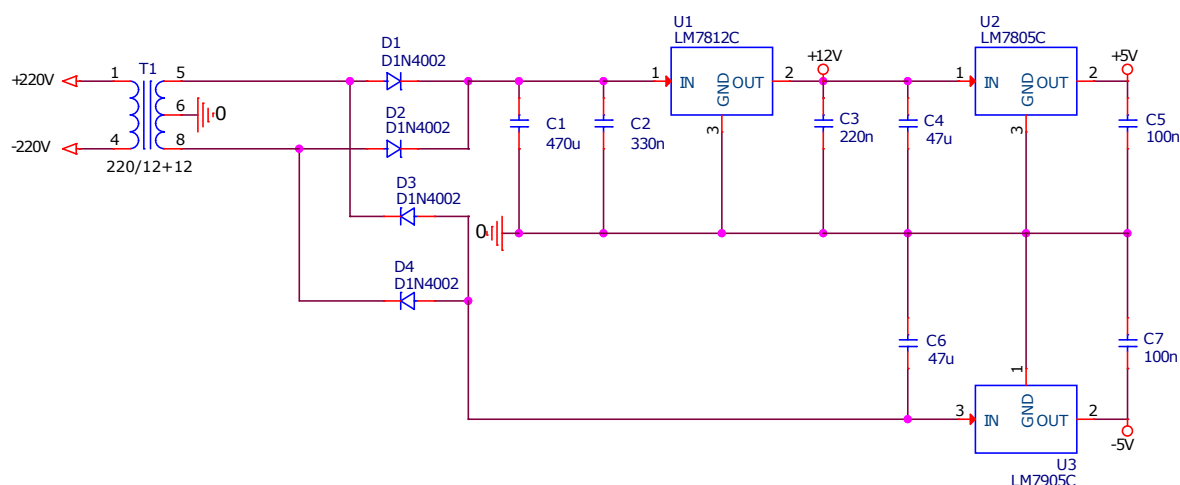


Figura 2.17: Circuito de alimentaciones

Como su nombre indica, el circuito de la figura 2.17 es el encargado de proporcionar las alimentaciones necesarias, de +12, +5 y -5V dentro del circuito de control de potencia.

Como podemos ver en los esquemáticos de los demás circuitos que conforman el actuador, y también en el circuito de acondicionamiento del sensor de temperatura (2.27), todos los componentes están alimentados a estos valores de tensión: +5V, +12V y -5V. Para ello, se utilizarán respectivamente los reguladores de tensión LM7805, LM7812 y LM7905.

Se trata de unos dispositivos de gran precisión y adecuados para este tipo de aplicaciones que, si bien son menos eficientes que los conmutados, se seleccionan porque la electrónica no presenta un consumo elevado. Presentan una tolerancia de la tensión de salida entre el 2 % y el 4 % y operan en un rango de temperatura entre -40°C y 150°C, por lo que incluso aunque el circuito se sitúe próximo a la planta no deberían excederse estos valores de temperatura. Para su correcto funcionamiento requieren una tensión de entrada al menos 3V superior a la de salida. Este hecho es el que condiciona la elección del transformador.

Se opta por un transformador de toma intermedia de 220/12+12V (primario 220V, secundario 12-0-12V) que, a partir de la tensión de red, ofrece dos tensiones alternas de valor eficaz 12V en oposición de fase (desfase de 180°). Para ello, debemos tomar como referencia del circuito la toma intermedia del secundario del transformador.

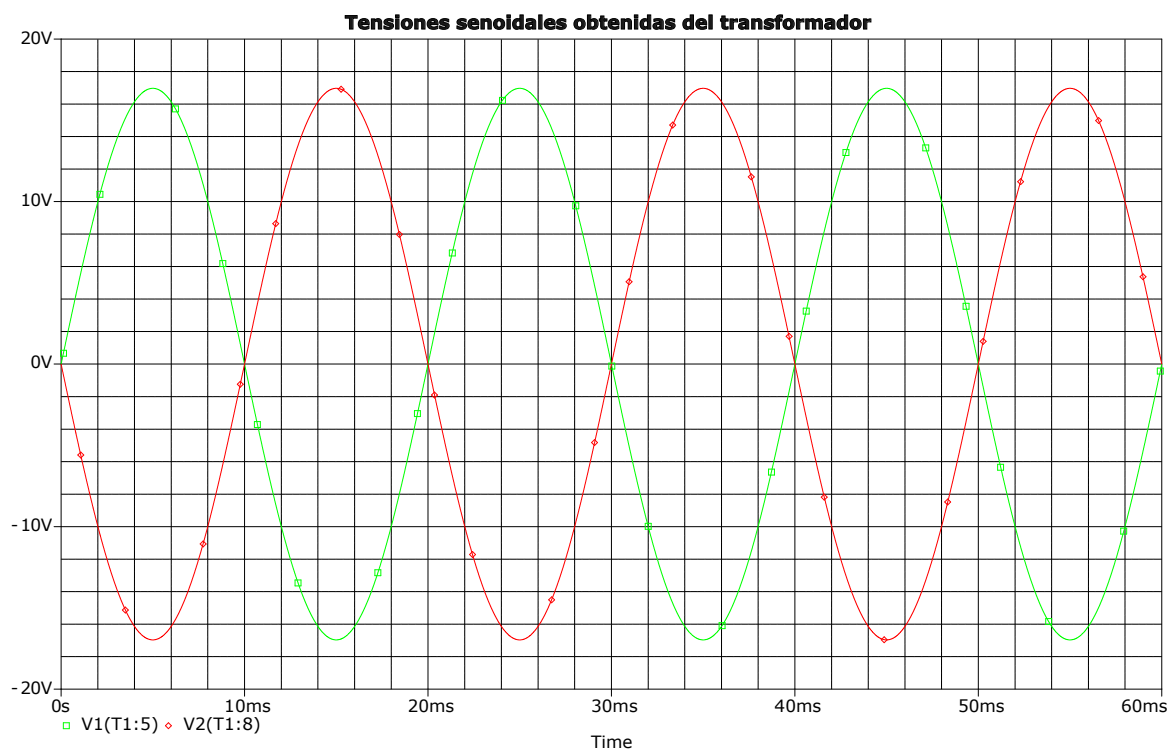


Figura 2.18: Tensiones senoidales obtenidas del transformador

Ambas tensiones son rectificadas mediante dos diodos 1N4002, uno para rectificar los semiciclos positivos y otro para rectificar los semiciclos negativos, por lo que el circuito que se implementa es un rectificador de onda completa (uno para cada tensión senoidal). De esta forma, a la salida del puente de diodos obtenemos dos señales de 100 Hz, tales que:

- $V_1 = 12\sqrt{12} \cdot |\text{sen}(2\pi \cdot 50 \cdot t)|$
- $V_2 = -12\sqrt{12} \cdot |\text{sen}(2\pi \cdot 50 \cdot t)|$

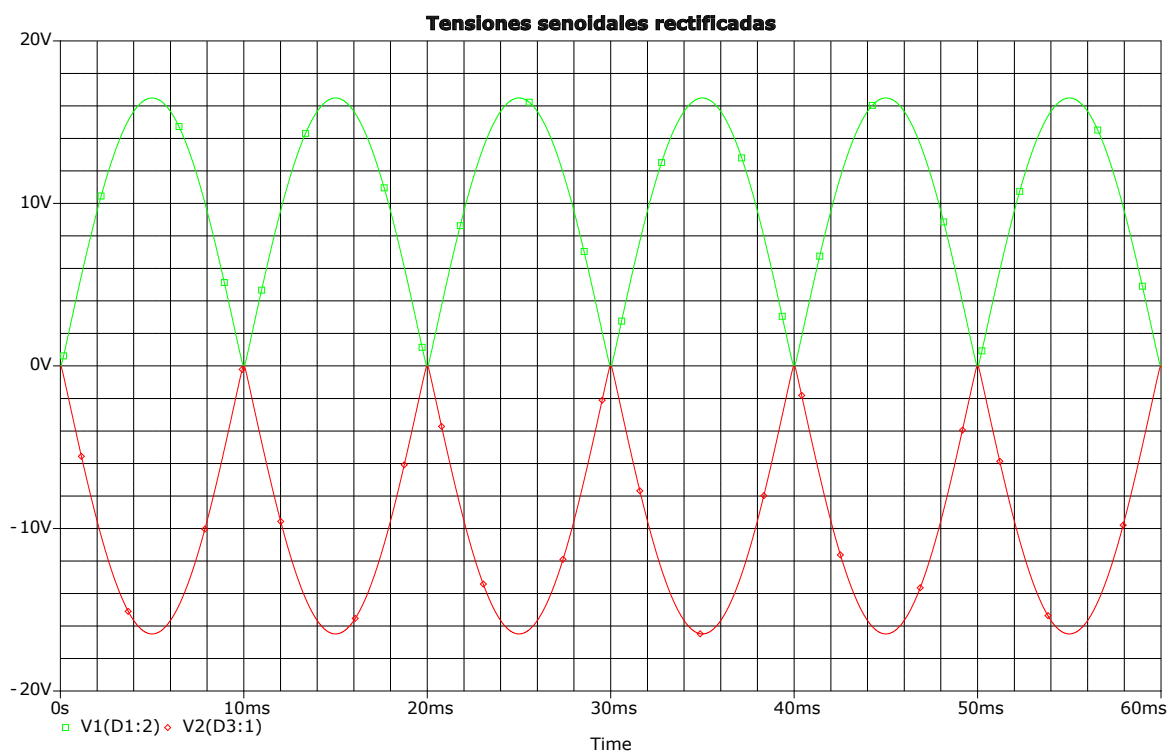


Figura 2.19: Tensiones senoidales rectificadas

La señal V_1 se filtra a través de los condensadores C_1 y C_2 , de forma que a la entrada del LM7812 existe una tensión prácticamente continua de aproximadamente 17V ($12 \cdot \sqrt{2}$ V), que es el valor de pico de la señal. Como esta tensión es superior a la señal de 12V que queremos obtener en más de 3V (unos 5 voltios aproximadamente), obtenemos a la salida una tensión continua de 12V regulada.

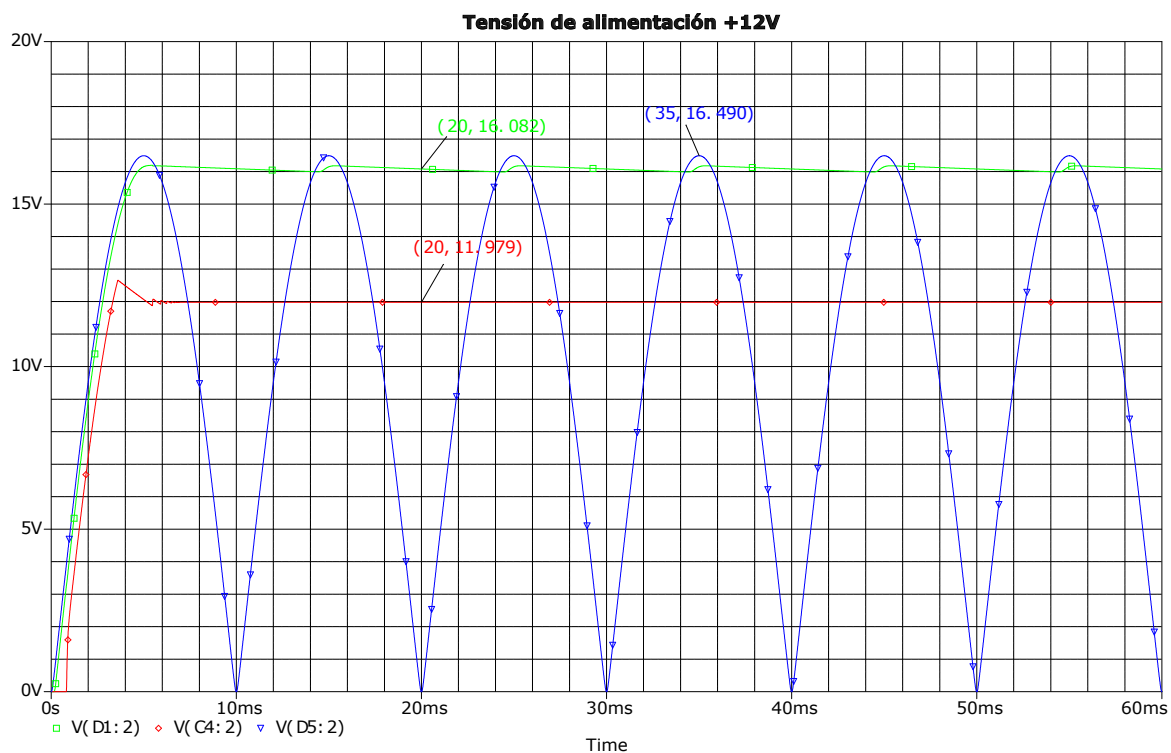


Figura 2.20: **Azul:** Señal senoidal rectificada
Verde: Señal senoidal rectificada y filtrada
Rojo: Tensión continua de 12V a la salida del LM7812

Esta tensión continua de 12V es la que se utiliza como tensión de entrada del LM7805 para obtener a partir de ella una tensión continua y estable de +5V.

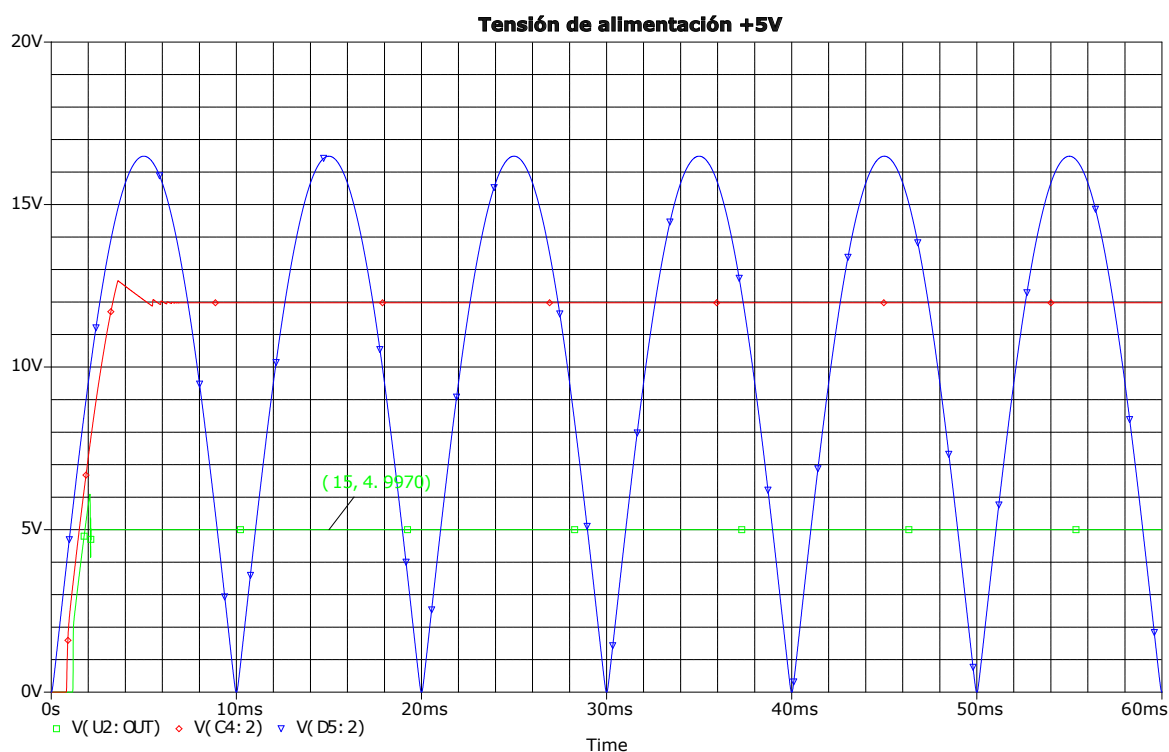


Figura 2.21: **Azul:** Señal senoidal rectificada
Rojo: Tensión continua de 12V a la entrada del LM7805
Verde: Tensión continua de 5V a la salida del LM7805

Para obtener la tensión continua de -5V se sigue un procedimiento parecido al que se utiliza para obtener la tensión de 12V. Se parte de la señal V_2 , la señal rectificada de semiciclos negativos, y se filtra a través del condensador C_6 para obtener una señal que se asemeje a una tensión continua de -17V, que se lleva a la entrada del LM7905.

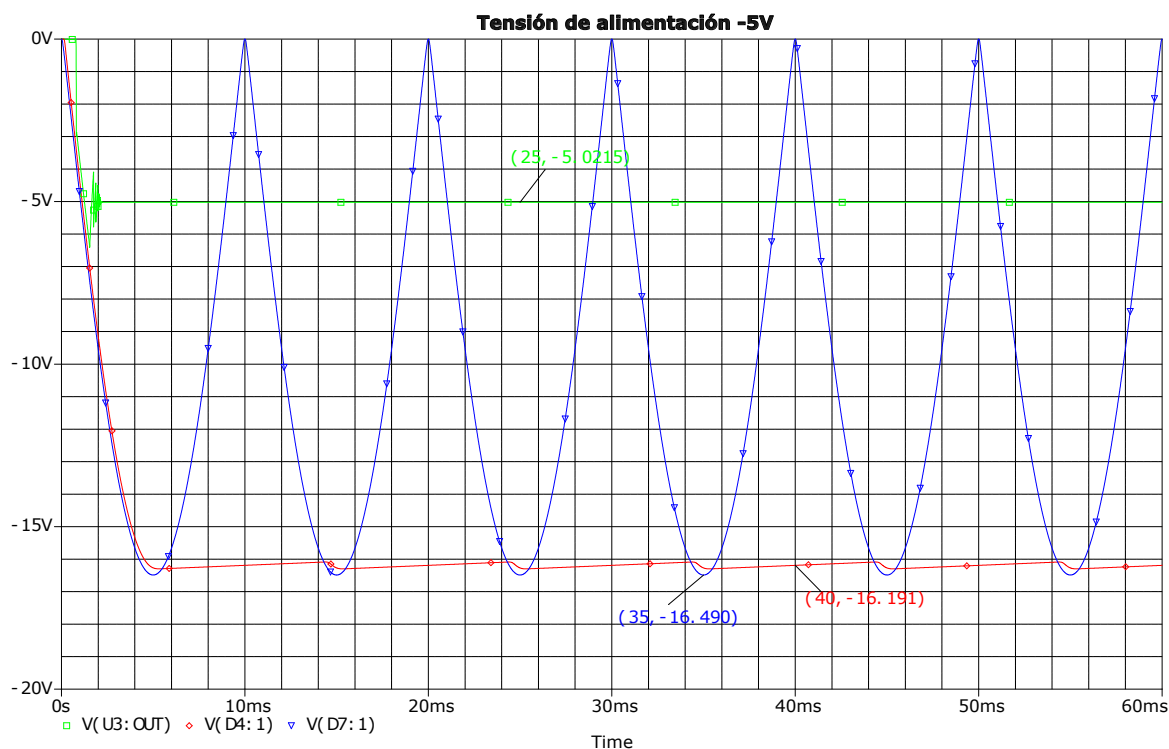


Figura 2.22: **Azul:** Señal senoidal rectificada
Rojo: Señal senoidal rectificada y filtrada
Verde: Tensión continua de -5V a la salida del LM7905

2.8.4. Sensor

Para poder realimentar el lazo de control es necesario contar con la medida de la magnitud de salida de la planta. En este caso, como el objetivo es regular la temperatura de la planta, se emplazará un sensor de temperatura en el interior de la misma.

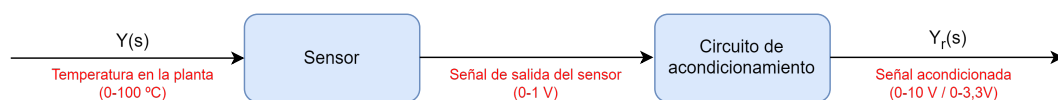
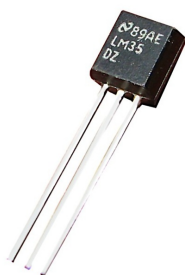


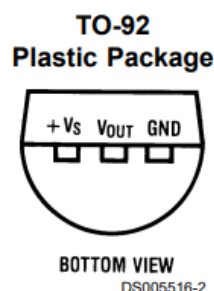
Figura 2.23: Bloque de realimentación

Aunque en la *Descripción general* (2.8.1) se haya representado la realimentación como un único bloque, la realidad es que la señal generada por el sensor de temperatura debe ser acondicionada para adaptarla al rango deseado.

El sensor empleado es el LM35 con encapsulado de tipo TO-92 de tres pines, de la casa National Semiconductor.



(a) Encapsulado real



(b) Esquemático encapsulado TO-92

Figura 2.24: Sensor LM35

De acuerdo con la hoja de características proporcionada por el fabricante, las principales características del sensor son:

- Calibración directa en grados Celsius.
- Sensibilidad lineal de $+10 \text{ mV}/^{\circ}\text{C}$.
- Exactitud garantizada de $0,5^{\circ}\text{C}$ a 25°C .
- Rango de medida de -55°C a $+150^{\circ}\text{C}$.
- Alimentación entre 4 y 30 voltios.
- Consumo de corriente inferior a $60 \mu\text{A}$.
- Bajo autocalentamiento, de $0,08^{\circ}\text{C}$ en condiciones normales.
- Alta linealidad, con error de no linealidad máximo de $\pm \frac{1}{4}^{\circ}\text{C}$ típicamente.
- Baja impedancia de salida, de $0,1 \Omega$ para una carga de 1 mA .

El hecho de que el sensor venga calibrado para ofrecer una tensión de salida de 0V cuando la temperatura medida es de 0°C simplifica enormemente la etapa de acondicionamiento, pues únicamente será necesario aplicar una etapa de ganancia.

Además, el sensor presenta buenas características en lo referente a exactitud, consumo o auto-calentamiento, y cuenta con un rango de funcionamiento adecuado para esta aplicación, donde se trabajará con temperaturas inferiores a 100°C .

En los equipos del laboratorio de optimización y control este sensor LM35 se encuentra integrado en una placa de circuito impreso para su conexión remota. Esta placa cuenta con 8 pines hembra para la colocación del sensor y un conector RJ45 (Registered Jack 45), pues la conexión se va a realizar a través de cable modular RJ45. Este es el motivo por el que se disponen 8 pines en la placa, si bien el sensor únicamente utiliza 3 (tensión de alimentación, tensión de salida y masa), pues el cable RJ45 cuenta con 8 hilos de par trenzado, cada uno

de los cuales se conecta a través del conector RJ45 a cada uno de los pines de la placa. El sensor estará alimentado a +12V a través del circuito de alimentaciones.

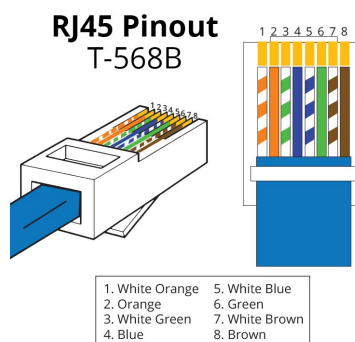


Figura 2.25: Disposición de los pines en el extremo del cable modular RJ45

Los hilos que se utilizan son los que aparecen referenciados en la figura 2.25 como:

- **1. Naranja/blanco**, para la conexión de la masa del circuito.
- **2. Naranja**, para la transmisión de la señal generada por el sensor.
- **3. Verde/blanco**, para alimentar el sensor a +12V.

Estos se disponen en la placa alojada dentro de la planta tal como se muestra en la figura 2.26.

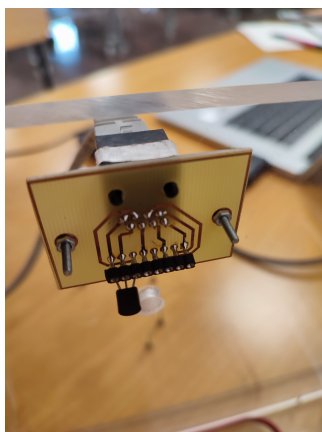


Figura 2.26: Placa para la conexión del sensor de temperatura

Al igual que en la placa del sensor situada en el interior del horno, la placa de prototipo sobre la que se realiza el montaje contará con otra tira de 8 pines donde se conectarán los 8 hilos de par trenzado del cable RJ45, 3 de los cuales se utilizan para transmitir las señales del sensor.

2.8.4.1. Circuito de acondicionamiento

El sensor presenta una salida en tensión nula para una medida de temperatura de 0°C , que es el extremo inferior de nuestro rango de medida. La sensibilidad que presenta el sensor es de $+10\text{ mV}/^{\circ}\text{C}$, por lo que, para el extremo superior del rango de medida ($+100^{\circ}\text{C}$), la tensión de salida es de 1V. Esta señal va a ser procesada por un equipo de adquisición de datos, por lo que es necesario adaptarla al rango de tensiones de entrada que maneja el equipo de adquisición de datos.

Según el modo de control que utilicemos (TAD o Raspberry Pi), este rango de tensiones deseado será diferente, por lo que existirán dos circuitos de acondicionamiento diferentes para adaptar la tensión generada por el sensor a ambos rangos de funcionamiento:

- La TAD debe recibir esta señal de temperatura en un rango de 0 a 10 voltios, por lo que la etapa de amplificación del circuito de acondicionamiento debe tener ganancia 10.
- Si trabajamos con la Raspberry Pi, la señal de temperatura va directamente al microcontrolador en vez de a la tarjeta de adquisición de datos. La entrada analógica del micro (A0) acepta tensiones de entrada de hasta 3,2V.

El circuito de acondicionamiento empleado en cada caso se representa en la figura 2.27.

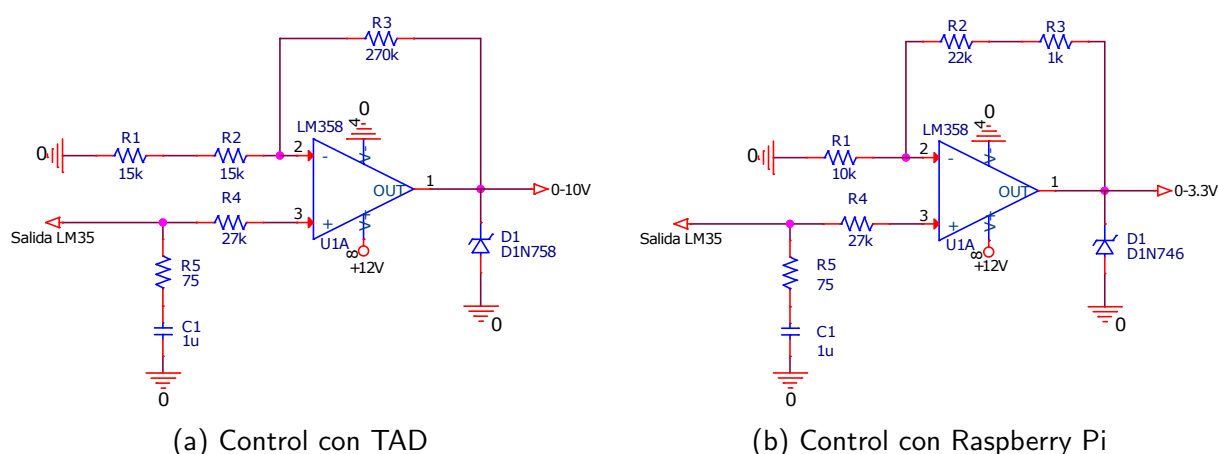


Figura 2.27: Circuito de acondicionamiento de la señal del sensor LM35

Se trata de un circuito amplificador no inversor que aplica una ganancia a la señal del sensor. Para implementar este no inversor se utiliza un amplificador operacional LM358 alimentado de forma unipolar entre 0 y 12 voltios. No es necesaria una alimentación bipolar ya que en el rango de trabajo nunca vamos a trabajar con señales de tensión negativas.

Por recomendación del fabricante, es recomendable filtrar la señal por posibles interferencias o ruido eléctrico antes de la etapa de amplificación, mediante una resistencia de 75Ω (R5) y un condensador de $1\mu\text{F}$ (C1).

A la salida del operacional se coloca un diodo zener para proteger el equipo de adquisición de datos evitando sobretensiones que superen los 10V (el diodo zener empleado es el 1N758, $V_z = 10\text{V}$) en caso del circuito 2.27a y superiores a 3,3V (el diodo zener empleado es el 1N746, $V_z = 3.3\text{V}$) en el circuito 2.27b. No se limita la salida por debajo de los 0V ya que el amplificador operacional LM358 está alimentado de forma unipolar, imposibilitando que aparezcan tensiones negativas a la salida.

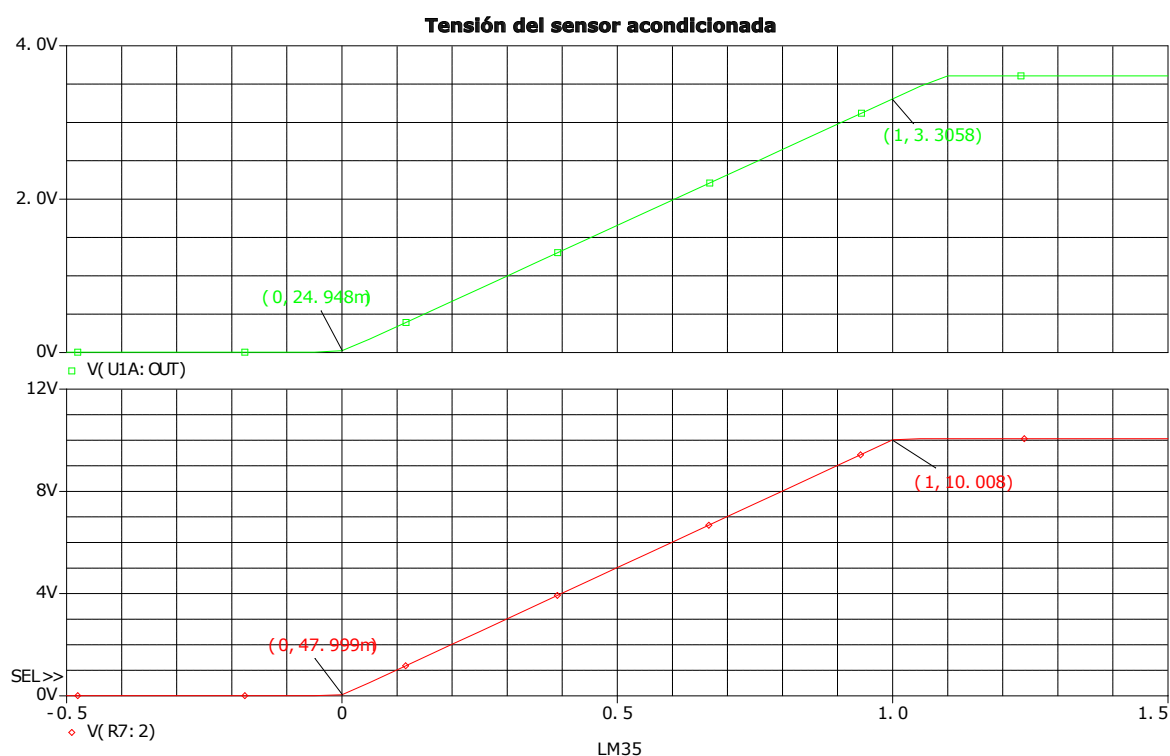


Figura 2.28: **Verde:** Salida del sensor a condicionada en el rango 0-3.3V
Rojo: Salida del sensor acondicionada en el rango 0-10V
 Tensión de salida del circuito de acondicionamiento frente a tensión proporcionada por el sensor LM35 (figura 2.27)

2.8.5. Software

En esta sección se explica detalladamente el proceso de control mediante WiFi empleando Node-Red para implementar el algoritmo del controlador PID y Raspberry Pi para alojar este servicio así como el servidor MQTT que nos permitirá realizar la comunicación, como se explica posteriormente en esta misma sección.

El microcontrolador que se emplea es el ESP8266, implementado en una placa de desarrollo Wemos D1, cuyo funcionamiento es similar al de un Arduino Uno pero permite mayores opciones de comunicación, como WiFi, que es la opción que se va a emplear para el desarrollo de este proyecto.

2.8.5.1. Adquisición de datos y control de potencia con Wemos D1

Wemos D1 (figura 2.29) es una tarjeta de desarrollo similar a Arduino, especialmente orientada al Internet de las cosas (IoT). Está basado en el SoC (System on Chip) ESP8266. Algunas de sus principales características son:

- Tensión de alimentación de 5V (CC)
- Tensión de entrada/salida digital de 3,3V (CC)

- Frecuencia de reloj de 80 MHz
- 32 kB de memoria RAM de programa
- 96 kB de memoria RAM de datos
- 4 MB de memoria Flash
- 11 entradas/salidas digitales (GPIO)
- 1 entrada analógica (ADC de 10 bits, hasta 1V)
- SoC ESP8266



Figura 2.29: Placa de desarrollo Wemos D1

Se selecciona esta placa principalmente por la capacidad de comunicación vía WiFi gracias al módulo ESP8266. Además, permite ser programado mediante el IDE de Arduino al instalar el plugin de ESP8266, lo que supone una gran facilidad. La Wemos cumple dos funciones dentro del sistema de control: la adquisición de la temperatura de la planta y el control del circuito de disparo.

La Wemos recibe, a través de la entrada analógica A0, la señal de temperatura procedente del circuito de acondicionamiento del sensor, convierte esta señal de tensión a un valor entero de temperatura en grados centígrados y lo transmite (publica) al servidor MQTT. El algoritmo de control de Node-Red ubicado en la Raspberry Pi lee (suscribe) este valor del servidor y calcula la señal de control, que envía también al servidor MQTT, de donde la Wemos tomará el valor. La Wemos toma este valor de la señal de control, cuyo valor está en el rango 0-100, y lo utiliza como índice para tomar del array **tiempo[]** el valor entero a cargar en el Timer1 que permita realizar la temporización correspondiente. Estos valores de temporización para cada nivel de potencia se calculan y recogen en el anexo 3.3.4.

Como se expuso en la sección *Circuito de disparo*, el objetivo de detectar el paso por cero de la señal de red es emplear ese instante para iniciar una cuenta en el microcontrolador. Cuando esta cuenta se termina, se activa en el pin digital D8 un pulso de 3,3 voltios durante 200 microsegundos.

Teniendo en cuenta lo anterior, el sistema de temporización funciona de la siguiente forma:

1. El microcontrolador lee el paso por cero a través de la entrada digital D5.
2. Cada vez que se produce un paso por cero, se inicia una temporización. El valor de tiempo que debe durar la cuenta vendrá determinado por el porcentaje de potencia que se quiera entregar a la carga, de forma que cuánta más potencia se quiera suministrar, menos debe durar esta cuenta, pues antes se ha de disparar el triac que cierra el circuito.
3. Una vez que se finalice la cuenta, se activa a nivel alto la salida digital D8 que va al circuito de disparo durante $200\ \mu s$, cerrando el circuito y haciendo que la carga reciba alimentación.

El programa a implementar en la Wemos se realiza en IDE de Arduino, utilizando el plugin específico para placas basadas en ESP8266 y las librerías *ESP8266WiFi* y *PubSubClient*, que permiten utilizar instrucciones específicas para la comunicación por WiFi utilizando el protocolo MQTT. El código del programa se recoge en el anexo 3.4.1, y su lógica se recoge en el diagrama de flujo de la figura 2.30.

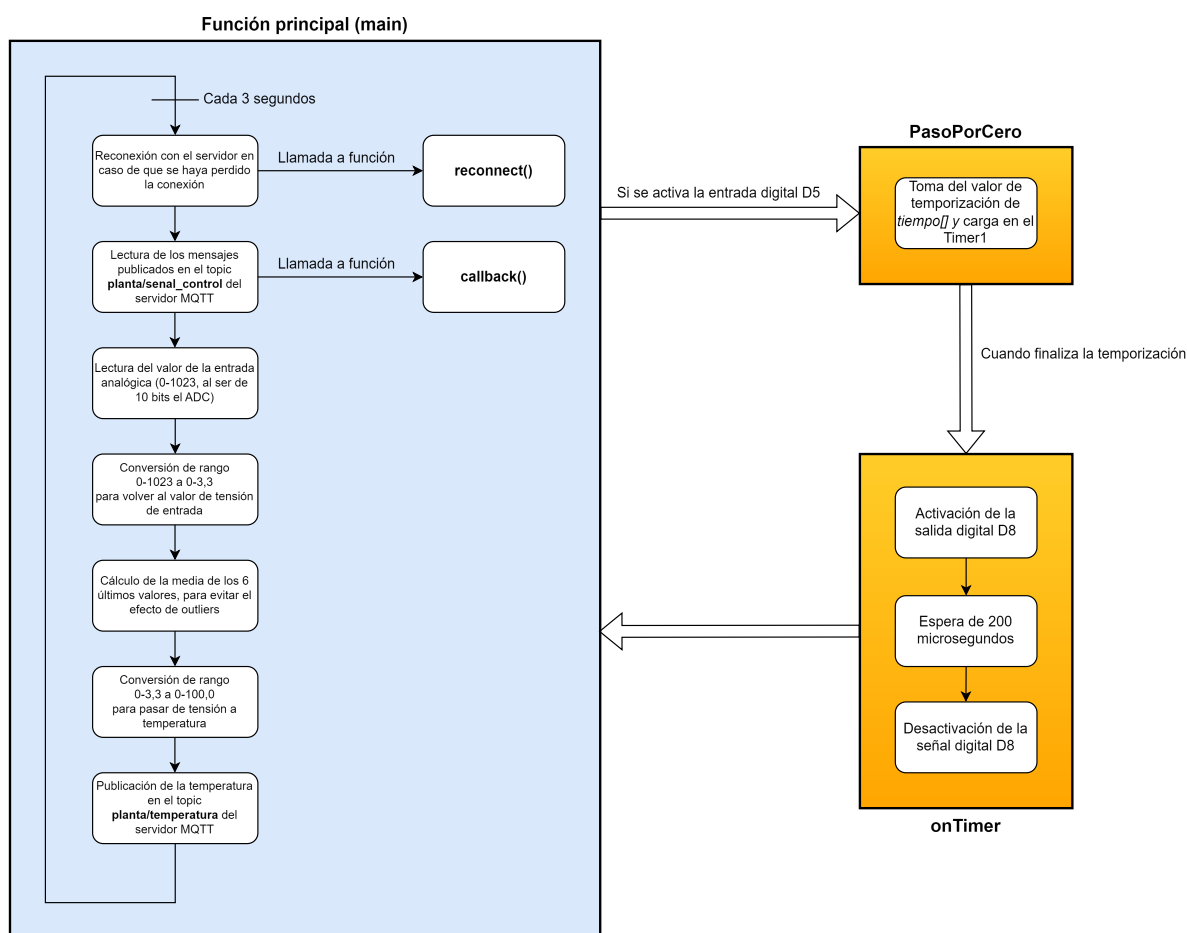


Figura 2.30: Esquema del funcionamiento del programa de la Wemos

Dentro de este programa, la tarea que se ejecuta de forma cíclica es la medida de la temperatura y la lectura de las señales recibidas por el servidor MQTT, mientras que la activación de la señal de disparo se gestionan a través de interrupciones:

1. Cuando se detecta el paso por cero de la señal de red, se ejecuta la función de interrupción **PasoPoCero()**, donde se inicia la temporización del Timer1 cargándole el valor correspondiente almacenado en el array **tiempo[]**.
2. Una vez que finaliza la temporización, se ejecuta la interrupción **onTimer()**, activando la salida digital D8 durante 200 μs y disparando el triac

El programa se ejecuta de forma cíclica cada 3 segundos, de forma que el programa se queda detenido al alcanzar el final hasta que se cumpla este tiempo, procesando únicamente las interrupciones. De esta forma, se garantiza que el tiempo transcurrido entre dos lecturas de temperatura consecutivas sea siempre el mismo. Se selecciona un tiempo de muestreo de 3 segundos ya que el sistema es lento y no es necesario muestrear con frecuencias superiores, que además provocarían mayor acumulación de mensajes en el servidor MQTT pudiendo ocasionar retardos considerables.

El tiempo de activación de la señal digital *D8*, que hace disparar el triac, se selecciona de forma arbitraria teniendo en cuenta las características eléctricas del triac BT137. Según la hoja de características del BT137, el tiempo mínimo durante el que se ha de aplicar tensión a la base del triac para dispararlo es de 2 μs . Se ha de seleccionar un tiempo ampliamente superior para evitar fallos por un deterioro en la respuesta del componente electrónico o por una imprecisión en la temporización del micro, pero cabe tener en cuenta que el programa queda detenido mientras se espera para volver a desactivar la salida *D8*, por lo que un tiempo demasiado elevado podría generar retrasos en la ejecución del programa y un funcionamiento inadecuado.

2.8.5.2. Servidor MQTT con Raspberry Pi

Como se ha venido indicando hasta ahora, el protocolo empleado para comunicar los diferentes elementos del sistema de control es MQTT (Message Queuing Telemetry Transport). Es un protocolo de comunicación M2M (machine-to-machine) basado en TCP/IP. MQTT funciona como un servicio de mensajería con patrón publicador/suscriptor (pub-sub). En este tipo de infraestructuras, los clientes se conectan con un servidor central o **broker**, por el que pasan todos los mensajes. Para organizar estos mensajes de una forma jerárquica se dispone de **topics**, que son etiquetas asociadas a los mensajes. Un cliente puede publicar un mensaje en un determinado topic. Otros clientes pueden suscribirse a este topic, y el broker les hará llegar los mensajes suscritos. De esta forma, cada cliente recibe únicamente los mensajes asociados a aquellos topics que requiere.

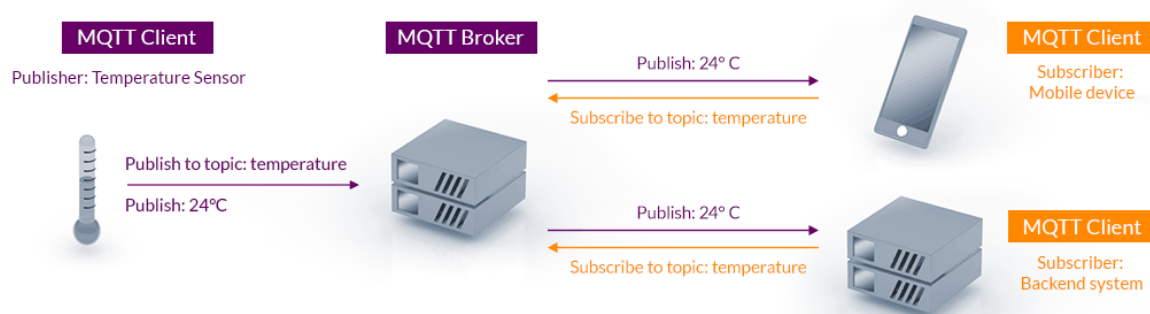


Figura 2.31: Ejemplo para comprender el funcionamiento de MQTT

Para alojar el servidor MQTT empleado en el proyecto, se va a utilizar la placa de microprocesador Raspberry Pi 3 Model B.

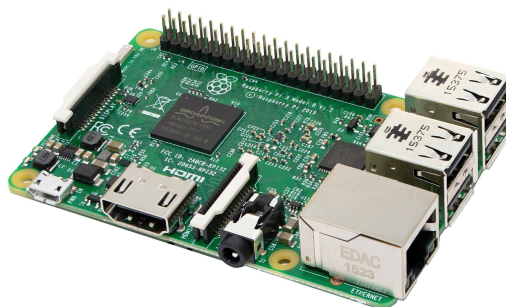


Figura 2.32: Raspberry Pi 3 Model B

Las Raspberry Pi son ordenadores de bajo coste que funcionan normalmente sobre sistemas operativos basados en Linux. Raspberry Pi no cuenta con memoria no volátil interna, por lo que se ha de utilizar una tarjeta microSD para almacenar el sistema operativo. Para la realización de este proyecto se emplea Raspberry Pi OS, el sistema operativo desarrollado y recomendado por el fabricante. Anteriormente, este sistema operativo era conocido como Raspbian, y básicamente es una versión personalizada de Debian.

El funcionamiento es el mismo que un ordenador de sobremesa, ya que a nivel fundamental tiene los mismos componentes. Las placas Raspberry Pi se basan en un SoC de arquitectura ARM de bajo consumo y buen rendimiento. Se acompañan por un modelo de memoria RAM, cuya capacidad varía según el modelo. Disponen de varias salidas de vídeo y en las versiones más nuevas se agrega un conector jack de 4 polos, para entrada de micro y salida de audio. Cuentan con un lector de tarjetas donde se instala el sistema operativo y varios puertos USB. También dispone de una gran cantidad de conectores GPIO. En este proyecto únicamente se emplea para alojar los servicios de servidor MQTT y Node-Red (sección 2.8.5.3), por lo que no es necesario entrar en demasiados detalles sobre las características hardware del modelo concreto, aunque cabe decir que la selección del modelo de Raspberry sí tiene cierto impacto a la hora de decidir donde ubicar la herramienta **InfluxDB**, para la realización de bases de datos de series temporales.

Las versiones de Raspberry Pi anteriores a la 4 no son compatibles con el cliente web de InfluxDB, por lo que, de instalar InfluxDB sobre la Raspberry, se tendría que trabajar exclusivamente con comandos a través de la terminal. Por este motivo, se decide ejecutar InfluxDB sobre el PC en lugar de la Raspberry, pues podremos trabajar con la interfaz gráfica desde el navegador.

Para implementar el servidor (broker) MQTT se va a emplear la herramienta **Mosquitto**. Mosquitto es un broker MQTT de código abierto compatible con Raspberry Pi OS.



Figura 2.33: Logo de Mosquitto

Existen una serie de comandos que nos permiten ejecutar una serie de acciones sobre Mosquitto:

- **service mosquitto start**: Habilita el broker Mosquitto.
- **service mosquitto stop**: Deshabilita el broker Mosquitto.
- **service mosquitto status**: Permite consultar el estado del servidor, como se muestra en la figura 2.34.

```
ivanherndan@raspberrypi:~$ service mosquitto status
● mosquitto.service - Mosquitto MQTT Broker
   Loaded: loaded (/lib/systemd/system/mosquitto.service; enabled; preset: ena
   Active: active (running) since Sat 2024-09-07 12:31:55 CEST; 3min 15s ago
     Docs: man:mosquitto.conf(5)
           man:mosquitto(8)
   Process: 682 ExecStartPre=/bin/mkdir -m 740 -p /var/log/mosquitto (code=exi
   Process: 684 ExecStartPre=/bin/chown mosquitto /var/log/mosquitto (code=exi
   Process: 686 ExecStartPre=/bin/mkdir -m 740 -p /run/mosquitto (code=exited,
   Process: 688 ExecStartPre=/bin/chown mosquitto /run/mosquitto (code=exited,
   Main PID: 689 (mosquitto)
      Tasks: 1 (limit: 1557)
         CPU: 151ms
    CGroup: /system.slice/mosquitto.service
            └─689 /usr/sbin/mosquitto -c /etc/mosquitto/mosquitto.conf

sep 07 12:31:54 raspberrypi systemd[1]: Starting mosquitto.service - Mosquitto
sep 07 12:31:55 raspberrypi systemd[1]: Started mosquitto.service - Mosquitto M
lines 1-17/17 (END)...skipping...
● mosquitto.service - Mosquitto MQTT Broker
   Loaded: loaded (/lib/systemd/system/mosquitto.service; enabled; preset: enabled)
   Active: active (running) since Sat 2024-09-07 12:31:55 CEST; 3min 15s ago
     Docs: man:mosquitto.conf(5)
           man:mosquitto(8)
   Process: 682 ExecStartPre=/bin/mkdir -m 740 -p /var/log/mosquitto (code=exited, status=0/SUCCESS)
   Process: 684 ExecStartPre=/bin/chown mosquitto /var/log/mosquitto (code=exited, status=0/SUCCESS)
   Process: 686 ExecStartPre=/bin/mkdir -m 740 -p /run/mosquitto (code=exited, status=0/SUCCESS)
   Process: 688 ExecStartPre=/bin/chown mosquitto /run/mosquitto (code=exited, status=0/SUCCESS)
   Main PID: 689 (mosquitto)
      Tasks: 1 (limit: 1557)
         CPU: 151ms
    CGroup: /system.slice/mosquitto.service
            └─689 /usr/sbin/mosquitto -c /etc/mosquitto/mosquitto.conf

sep 07 12:31:54 raspberrypi systemd[1]: Starting mosquitto.service - Mosquitto MQTT Broker...
sep 07 12:31:55 raspberrypi systemd[1]: Started mosquitto.service - Mosquitto MQTT Broker.
```

Figura 2.34: Consulta del estado del broker Mosquitto

Además, para mayor comodidad, podemos hacer que Mosquitto se inicie automáticamente al dar alimentación a la Raspberry con el comando:

```
>> sudo systemctl enable mosquitto.service
```

Para comprobar el tráfico de mensajes en el servidor, tenemos varias opciones:

- **Utilizar la herramienta MQTT Explorer.** Esta herramienta nos permite, especificando la dirección IPv4 del servidor (en este caso es la de la Raspberry) y el puerto en el que se ubica (en el caso de Mosquitto es el 1883), visualizar todos los mensajes enviados al servidor y sus respectivos topics.

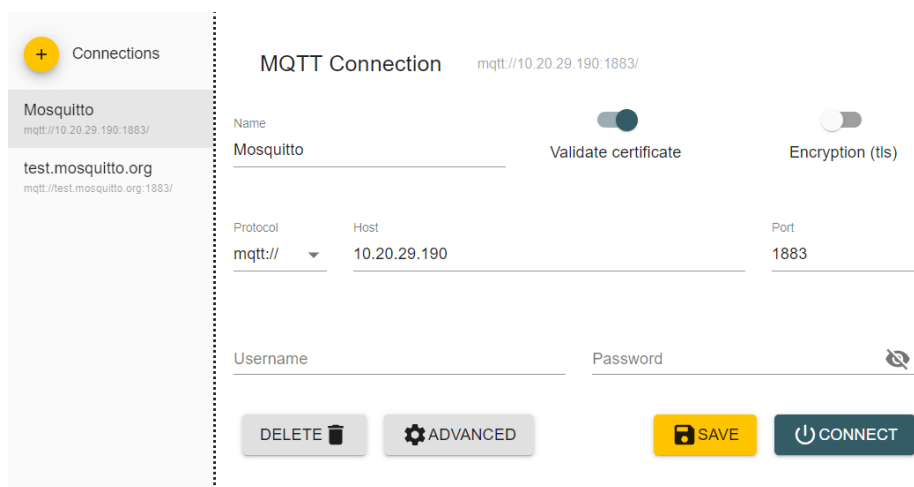


Figura 2.35: Acceso al servidor MQTT con MQTT Explorer

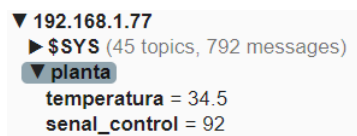


Figura 2.36: Visualización de mensajes con MQTT Explorer

- **Mediante comandos en la terminal de la Raspberry.** Con los comandos:

```
>> mosquitto_sub -h localhost -t planta/temperatura
>> mosquitto_sub -h localhost -t planta/senal_control
```

Podemos visualizar una lista con los mensajes enviados con el topic especificado según se van recibiendo en el servidor.

Además, también podemos utilizar los comandos:

```
>> mosquitto_pub -h localhost -t planta/temperatura -m
>> mosquitto_pub -h localhost -t planta/senal_control -m
```

Para publicar manualmente mensajes en estos topics, especificando el contenido del mensaje al final de la instrucción.

Los elementos que van a publicar mensajes en el broker Mosquitto son:

- **Wemos D1.** Como se vio en la sección 2.8.5.1, la Wemos lee la temperatura y la publica cada 3 segundos en el topic **planta/temperatura**.
- **Node-RED.** El programa del controlador PID (sección 2.8.5.3) implementado en Node-RED publica, también cada 3 segundos, el valor de la señal de control (es decir, el % de potencia a aplicar a la planta) en el topic **planta/senal_control**.

Para el trabajo en el Laboratorio de optimización y control se emplea la red **udcdocencia**, donde estarán conectados los diferentes elementos del sistema de control. La red a desplegar se muestra de forma esquemática en la figura 2.37.

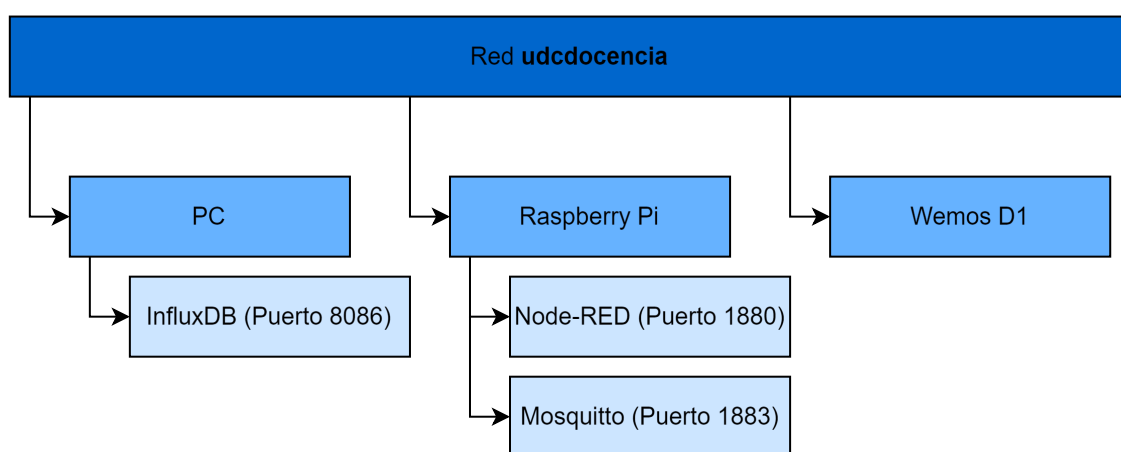


Figura 2.37: Esquema de la red desplegada para el sistema de control

2.8.5.3. Regulador PID con Node-RED

Node-RED es una herramienta de desarrollo de código abierto basada en programación visual, que permite conectar dispositivos hardware, APIs y servicios en línea.

Node-RED permite la interconexión de forma gráfica de bloques predefinidos, llamados nodos, para desarrollar una tarea concreta. La conexión de los nodos, habitualmente una combinación de nodos de entrada, nodos de procesamiento, nodos de salida y nodos de configuración, forman lo que conocemos como flow o flujo.

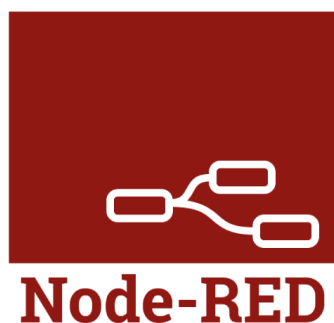


Figura 2.38: Logo de Node-RED

Al igual que el broker Mosquitto, Node-RED irá instalado sobre la Raspberry Pi. Al igual que con Mosquitto, existen una serie de comandos que se pueden ejecutar desde la terminal de Raspberry relacionadas con Node-RED:

- **node-red-start**: Inicia Node-RED.
- **node-red-stop**: Detiene Node-RED.
- **node-red-restart**: Reinicia Node-RED.
- **node-red-log**. Permite iniciar sesión con un usuario de Node-RED.

Como con Mosquitto, también es posible hacer que Node-RED se inicie de forma automática al dar alimentación a la Raspberry, con el comando:

```
>> sudo systemctl enable nodered.service
```

Si bien la herramienta está instalada en la Raspberry Pi, trabajaremos con ella desde el PC. Para acceder a la interfaz gráfica de Node-RED (figura 2.39), debemos acceder desde el navegador del PC al siguiente dominio:

```
http://WWW.XXX.YYY.ZZZ:1880
```

Donde la secuencia **WWW.XXX.YYY.ZZZ** es la dirección IP de la Raspberry Pi, y el puerto 1880 es el puerto donde se ubica el servicio de Node-RED. Si desconocemos la dirección IP de la Raspberry podemos comprobarla también desde una terminal con diferentes comandos, como **hostname -I** o **ifconfig**.

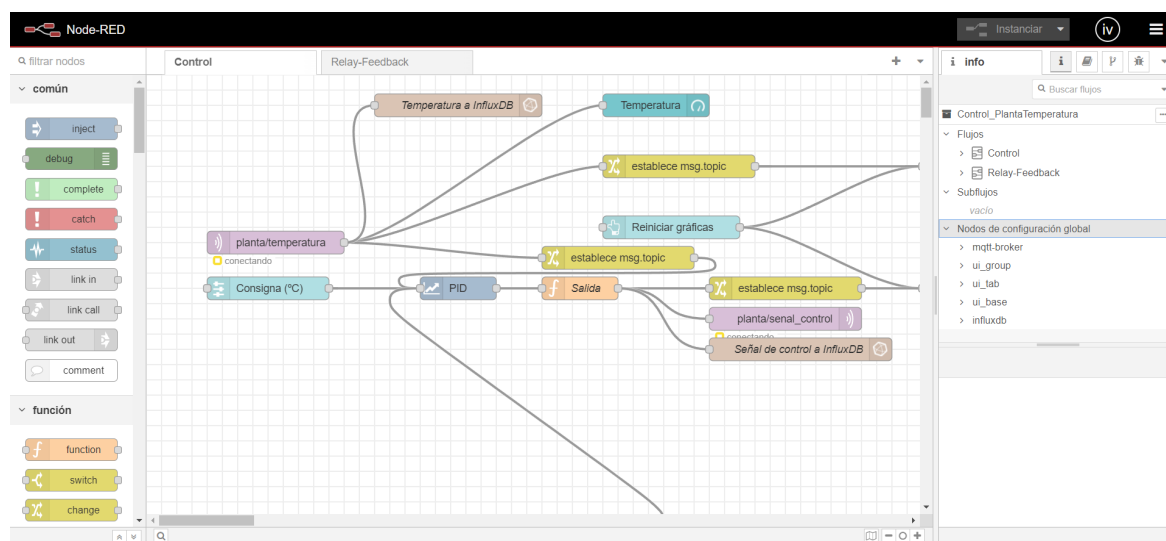


Figura 2.39: Interfaz gráfica de Node-RED

Dentro de la interfaz gráfica de Node-RED se distinguen varias secciones diferenciadas:

- **Paleta de nodos.** A la izquierda de la ventana, se presenta una lista con todos los nodos disponibles para elaborar los flujos: nodos de entrada, funciones matemáticas, conversión y tratamiento de datos... Además de los nodos por defecto podemos añadir librerías con nuevos nodos que agregan diferentes funcionalidades. En el desarrollo de este proyecto se van a instalar librerías que añaden nodos para realizar funciones específicas, como comunicación MQTT o control PID.
- **Editor de flujos.** La parte central y más amplia está dedicada a la ventana gráfica donde podemos editar nuestros nodos y conectarlos para elaborar flujos que realicen tareas concretas.
- **Ventana auxiliar.** A la derecha de todo tenemos una ventana con varias opciones: información de los flujos, subflujos y nodos; ventana de depuración; ayuda e historial.
- **Barra de herramientas.** Ubicada en la parte superior de la interfaz nos permite diversas opciones como exportar los flujos en formato JSON, instalar librerías de nodos o instanciar (compilar) los cambios introducidos en los flujos activos.

El programa de Node-RED que se va a implementar para el control de la planta ha de cumplir 4 funciones:

- Implementar un regulador PID que determine la señal de control a aplicar sobre el actuador a partir de las lecturas de temperatura.
- Comunicarse con el broker Mosquitto para publicar la señal de control en **planta/señal_control** y suscribir de **planta/temperatura** los valores de temperatura enviados por la Wemos D1.
- Permitir el control y la visualización desde una dashboard. Se ha de permitir activar y desactivar el control PID y mostrar gráficamente y/o con indicadores la señal de control y la temperatura.

- Comunicarse con InfluxDB para enviar al bucket correspondiente de la base de datos la temperatura y la señal de control.

Para esto, se implementa el flujo que se muestra en la figura 2.40 y cuyo código JSON se recoge en el anexo 3.4.2.

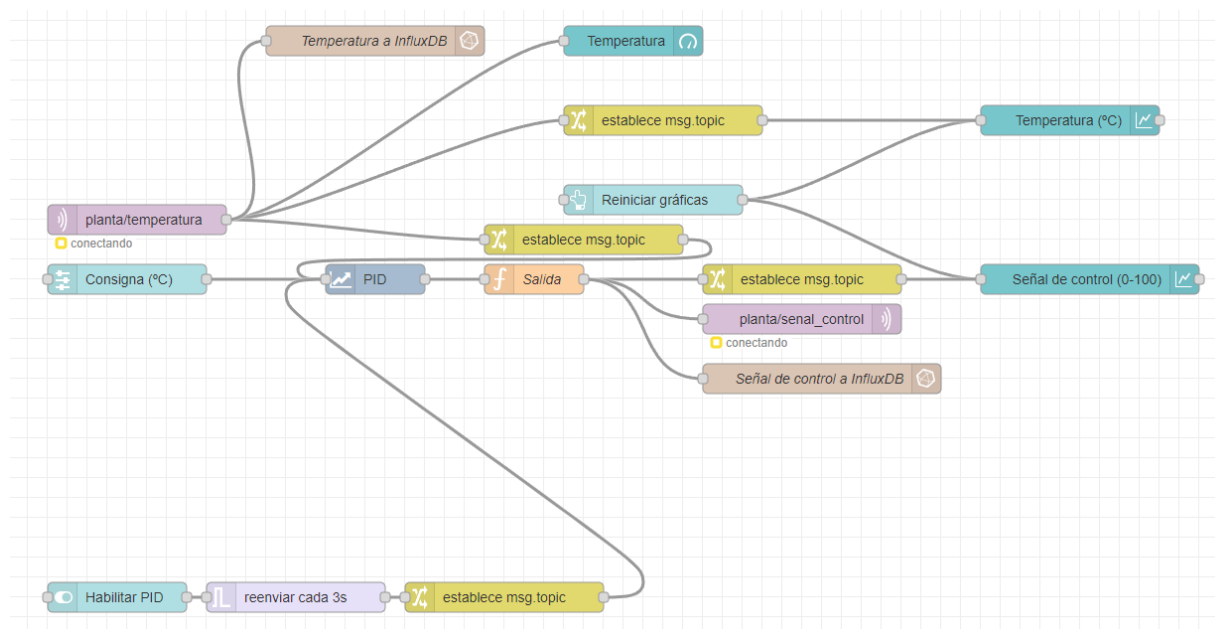


Figura 2.40: Flujo de Node-RED para el control de la planta de temperatura

La estructura del programa es bastante comprensible de por sí, pues Node-RED presenta un entorno de programación visual bastante intuitivo. La estructura del programa funciona de la siguiente forma:

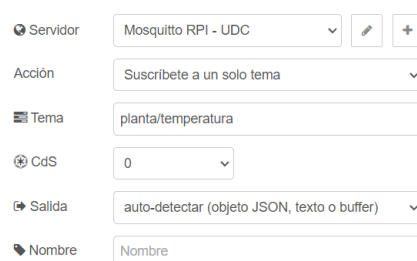
1. Se recibe la lectura de temperatura en la planta a través del nodo **mqtt in**, disponible en la paleta **Red**. En este nodo debemos configurar el broker MQTT del que queramos suscribir los datos, especificando la dirección IP del broker Mosquitto, que este caso coincide con la dirección IPv4 de la Raspberry Pi. Además, hay que especificar el puerto concreto en que se ubica el broker, que en el caso de Mosquitto es el 1803. Esta configuración se muestra en la figura 2.41.

Figura 2.41: Configuración del broker MQTT en Node-RED

Una vez se haya establecido el nodo de configuración del broker MQTT, hay que configurar el nodo mqtt in indicando el topic que se quiere suscribir, que en este caso será **planta/temperatura**. Otro parámetro importante en la configuración del nodo es la CdS (Calidad del Servicio), que sirve para definir cómo se asegura la entrega de los mensajes entre los clientes y el broker. Los niveles de CdS son:

- CdS 0, entrega como mucho una vez. El mensaje se entrega una vez, pero sin garantía de recepción. No existe confirmación de que el mensaje haya sido recibido por el receptor. Es el nivel de servicio más rápido en cuanto al tiempo requerido para el intercambio del mensaje, pero también es el menos fiable.
- CdS 1, entrega al menos una vez. Cuando se entrega el mensaje, el broker espera hasta que recibe una confirmación de recepción. De esta forma, se garantiza que el mensaje llega al cliente, enviándose varias veces de ocurrir un error en la primer, por lo que es más lento.
- CdS 2, entrega exactamente una vez. El mensaje se entrega exactamente una vez, garantizando que no habrá duplicación ni pérdida del mensaje. Este nivel es el más costoso en términos de tiempo y recursos, ya que requiere una serie de intercambios de mensajes para garantizar la entrega única.

Para este proyecto se ha decidido emplear el nivel de CdS 0, pues se trata de un sistema lento en el que la pérdida de un dato concreto de temperatura no supone graves variaciones en el proceso, por lo que se busca optimizar la velocidad de envío de los mensajes. La configuración del nodo mqtt in se muestra en la figura 2.42.



Servidor	Mosquitto RPI - UDC		
Acción	Suscríbete a un solo tema		
Tema	planta/temperatura		
CdS	0		
Salida	auto-detectar (objeto JSON, texto o buffer)		
Nombre	Nombre		

Figura 2.42: Configuración del nodo mqtt in

2. Esta lectura de temperatura se envía a InfluxDB, se muestra en un dashboard a través de un indicador de aguja y de una gráfica que muestra las lecturas de temperatura durante la última hora y se envía al regulador PID.

Para realizar el envío a InfluxDB se requiere el uso del nodo influxdb out, cuyo funcionamiento se explica en la sección 2.8.5.4.

En el dashboard se incluirá también un botón que permita reiniciar la representación de datos en las gráficas, así como un botón que permita activar y desctivar el controlador PID. También desde la dashboard se podrá seleccionar la consigna a alcanzar por el sistema, mediante un control de tipo slider. Esta dashboard se muestra en la figura 2.43.

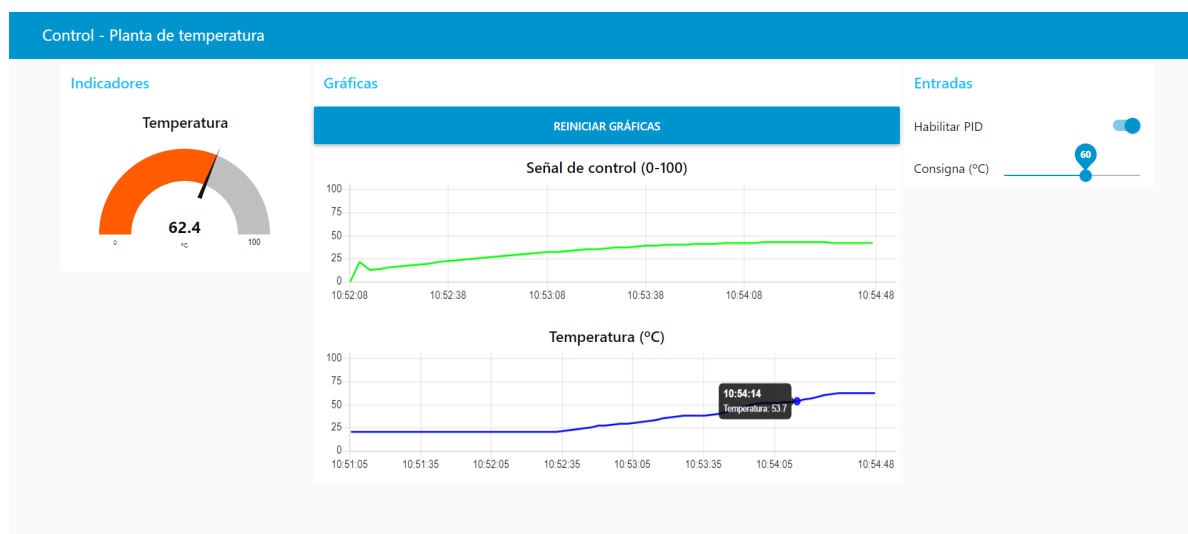


Figura 2.43: Dashboard del control PID en Node-RED

Para acceder a la visualización de la dashboard, debemos dirigirnos al mismo dominio que para acceder a Node-RED, incorporando `/ui` al final:

`http://WWW.XXX.YYY.ZZZ:1880/ui`

Antes del nodo de representación en la gráfica de la temperatura se incluye un nodo para cambiar la propiedad `msg.topic` del mensaje a **Temperatura**, pues será el nombre con el que se registre la variable graficada.

Este mismo nodo se utiliza para cambiar el topic del mensaje para su envío al regulador PID, pues este nodo dispone de hasta 3 entradas, de forma que en el topic de los mensajes de entrada debemos especificar a cuál de las entradas va asociado el mensaje.

3. Esta temperatura se emplea como variable de proceso en el regulador PID. El regulador PID utilizado es el de la librería «`node-red-contrib-easy-pid-controller`». Este nodo requiere tres argumentos de entrada, que se deben especificar en el topic del mensaje:

- **SV**. Es la consigna que ha de alcanzar en el regulador. En este caso, se fija mediante el slider de la dashboard.
- **PV**. Es la variable de proceso, la variable que se quiere manipular a través del sistema de control y que, en este programa, será la temperatura.
- **auto**. Es una entrada de tipo booleano que permite la activación y desactivación del regulador PID. Estará controlada por el correspondiente botón de la dashboard.

Además de estas entradas, se ha de configurar el nodo especificando el valor de las constantes del controlador PID, como se muestra en la figura 2.44.

Figura 2.44: Configuración del controlador PID

Además, podemos especificar si se va a utilizar para un sensor con salida de 4 a 20 mA o de 0 a 10V. En este caso nos da igual esta selección mientras hagamos correctamente la conversión del valor máximo y mínimo al rango 0-100 con un bloque function, como se muestra en la figura 2.45.

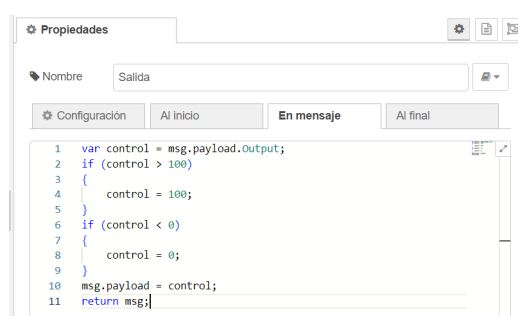


Figura 2.45: Limitación del rango de salida del regulador PID

4. La señal de salida del nodo PID es la señal de control que nos marca la potencia a aplicar a la planta. Esta señal de control se envía a la Wemos mediante un nodo mqtt out que publica en el topic **planta/senal_control**.

Esta señal de control también se representa en una gráfica en la dashboard y se envía a un bucket de InfluxDB con un nodo influxdb out.

2.8.5.4. Base de datos de series temporales con InfluxDB

InfluxDB es un servidor de base de datos de series temporales de código abierto, adecuado para logs o datos para gráficas que se generen en vivo (dashboards, ...). Es una base de datos que puede utilizarse para monitorizar eventos a nivel de sistema/red. Por eso, es útil para hacer un seguimiento de los valores de interés del sistema de control.



Figura 2.46: Logo de InfluxDB

Para iniciar en InfluxDB debemos crear una cuenta de usuario así como una organización, en donde se almacenarán todos los datos que se van a enviar a InfluxDB desde Node-RED. En InfluxDB, una organización se puede definir como una entidad que agrupa usuarios y recursos. Sirve como un contenedor para gestionar y aislar recursos, tales como bases de datos, buckets, dashboards y tareas, y otros objetos relacionados con el envío, el almacenamiento y la consulta de series de datos. Las principales características y funciones de las organizaciones en InfluxDB son:

- **Administración de usuarios.** Una organización puede estar formada por varios usuarios, los cuales pueden tener asignados diferentes permisos, de modo que se pueden definir usuarios con permisos de administrador o usuarios con acceso restringido a los datos.
- **Gestión y creación de buckets.** En InfluxDB, los datos de las series temporales se almacenan en buckets, que son equivalentes a bases de datos en otros sistemas. Para cada bucket se puede definir el tiempo de retención de los datos recibidos, e incluso que estos datos no se pierdan nunca (figura 2.47).

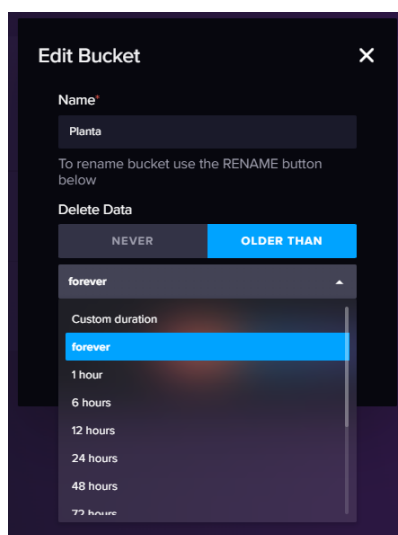


Figura 2.47: Configuración de un bucket en InfluxDB

- **Gestión de los espacios de trabajo.** Cada organización puede tener su propio conjunto de dashboards y tareas programadas para realizar consultas o análisis automáticos sobre los datos.
- **Seguridad y control de acceso.** Cada organización tiene su propio conjunto de usuarios y recursos, lo que permite controlar qué datos son accesibles por cada usuario.

Para la recopilación de datos durante este proyecto se ha procedido de forma que se ha creado un único bucket, de tiempo de retención de datos infinito, sobre el que se volcarán todos los experimentos que se hagan con el sistema, y se ha creado una dashboard asociada a dicho bucket para visualizar los datos registrados. Respecto a esta cuestión existen varias formas de proceder, pues también se podrían definir diferentes buckets en función de la naturaleza del experimento para una mayor diferenciación.

InfluxDB permite recoger datos de una gran cantidad de fuentes diversas: Arduino, Python, Java, etc, y, la que es de interés para este proyecto: Node-RED. La integración de InfluxDB con Node-RED es realmente sencilla gracias a librería «node-red-contrib-infludb». Esta añade varios nodos a la paleta de Node-RED, de los cuales se van emplea únicamente uno en el código del controlador PID explicado en la sección 2.8.5.3. El nodo **influxdb out** permite el envío de datos a un servidor de InfluxDB. Antes de entrar a configurar este nodo, debemos definir un nodo de configuración con los datos del servidor InfluxDB, como se muestra en la figura 2.48.

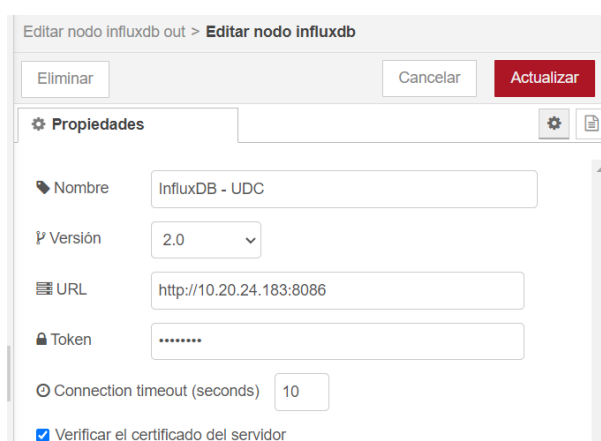


Figura 2.48: Configuración del servidor InfluxDB en Node-RED

Debemos asignar un nombre al servidor que deseamos configurar, del que debemos especificar la dirección IP (en este caso, coincide con la dirección IP del PC, y el puerto empleado es el 8086) y el token de acceso. El token es una cadena de caracteres proporcionada por el propio Influx que es requerida para acceder al servidor, de forma que sirve como mecanismo de autenticación.

Una vez que se ha definido en Node-RED el servidor de InfluxDB, se ha de configurar el nodo **influxdb out** indicando el bucket donde se almacenarán los datos y el nombre que se desea asignar a la serie de datos, como se muestra en la figura 2.49.

Editar nodo influxdb out

Eliminar Cancelar Hecho

Propiedades

Nombre: Temperatura a InfluxDB

Servidor: [v2.0] InfluxDB - UDC

Organización: 2324_GEIA_7

Bucket: Planta

Medición: Temperatura

Precisión: Milliseconds (ms)

Editar nodo influxdb out

Eliminar Cancelar Hecho

Propiedades

Nombre: Señal de control a InfluxDB

Servidor: [v2.0] InfluxDB - UDC

Organización: 2324_GEIA_7

Bucket: Planta

Medición: Señal de control

Precisión: Milliseconds (ms)

Figura 2.49: Configuración de los nodos influxdb out

Desde la ventana **Data Explorer** podremos visualizar los datos registrados en los diferentes buckets que integran la organización. Estos datos se pueden visualizar tanto en formato tabla (figura 2.51) como en una gráfica temporal (figura 2.50), seleccionando el intervalo de tiempo que se quiere visualizar. Desde esta ventana también podremos exportar los datos seleccionados a un archivo .CSV, lo que resulta muy útil para su tratamiento.

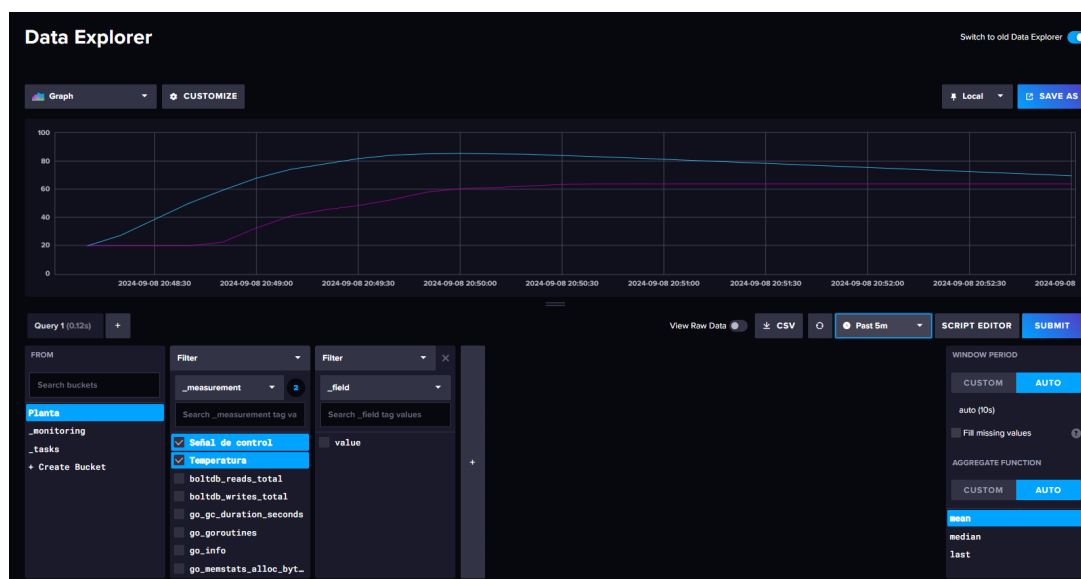


Figura 2.50: Visualización de datos en el Data Explorer de forma gráfica

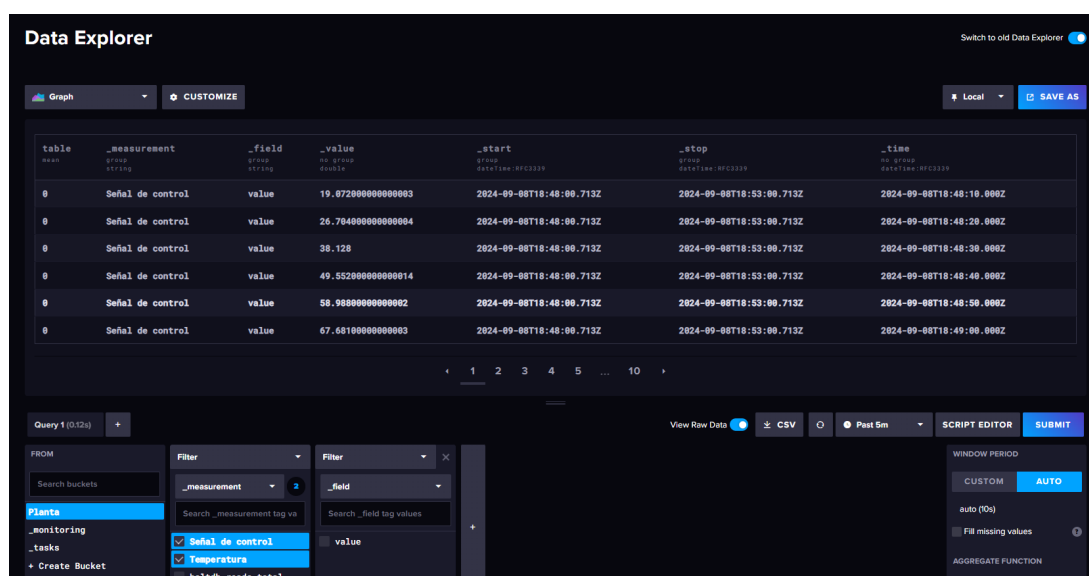


table	_measurement	_field	_value	_start	_stop	_time
mean	group	group	double	group	group	group
0	Señal de control	value	19.872800000000003	2024-09-08T18:48:00.713Z	2024-09-08T18:53:00.713Z	2024-09-08T18:48:10.000Z
0	Señal de control	value	26.704000000000004	2024-09-08T18:48:00.713Z	2024-09-08T18:53:00.713Z	2024-09-08T18:48:20.000Z
0	Señal de control	value	38.128	2024-09-08T18:48:00.713Z	2024-09-08T18:53:00.713Z	2024-09-08T18:48:30.000Z
0	Señal de control	value	49.552000000000014	2024-09-08T18:48:00.713Z	2024-09-08T18:53:00.713Z	2024-09-08T18:48:40.000Z
0	Señal de control	value	58.98000000000002	2024-09-08T18:48:00.713Z	2024-09-08T18:53:00.713Z	2024-09-08T18:48:50.000Z
0	Señal de control	value	67.68100000000003	2024-09-08T18:48:00.713Z	2024-09-08T18:53:00.713Z	2024-09-08T18:49:00.000Z

Figura 2.51: Visualización de datos en el Data Explorer en forma de tabla

También podemos crear dashboards asociadas a los diferentes buckets que permiten mayores opciones de visualización, como se observa en la figura 2.52.



Figura 2.52: Visualización de datos en dashboard de InfluxDB

Aunque no se utiliza en este proyecto, también se pueden crear tareas, que son procesos automatizados que permiten ejecutar consultas y operaciones de manera programada y recurrente dentro de la plataforma.

2.9. Orden de prioridad entre los documentos

El orden de prioridad entre los documentos que integran el presente proyecto se establece de la siguiente forma:

- Planos
- Memoria
- Anexos
- Presupuesto
- Mediciones

Título Diseño de equipo de adquisición de datos inalámbrico
para plantas de control de temperatura

Anexos

Peticionario Escuela Politécnica de Ingeniería de Ferrol
Rúa Mendizábal, s/n, Campus de Esteiro,
15403, Ferrol

Fecha Septiembre 2024

Autor El Alumno

Fdo. Iván Hermida Mella

Índice general

3.1	Contenido	55
3.2	Documentación de partida	55
3.3	Cálculos	58
3.3.1	Circuito detector de paso por cero	58
3.3.2	Circuito de disparo	63
3.3.3	Circuito de acondicionamiento de la señal de control	64
3.3.4	Temporización del ESP8266	65
3.3.5	Ajuste del regulador PID	70
3.4	Anexos en función del ámbito de aplicación del trabajo	74
3.4.1	Código de Wemos D1	75
3.4.2	Código controlador PID en Node-Red	78
3.4.3	Código controlador Relay-Feedback en Node-Red	89

3.1. Contenido

Se incluye en esta sección toda la información adicional que no se ha tratado anteriormente y que resulta necesaria para el completo desarrollo del presente proyecto.

3.2. Documentación de partida

Número de traballo
2324_GEIA_7
Titulación
Grao en Enxeñaría Electrónica e Automática
Título do proxecto (Título en Galego)
Deseño de equipo de adquisición de datos inalámbrico para plantas de control de temperatura
Título del proyecto (Título en Castellán)
Diseño de equipo de adquisición de datos inalámbrico para plantas de control de temperatura
Project Title (Título en Inglés)
Design of wireless data acquisition equipment for temperature control plants
Tipoloxía do proxecto
Proxectos de enxeñaría: relativos o deseño, fabricación, desenrolo e/ou implantación de produtos, obras, instalacións, plantas, procesos, servizos ou software
Grado de dificultade
Media
¿É unha proposta consensuado con un alumno para a súa asignación?
Si
Nome do Titor/a
Francisco Zayas Gato
Nome do Titor/a (Só se hai dous titores)
Paula Patricia Arcano Bea
Empresa do Titor (No caso de non ser da UDC)
Antecedentes detallados do proxecto
O Laboratorio de Optimización da EPEF conta con diversas plantas industriais a escala, que representan unha ferramenta moi importante para a formación do alumnado. En concreto, as plantas de control de temperatura contan con diversos elementos hardware para formar un conxunto ou bloque e así desplegar un sistema de control completo. Entre eles, dispónse dun interfaz de control de potencia totalmente funcional pero que carece dalgunhas funcionalidades como capacidade de test, control de forma inalámbrica, máis opcións para alimentación ou máis interfaces de entrada e saída, entre outros.
23/10/23
14

Obxeto detallado do proxecto

O obxecto do proxecto é o deseño dun novo equipo electrónico de adquisición de datos e de interfaz para o control de potencia entregada ás plantas de control de temperatura, que incorpore novas funcionalidades de control e configuración de forma inalámbrica, maior número de interfaces de entrada e saída ou nova envolvente, entre outros. Ademais, o novo equipo deberá gardar compatibilidade co actual en canto a intercambio de sinais.

Alcance detallado do proxecto

- Estudo de opcións para a actualización do microcontrolador que xestiona o equipo.- Deseño de nova circuitería coas novas especificacións.- Deseño hardware do novo equipo.- Fabricación e montaxe de prototipo.- Deseño dunha nova envolvente.- Software ou app para test e/ou configuración de forma inalámbrica.

3.3. Cálculos

En esta sección se desarrollan todos los cálculos realizados para el diseño del presente proyecto. Los cálculos están estructurados por apartados siguiendo el mismo orden que en la memoria.

3.3.1. Circuito detector de paso por cero

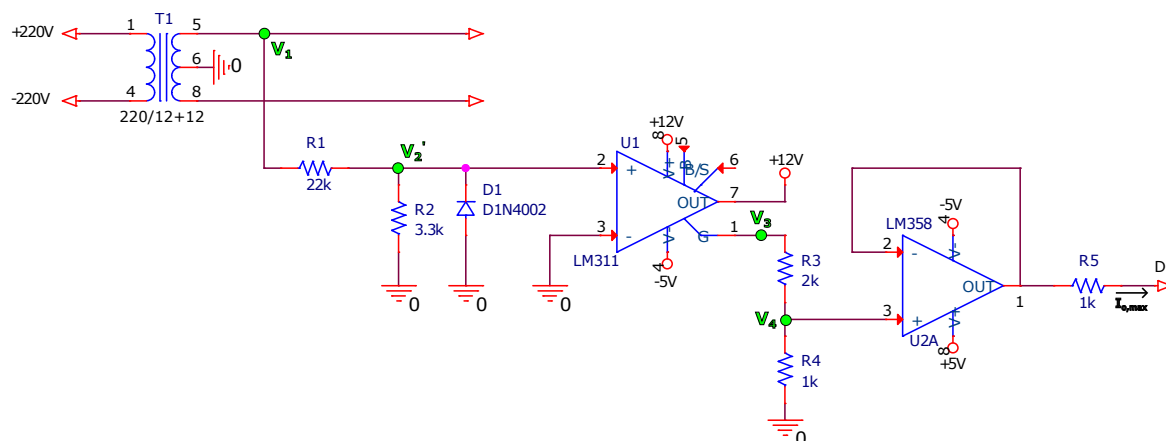


Figura 3.1: Circuito detector de paso por cero

Para la detección del paso por cero se toma la tensión V_1 del secundario del transformador para acondicionarla antes de llevarla al comparador.

Esta señal senoidal V_1 tiene un valor de tensión eficaz $V_{1,RMS} = 12V$, por lo que la amplitud de la señal de tensión se puede calcular como:

$$V_{1,RMS} = \sqrt{\frac{1}{T} \cdot \int_0^T V_1(\omega t)^2 d(\omega t)} \quad (3.1)$$

donde $V_{1,RMS}$ es el valor eficaz de tensión, T el periodo de la señal y $V_1(\omega t)$ el valor de tensión instantánea de V_1 expresado en función de la frecuencia angular y del tiempo.

Como esta señal de tensión es una señal senoidal, el valor de V_1 en un instante ωt viene dado por la expresión:

$$V_1(\omega t) = V_{1,m} \cdot \text{sen}(\omega t) \quad (3.2)$$

donde $V_{1,m}$ es la amplitud de la señal. Uniendo las ecuaciones 3.1 y 3.2 obtenemos la siguiente expresión de la que podemos obtener $V_{1,m}$:

$$V_{1,RMS} = \sqrt{\frac{1}{T} \cdot \int_0^T V_{1,m}^2 \cdot \text{sen}^2(\omega t) d(\omega t)} \quad (3.3)$$

$$V_{1,RMS}^2 = \frac{1}{T} \cdot \int_0^T V_{1,m}^2 \cdot \text{sen}^2(\omega t) d(\omega t) = \frac{V_{1,m}^2}{T} \cdot \int_0^T \text{sen}^2(\omega t) d(\omega t) \quad (3.4)$$

El periodo T de la señal expresado en radianes es de 2π al tratarse de una señal senoidal:

$$\int_0^{2\pi} \sin^2(\omega t) d(\omega t) = \left[-\frac{\sin(2x) - 2x}{4} \right]_0^{2\pi} = \pi \quad (3.5)$$

Conociendo el valor de la integral podemos obtener de la expresión 3.4 el valor de la amplitud $V_{1,m}$:

$$V_{1,RMS}^2 = \frac{V_{1,m}^2}{2\pi} \cdot \pi = \frac{V_{1,m}^2}{2} \quad (3.6)$$

$$V_{1,m} = \sqrt{2} \cdot V_{1,RMS} = \sqrt{2} \cdot 12 = 16,97V \quad (3.7)$$

Por lo tanto, V_1 es una tensión senoidal cuyo valor en un instante t viene dad por la expresión 3.8:

$$V_1 = V_{1,m} \cdot \sin(\omega t) = 16,97 \cdot \sin(100\pi \cdot t) \quad (3.8)$$

La amplitud de la señal V_1 se ve reducida por el divisor de tensión que forman R_1 y R_2 , de forma que la amplitud de la señal V_2 viene dada por la siguiente expresión:

$$V_{2,m} = \frac{R_2}{R_1 + R_2} \cdot V_{1,m} = \frac{3,3 \cdot 10^3}{22 \cdot 10^3 + 3,3 \cdot 10^3} \cdot 16,97 = 2,21V \quad (3.9)$$

La expresión que nos da el valor de V_2 en un instante cualquiera t es 3.10.

$$V_2 = V_{2,m} \cdot \sin(\omega t) = 2,21 \cdot \sin(100\pi \cdot t) \quad (3.10)$$

Esta expresión no es completa ya que hay que tener en cuenta también el efecto del diodo rectificador D_1 . Idealmente, haría que durante los semiciclos negativos la tensión V_2 fuera nula, si bien realmente existe una pequeña caída de tensión en el diodo cuyo valor aproximado es de 0,7V como se observa en la figura 3.2.

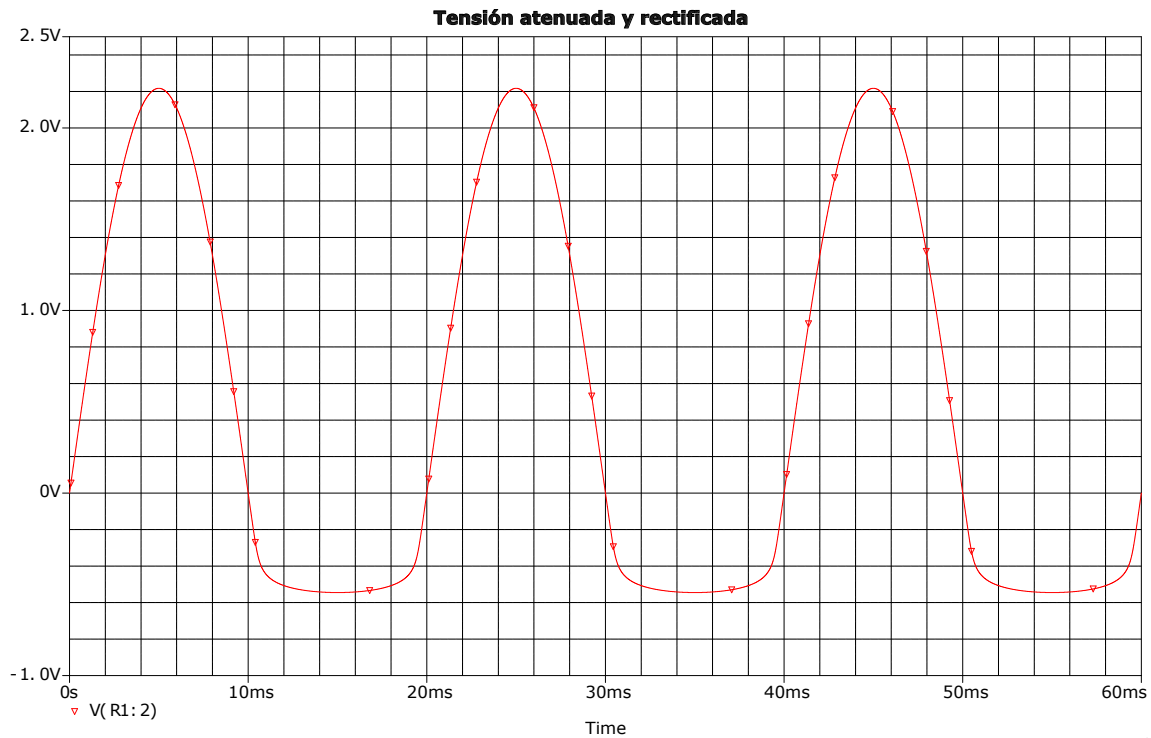


Figura 3.2: Tensión atenuada y rectificadora

Si llamamos V'_2 a este valor “verdadero” de tensión a la entrada de la patilla no inversora del LM311, su valor será:

$$V'_2(t) = \begin{cases} 2,21 \cdot \text{sen}(100\pi \cdot t) [V], & \text{si } 0 < t < 10\text{ms} \\ -0,7 [V], & \text{si } 10\text{ms} < t < 20\text{ms} \end{cases} \quad (3.11)$$

La tensión V_3 a la salida del comparador LM311, en el emisor (pin 1), será:

$$V_3 = \begin{cases} 0 [V], & \text{si } V'_2 > 0 \\ 10 [V], & \text{si } V'_2 < 0 \end{cases} \quad (3.12)$$

Buscamos adaptar esta tensión de forma que cuando la salida esté a nivel alto el valor en tensión sea de 3,3V, por lo que se emplea un divisor resistivo a través de las resistencias R_3 y R_4 para reducir el valor de la tensión continua:

$$V_{4,H} = \frac{R_4}{R_3 + R_4} \cdot V_3 = \frac{1 \cdot 10^3}{1 \cdot 10^3 + 2 \cdot 10^3} \cdot 10 = 3,3V \quad (3.13)$$

Esta tensión continua se lleva a la entrada digital D5 del microcontrolador, por lo que se utiliza un seguidor de tensión para evitar que esta tensión se vea modificada por la impedancia de entrada.

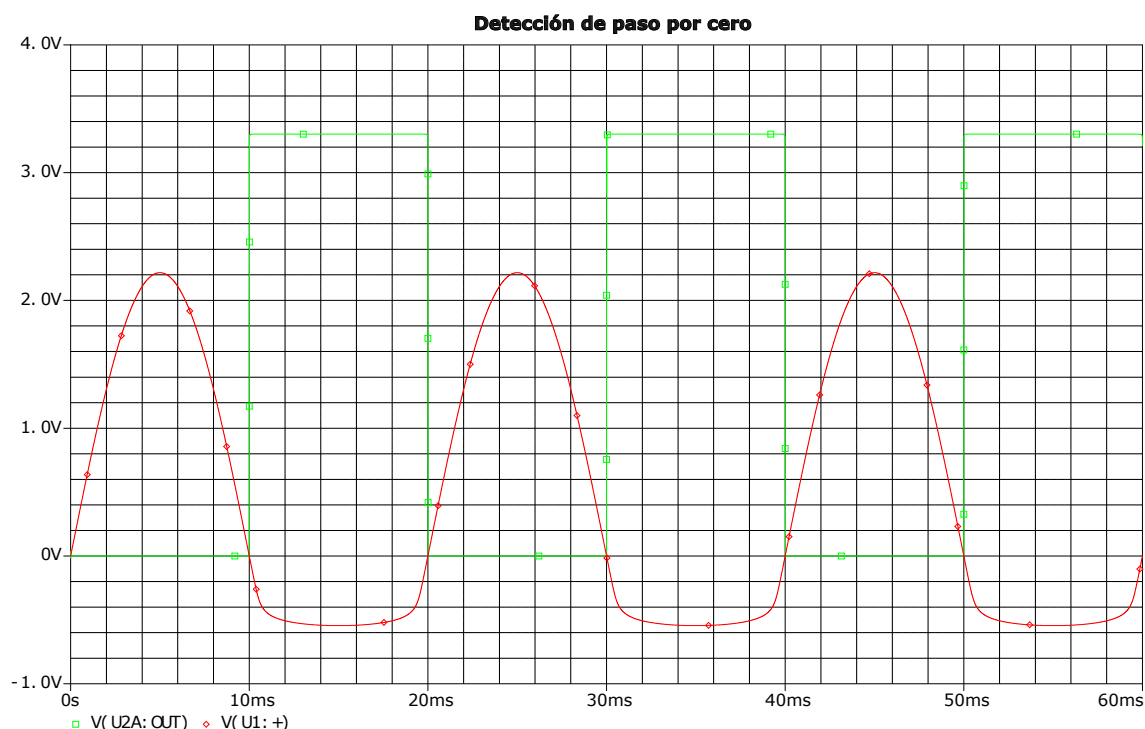


Figura 3.3: **Verde:** Señal de entrada no inversora
Rojo: Señal de salida del circuito
 Simulación del circuito detector de paso por cero

Se utiliza la resistencia R_5 para limitar la corriente, cuyo valor máximo será:

$$I_{o,max} = \frac{3,3}{R_5} = \frac{3,3}{1 \cdot 10^3} = 3,3mA \quad (3.14)$$

En este caso el consumo de corriente será muy inferior ya que, si bien el valor de la impedancia de entrada de los pines GPIO no se especifica en la hoja de características, tiene un valor del orden de megaohmios. En la hoja de características del ESP8266 se define que la corriente de entrada no debe superar los 12 mA, por lo que se cumple este requisito en cualquier caso con el diseño actual.

Las resistencias empleadas son de $\frac{1}{4}$ W, por lo que se debe comprobar que no se supera esta potencia máxima disipable. En el caso de la resistencia R_5 :

$$P_{R5,max} = I_{o,max}^2 \cdot R_5 = (3,3 \cdot 10^{-3})^2 \cdot 1 \cdot 10^3 = 10,89 \cdot 10^{-3}W = 10,89mW < \frac{1}{4}W \quad (3.15)$$

Para las resistencias R_3 y R_4 :

$$I_{R3,max} = \frac{V_3 - V_4}{R_3} = \frac{10 - 3,3}{2 \cdot 10^3} = 3,35 \cdot 10^{-3}A = 3,35mA \quad (3.16)$$

$$I_{R4,max} = \frac{V_4}{R_4} = \frac{3,3}{1 \cdot 10^3} = 3,3 \cdot 10^{-3}A = 3,3mA \quad (3.17)$$

$$P_{R3,max} = I_{R3,max}^2 \cdot R_3 = (3,35 \cdot 10^{-3})^2 \cdot 2 \cdot 10^3 = 22,45 \cdot 10^{-3}W = 22,45mW < \frac{1}{4}W \quad (3.18)$$

$$P_{R4,max} = I_{R4,max}^2 \cdot R_4 = (3,3 \cdot 10^{-3})^2 \cdot 1 \cdot 10^3 = 10,89 \cdot 10^{-3} W = 10,89 mW < \frac{1}{4} W \quad (3.19)$$

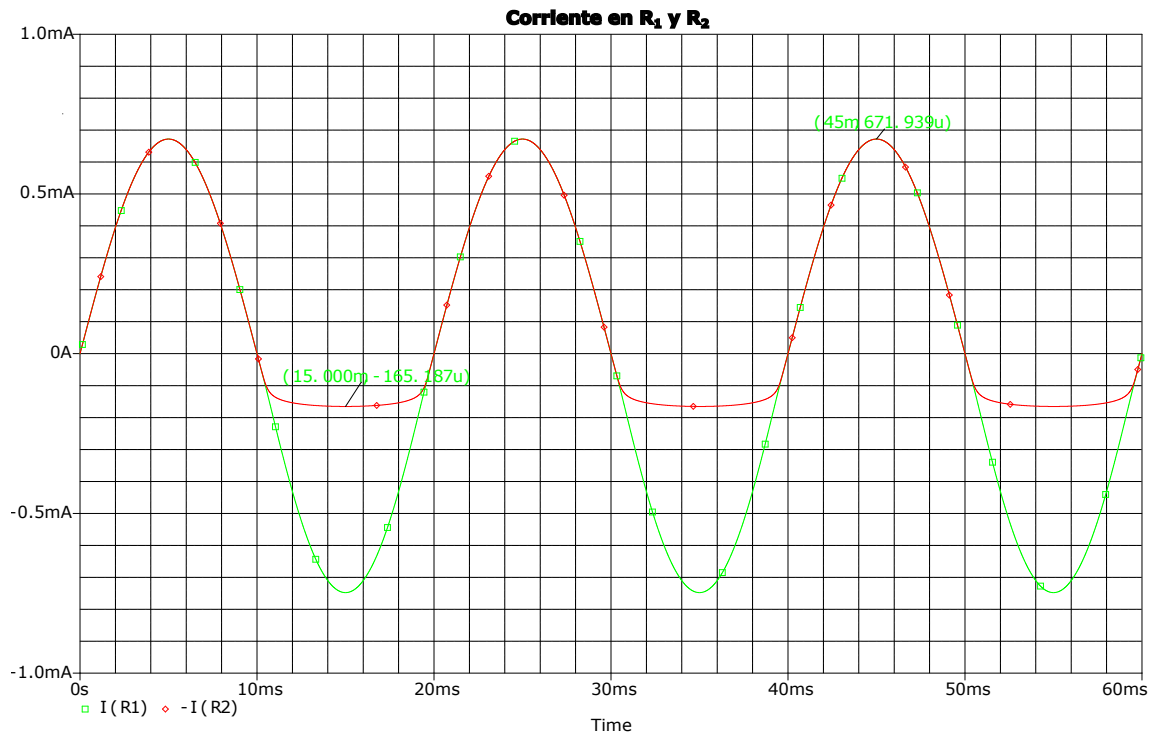


Figura 3.4: Corriente en R_1 y R_2

En la figura 3.4 se muestran los valores de corriente que circulan por R_1 y R_2 que son, respectivamente:

$$I_{R1} = 671,94 \cdot 10^{-6} \cdot \text{sen}(100\pi t) [A] \quad (3.20)$$

$$I_{R2} = \begin{cases} 671,94 \cdot 10^{-6} \cdot \text{sen}(100\pi t) [A], & \text{si } 0 < t < 10ms \\ -165,19 \cdot 10^{-6} [A], & \text{si } 10ms < t < 20ms \end{cases} \quad (3.21)$$

Se ha supuesto que la tensión y la corriente en R_2 son constantes en los semiciclos negativos, cosa que no es del todo correcta, pero que simplifica los cálculos introduciendo un error muy reducido. La corriente eficaz en ambas resistencias viene dada por las expresiones 3.22 y 3.23.

$$I_{R1,RMS} = \sqrt{\frac{1}{20 \cdot 10^{-3}} \cdot \int_0^{20 \cdot 10^{-3}} (671,94 \cdot 10^{-6} \cdot \text{sen}(100\pi t))^2 dt} = 475,13 \mu A \quad (3.22)$$

$$I_{R2,RMS} = \sqrt{\frac{1}{20 \cdot 10^{-3}} \cdot \left[\int_0^{10 \cdot 10^{-3}} (671,94 \cdot 10^{-6} \cdot \text{sen}(100\pi t))^2 dt + \int_{10 \cdot 10^{-3}}^{20 \cdot 10^{-3}} (-165,19 \cdot 10^{-6})^2 dt \right]} = 355,7 \mu A \quad (3.23)$$

Conociendo la corriente, podemos calcular la potencia disipada en R_1 y R_2 :

$$P_{R1} = I_{R1,RMS}^2 \cdot R_1 = (475,13 \cdot 10^{-6})^2 \cdot 22 \cdot 10^3 = 4,97 \cdot 10^{-3} W = 4,97 mW < \frac{1}{4} W \quad (3.24)$$

$$P_{R2} = I_{R2,RMS}^2 \cdot R_2 = (355,7 \cdot 10^{-6})^2 \cdot 3,3 \cdot 10^3 = 417,52 \mu W < \frac{1}{4} W \quad (3.25)$$

Se cumple por tanto la condición de máxima potencia disipable en todos los elementos resistivos del circuito.

3.3.2. Circuito de disparo

La tensión V_{D8} aplicada al ánodo del emisor del MOC3020 será nula en todo momento salvo cuando se le aplique un pulso de 3,3V, cuya duración es de 200 microsegundos, por lo que el valor de la tensión eficaz será muy reducido:

$$V_{D8,RMS} = \sqrt{\frac{1}{T} \cdot \int_0^T V_{D8}^2(\omega t) d(\omega t)} \quad (3.26)$$

El periodo de la señal T será de $\frac{1}{100} = 10ms$, pues este pulso se activa una vez por cada semiciclo de la señal red, es decir, con una frecuencia de 100 Hz (el doble de la red). El valor de V_{D8} en función del tiempo y de la frecuencia angular será:

$$V_{D8}(\omega t) = \begin{cases} 3,3 [V], & \text{si } 0 < t < 200 \mu s \\ 0 [V], & \text{si } 200 \mu s < t < 10ms \end{cases} \quad (3.27)$$

Sustituyendo estos valores en la expresión 3.26:

$$V_{D8,RMS} = \sqrt{\frac{1}{10 \cdot 10^{-3}} \cdot \int_0^{200 \cdot 10^{-6}} 3,3^2 d(\omega t)} = 466,69 \cdot 10^{-3} V = 466,69 mV \quad (3.28)$$

Podemos calcular así la corriente a través de la resistencia R_2 sabiendo que en el emisor del MOC3020 se produce una caída de tensión de 1,2V:

$$I_{R2}(\omega t) = \begin{cases} \frac{3,3-1,2}{470} = 4,47 \cdot 10^{-3} [A] = 4,47 [mA], & \text{si } 0 < t < 200 \mu s \\ 0 [A], & \text{si } 200 \mu s < t < 10ms \end{cases} \quad (3.29)$$

Aplicando a esta corriente la misma expresión del valor eficaz que se ha utilizado para la tensión, podemos saber la corriente eficaz en la resistencia R_2 :

$$\begin{aligned} I_{R2,RMS} &= \sqrt{\frac{1}{T} \cdot \int_0^T I_{R2}^2(\omega t) d(\omega t)} = \sqrt{\frac{1}{10 \cdot 10^{-3}} \cdot \int_0^{200 \cdot 10^{-6}} (4,47 \cdot 10^{-3})^2 d(\omega t)} = \\ &= 632,15 \cdot 10^{-6} A = 632,15 \mu A \end{aligned} \quad (3.30)$$

Conociendo la corriente podemos calcular la potencia disipada por R_2 y comprobar que no se supere la potencia máxima de $\frac{1}{4} W$:

$$P_{R2,max} = I_{R2,RMS}^2 \cdot R_2 = (632,15 \cdot 10^{-6})^2 \cdot 470 = 187,82 \cdot 10^{-6} W = 187,82 \mu W < \frac{1}{4} W \quad (3.31)$$

3.3.3. Circuito de acondicionamiento de la señal de control

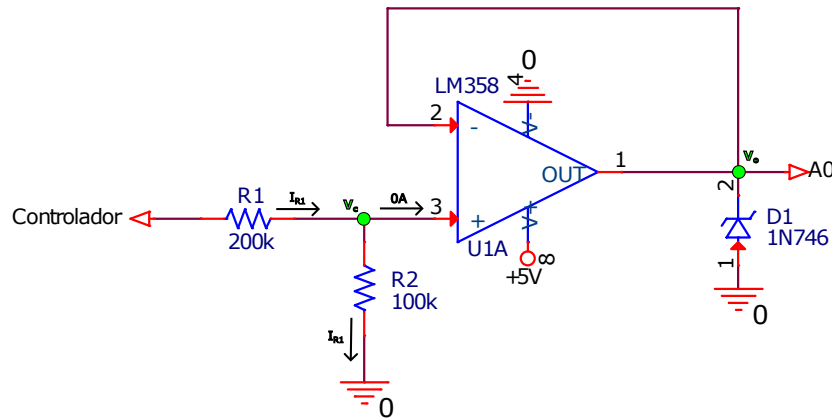


Figura 3.5: Circuito de acondicionamiento de la señal de control

Para acondicionar la señal de control proporcionada por el equipo de adquisición de datos, V_c , se utiliza un divisor resistivo implementado con las resistencias $R_1 = 200k\Omega$ y $R_2 = 100k\Omega$. El valor máximo de corriente a través de la resistencia R_1 se dará cuando el valor de la señal de control V_c se el máximo, es decir, 10V, momento en el que la tensión de salida, V_o , alcanza también su valor máximo de:

$$V_{o,max} = V_{c,max} \cdot \frac{R_2}{R_1 + R_2} = 10 \cdot \frac{100 \cdot 10^3}{100 \cdot 10^3 + 200 \cdot 10^3} = 3,3V \quad (3.32)$$

El máximo valor de corriente en R_1 será:

$$I_{R1,max} = \frac{V_{c,max} - V_{o,max}}{R_1} = \frac{10 - 3,3}{200 \cdot 10^3} = 33,5\mu A \quad (3.33)$$

En la figura 3.6 se observa la variación de la corriente en las resistencias en función de la señal de control V_c .

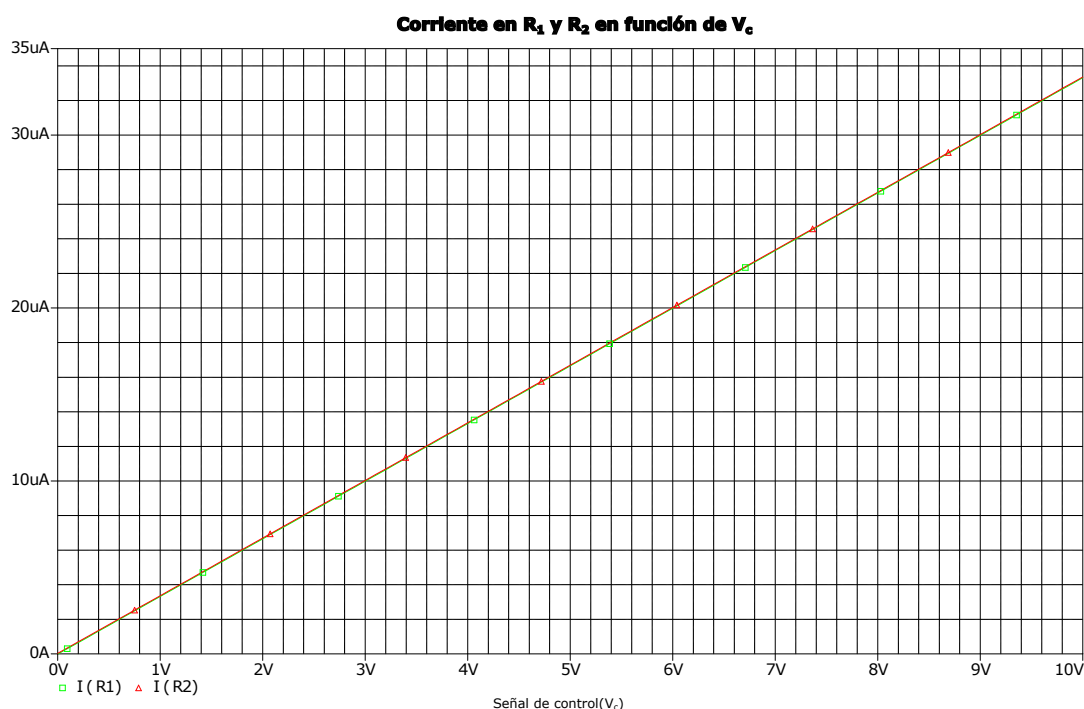


Figura 3.6: Corriente en las resistencias en función de la señal de control

Se puede considerar que la corriente en ambas resistencias es la misma, pues la corriente de entrada al amplificador tiene un valor prácticamente nulo. En la hoja de características del LM358 podemos ver que la corriente de polarización tiene un valor típico de 45 nA y máximo de 250 nA , por lo que podemos considerar despreciable su efecto a la hora de calcular la potencia disipada en las resistencias. Aplicando esta suposición, se obtiene un valor de corriente ligeramente superior al real, por lo que si en estas condiciones se cumple la condición de potencia disipada máxima, en el funcionamiento real la potencia será incluso menor. La potencia máxima disipada en las resistencias R_1 y R_2 será, respectivamente:

$$P_{R1,max} = I_{R1,max}^2 \cdot R_1 = (33,5 \cdot 10^{-6})^2 \cdot 200 \cdot 10^3 = 224,45\mu\text{W} < \frac{1}{4}\text{W} \quad (3.34)$$

$$P_{R2,max} = I_{R2,max}^2 \cdot R_2 = (33,5 \cdot 10^{-6})^2 \cdot 100 \cdot 10^3 = 112,23\mu\text{W} < \frac{1}{4}\text{W} \quad (3.35)$$

Se cumple para ambas resistencias que la potencia disipada nunca supera el valor máximo admisible.

3.3.4. Temporización del ESP8266

Para que el tiempo de duración del conteo realizado por el ESP8266 coincida exactamente con el tiempo deseado para entregar la potencia justa a la carga, se ha determinado para cada valor de potencia posible (0 % a 100 %) cuánto tiempo debe permanecer cerrado el circuito para que la potencia entregada sea la adecuada.

A la hora de obtener estos valores de temporización se ha de tener en cuenta que se busca que la entrega de potencia sea lineal.

Para poder obtener estos valores de temporización, debemos primero calcular cual es la máxima

potencia que se puede entregar a la planta y obtener la expresión de la potencia en función del ángulo de disparo α .

La carga a alimentar es una carga puramente resistiva, por lo que la potencia consumida será:

$$P = \frac{V_{RMS}^2}{R} \quad (3.36)$$

Donde P es la potencia, V_{RMS} la tensión eficaz y R la resistencia.

La tensión eficaz V_{RMS} se define, para una señal senoidal, con la expresión:

$$V_{RMS} = \sqrt{\frac{1}{2\pi} \cdot \int_0^{2\pi} V_m^2 \cdot \text{sen}^2(\omega t) d(\omega t)} = \sqrt{\frac{1}{\pi} \cdot \int_0^{\pi} V_m^2 \cdot \text{sen}^2(\omega t) d(\omega t)} \quad (3.37)$$

Donde V_m es la amplitud de la señal senoidal. Como la carga se va a alimentar con la señal de red, la amplitud de la señal será, aplicando la ecuación 3.7, $V_m = \sqrt{2} \cdot 220 = 311,13V$.

Si llamamos P_o a la potencia entregada a una carga resistiva cuando se le aplica una señal senoidal pura, el valor de P_o viene dado por:

$$P_o = \frac{\frac{1}{\pi} \cdot \int_0^{\pi} V_m^2 \cdot \text{sen}^2(\omega t) d(\omega t)}{R} \quad (3.38)$$

Esta potencia P_o se corresponde con la potencia máxima que se puede entregar a la carga; sería el 100 % de la potencia.

Si definimos ahora como P la potencia entregada a la carga cuando se dispara el triac en un instante α (en radianes), este valor viene dado por la expresión 3.39.

$$P = \frac{\frac{1}{\pi} \cdot \int_{\alpha}^{\pi} V_m^2 \cdot \text{sen}^2(\omega t) d(\omega t)}{R} \quad (3.39)$$

Si definimos como x el cociente $\frac{P}{P_o}$, esta x designa el porcentaje de potencia ($x = 0,01; 0,02; \dots; 1$) expresado en tanto por uno.

$$x = \frac{\frac{\frac{1}{\pi} \cdot \int_{\alpha}^{\pi} V_m^2 \cdot \text{sen}^2(\omega t) d(\omega t)}{R}}{\frac{\frac{1}{\pi} \cdot \int_0^{\pi} V_m^2 \cdot \text{sen}^2(\omega t) d(\omega t)}{R}} \quad (3.40)$$

Si simplificamos los factores comunes en numerador y denominador:

$$x = \frac{\int_{\alpha}^{\pi} \text{sen}^2(\omega t) d(\omega t)}{\int_0^{\pi} \text{sen}^2(\omega t) d(\omega t)} \quad (3.41)$$

Si calculamos el valor de las integrales:

$$\int_0^{\pi} \text{sen}^2(\omega t) d(\omega t) = \frac{\pi}{2} \quad (3.42)$$

$$\int_{\alpha}^{\pi} \text{sen}^2(\omega t) d(\omega t) = \left[\frac{2(\omega t) - \text{sen}(2\omega t)}{4} \right]_{\omega t=\alpha}^{\omega t=\pi} = \frac{\pi}{2} - \frac{2\alpha - \text{sen}(2\alpha)}{4} = \frac{2\pi - 2\alpha + \text{sen}(2\alpha)}{4} \quad (3.43)$$

Sustituimos estos valores en 3.41 y obtenemos una función de x y α :

$$x = \frac{\frac{2\pi - 2\alpha + \text{sen}(2\alpha)}{4}}{\frac{\pi}{2}} \quad (3.44)$$

$$x = \frac{2\pi - 2\alpha + \sin(2\alpha)}{2\pi} \quad (3.45)$$

A partir de esta ecuación podemos obtener el valor del ángulo de disparo α para cada valor de x , y pasar esta α de ángulo (radianes) a tiempo (milisegundos) para saber el tiempo que debe durar cada temporización.

Para realizar esta temporización se utiliza el *Timer1*, que es uno de los dos temporizadores de los que dispone el ESP8266. Se trata de un timer de 23 bits, por lo que el máximo valor que puede contar es:

$$2^{23} - 1 = 8388607 \quad (3.46)$$

La frecuencia de la señal de reloj es $f = 8MHz$, por lo que el período de la señal será:

$$T = \frac{1}{f} = \frac{1}{8 \cdot 10^6} = 125 \cdot 10^{-9} s = 125ns \quad (3.47)$$

El Timer se puede configurar con 3 opciones de prescaler: 1, 16 y 256, de forma que la frecuencia base de 80 MHz se divide entre el valor del prescaler, resultando en unas frecuencias resultantes de:

- Prescaler a 1 $\Rightarrow f = \frac{8 \cdot 10^6}{1} = 80MHz$
- Prescaler a 16 $\Rightarrow f = \frac{8 \cdot 10^6}{16} = 500kHz$
- Prescaler a 256 $\Rightarrow f = \frac{8 \cdot 10^6}{256} = 31,25kHz$

Al disminuir la frecuencia de reloj, aumenta el tiempo máximo que se puede temporizar, pero disminuye la sensibilidad, que en este caso coincide con el período de la señal: el intervalo de tiempo mínimo que puede detectar el timer.

- Prescaler a 1:
Período: $\frac{1}{80 \cdot 10^6} = 125 \cdot 10^{-9} s = 125ns$
Tiempo máximo a temporizar: $8388607 \cdot 125 \cdot 10^{-9} = 1,048s$
- Prescaler a 16:
Período: $\frac{1}{500 \cdot 10^3} = 2 \cdot 10^{-6} s = 2\mu s$
Tiempo máximo a temporizar: $8388607 \cdot 2 \cdot 10^{-6} = 16,777s$
- Prescaler a 256:
Período: $\frac{1}{31,25 \cdot 10^3} = 32 \cdot 10^{-6} s = 32\mu s$
Tiempo máximo a temporizar: $8388607 \cdot 32 \cdot 10^{-6} = 268,44s$

Como el tiempo máximo a temporizar es de 10 ms (cuando se aplique el 0 % de la potencia total), se emplea el valor de prescaler 1, pues nos permite realizar una temporización suficiente y es el que ofrece mayor precisión.

De esta forma, podemos obtener el valor que se ha de cargar en el timer para cada valor de potencia empleando la expresión 3.48.

$$N_{timer} = 8388607 \cdot \alpha \quad (3.48)$$

En esta expresión, N_{timer} es el valor entero a cargar en el timer para realizar la temporización y α el tiempo en milisegundos que se desea contabilizar. Aplicando esta expresión a los 101

valores de α obtenidos anteriormente, podemos obtener todos los valores a introducir en el timer para cada nivel de potencia, que son los elementos que forman el array “tiempo[]” que se incluye en el código del controlador, y que se recogen en la tabla 3.1.

Tabla 3.1: Temporización y valor a cargar en el timer para cada nivel de potencia

Potencia (%)	Ángulo de disparo (rad)	Tiempo de disparo (ms)	Valor Timer1
0	π	10	755200
1	2.777179	8.84	707200
2	2.679993	8.531	682480
3	2.610761	8.31	664800
4	2.554893	8.132	650560
5	2.507119	7.98	638400
6	2.464859	7.846	627680
7	2.426635	7.724	617920
8	2.391512	7.612	608960
9	2.358856	7.508	600640
10	2.328216	7.411	592880
11	2.299256	7.319	585520
12	2.27172	7.231	578480
13	2.245409	7.147	571760
14	2.220161	7.067	565360
15	2.195846	6.99	559200
16	2.172356	6.915	553200
17	2.1496	6.842	547360
18	2.127501	6.772	541760
19	2.105995	6.704	536320
20	2.085023	6.637	530960
21	2.064537	6.572	525760
22	2.044494	6.508	520640
23	2.024854	6.445	515600
24	2.005583	6.384	510720
25	1.986652	6.324	505920
26	1.968032	6.264	501120
27	1.949698	6.264	501120
28	1.931629	6.149	491920
29	1.913802	6.092	487360
30	1.8962	6.036	482880
31	1.878805	5.98	478400
32	1.8616	5.926	474080
33	1.84457	5.871	469680
34	1.827702	5.818	465440
35	1.810981	5.765	461200
36	1.794397	5.712	456960
37	1.777937	5.659	452720

38	1.76159	5.607	448560
39	1.745346	5.556	444480
40	1.729194	5.504	440320
41	1.713125	5.453	436240
42	1.69713	5.402	432160
43	1.6812	5.351	428080
44	1.665325	5.301	424080
45	1.649498	5.251	420080
46	1.633711	5.2	416000
47	1.617955	5.15	412000
48	1.602222	5.1	408000
49	1.586506	5.05	404000
50	$\frac{\pi}{2}$	5	400000
51	1.555087	4.95	396000
52	1.53937	4.9	392000
53	1.523637	4.85	388000
54	1.507882	4.8	384000
55	1.492094	4.749	379920
56	1.476267	4.699	375920
57	1.460393	4.649	371920
58	1.444463	4.598	367840
59	1.428467	4.547	363760
60	1.412399	4.496	359680
61	1.396247	4.444	355520
62	1.380002	4.393	351440
63	1.363656	4.341	347280
64	1.347195	4.288	343040
65	1.330611	4.235	338800
66	1.313891	4.182	334560
67	1.297023	4.129	330320
68	1.279993	4.074	325920
69	1.262788	4.02	321600
70	1.245392	3.964	317120
71	1.22779	3.908	312640
72	1.209964	3.851	308080
73	1.191894	3.794	303520
74	1.173561	3.736	298880
75	1.154941	3.676	294080
76	1.136009	3.616	289280
77	1.116739	3.555	284400
78	1.097099	3.492	279360
79	1.077055	3.428	274240
80	1.056569	3.363	269040

81	1.035598	3.296	263680
82	1.014091	3.228	258240
83	0.991993	3.158	252640
84	0.969237	3.085	246800
85	0.945747	3.01	240800
86	0.921432	2.933	234640
87	0.896184	2.853	228240
88	0.869872	2.769	221520
89	0.842337	2.681	214480
90	0.813377	2.589	207120
91	0.782736	2.492	199360
92	0.750081	2.388	191040
93	0.714958	2.276	182080
94	0.676734	2.154	172320
95	0.634474	2.02	161600
96	0.586699	1.868	149440
97	0.530831	1.69	135200
98	0.4616	1.469	117520
99	0.364413	1.16	92800
100	0	0	4800

Los valores extremos (0 % y 100 % de potencia) no coinciden exactamente con los que se han obtenido previamente, pues al valer justamente 10 y 0 milisegundos respectivamente coinciden con el paso por cero de la señal de red y se han corregido empíricamente para asegurar un correcto funcionamiento.

3.3.5. Ajuste del regulador PID

Para obtener unos parámetros (K_p , K_i , K_d) del regulador PID adecuados al sistema que se pretende controlar se ha de realizar un ajuste empírico que permita obtener unos valores razonables para estas constantes para que la oscilación y el tiempo de subida se ajusten a los requerimientos del sistema de control.

El método que se emplea en el presente proyecto es el método del relé (Relay Feedback), que nos permite localizar empíricamente la ganancia crítica (K_c) y el periodo de oscilación sostenida (T_c) del sistema. Para esto, se lleva el sistema a un estado de oscilación con la incorporación de un relé como se muestra en la figura 3.7.

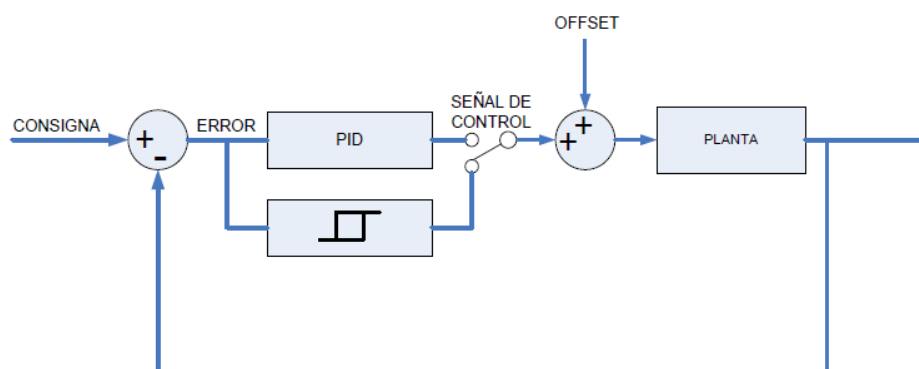


Figura 3.7: Esquema para la realización del Relay Feedback

Esta oscilación conseguida tiene como período aproximadamente el mismo valor que el periodo de oscilación sostenido. En el experimento es conveniente emplear un relé con histéresis cuyas características son las mostradas en la figura 3.8, una amplitud d y un ancho de la ventana de histéresis h .

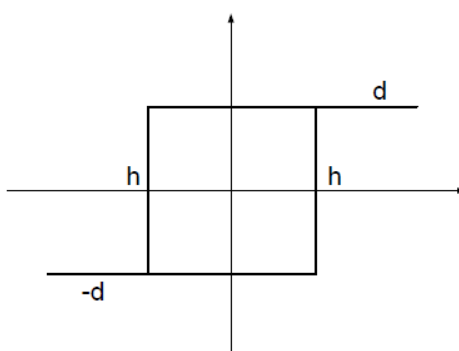


Figura 3.8: Ventana de histéresis utilizada en el relé

Cabe señalar que este método nos proporcionará unos valores adaptados al punto de consigna en el que estemos trabajando (el centro de la ventana de histéresis) y, si se cambia esta consigna, los valores de ganancia crítica y periodo de oscilación sostenida y, por tanto, las ganancias del regulador PID, se modificarán. Para evitar tener que realizar el experimento múltiples veces con diferentes consignas y obtener diferentes valores por rangos, se decide utilizar como consigna 50° , que es punto medio del rango de medida, y se utiliza una ventana de histéresis de ± 10 . El sistema funcionará de la siguiente forma:

- Partiendo del reposo (temperatura en torno a 20°), el controlador entregará un 100 % de potencia a la carga hasta que se alcanecen los 60° (consigna de 50° más los 10° de la ventana de histéresis).
- Cuando se supere esta temperatura, se abre el relé y el controlador pasa a entregar potencia nula (0 %) a la carga.
- Al no estar recibiendo potencia el elemento actuador, la planta se enfriará progresivamente. Cuando se alcance una temperatura inferior a 40° ((consigna de 50° menos los 10° de la ventana de histéresis) se cierra el relé.

- Al cerrarse el relé, se entrega un 100 % de potencia hasta que se vuelva a alcanzar el límite de 60°, repitiéndose el proceso desde el punto 2.

Se implementa en Node-Red el flujo de control con Relay Feedback de la figura 3.9. El código JSON del flujo se recoge en el anexo 3.4.3.

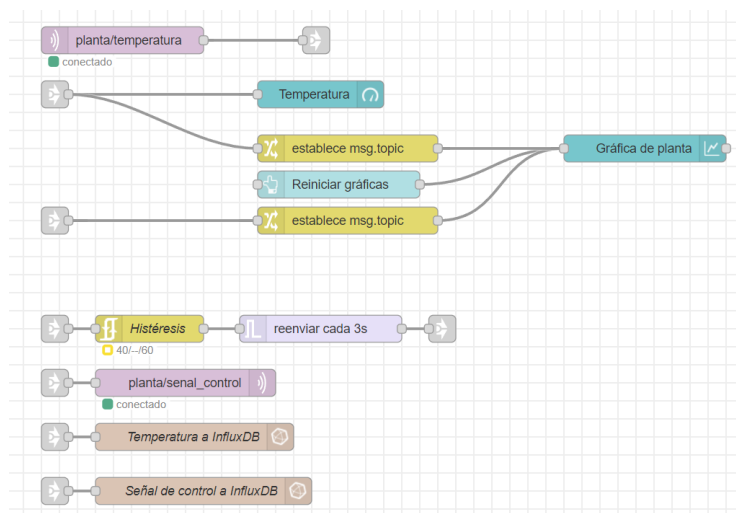


Figura 3.9: Flujo de Node-Red para realizar el control Relay Feedback

Al igual que con el control PID, se puede utilizar el dashboard de Node-Red para visualizar los valores (figura 3.10). También se vuelcan los datos de señal de control y temperatura sobre InfluxDB (figura 3.11), de donde vamos a obtener la respuesta del sistema, midiendo la señal de control y la temperatura cada 6 segundos durante 5 periodos completos.

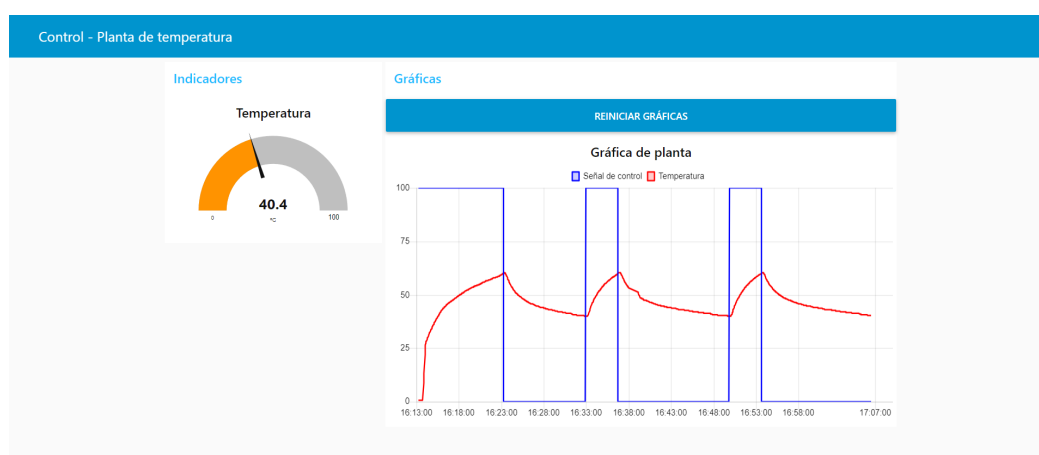


Figura 3.10: Visualización de valores del Relay Feedback en el dashboard de Node-Red

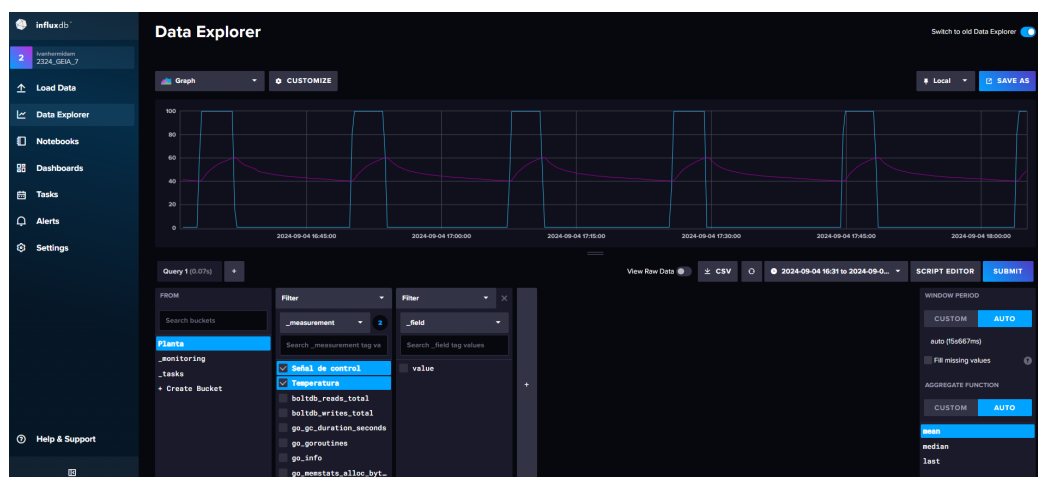


Figura 3.11: Visualización de valores del Relay Feedback en InfluxDB

La respuesta del sistema al control Relay Feedback se muestra en la figura 3.12.

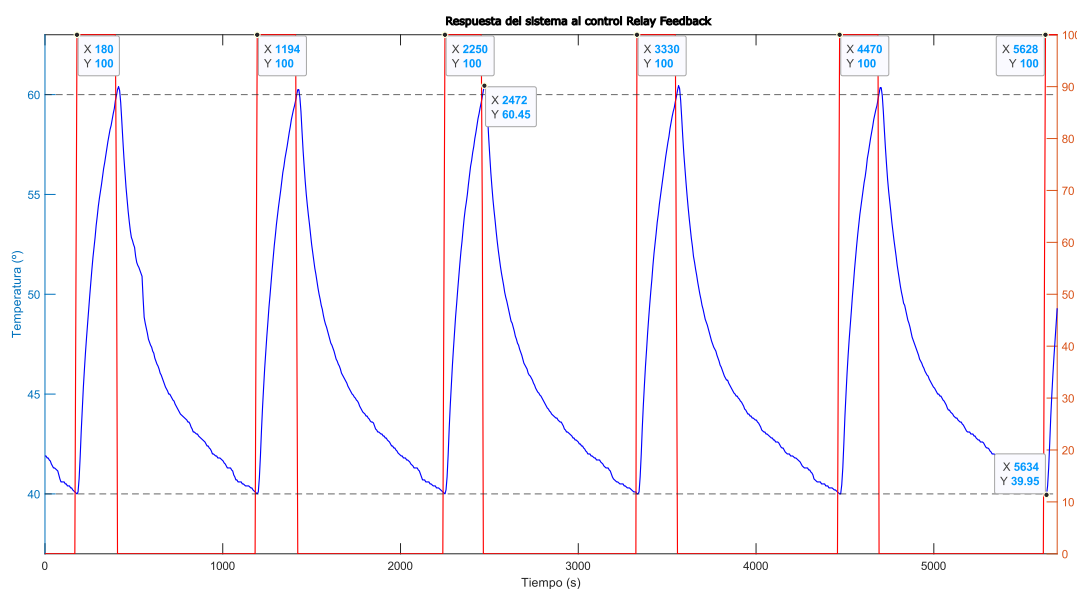


Figura 3.12: Respuesta del sistema al control Relay Feedback

A la hora de obtener la ganancia crítica y el periodo de oscilación sostenida, hemos de tomar los siguientes valores:

- El periodo de oscilación sostenida (T_c) es el periodo de la señal medida.
- La ganancia crítica (K_c) viene dada por la siguiente fórmula:

$$K_c = \frac{4d}{\pi \cdot \sqrt{a^2 - h^2}} \quad (3.49)$$

donde h es la ventana de histéresis, d la mitad de la amplitud de la señal de salida del relé y a la amplitud de la señal medida.

Durante los 5 periodos de señal de salida completos que se han medido, vemos que el periodo de la señal no es constante, si no que se observa una ligera tendencia: el sistema tarda más tiempo enfriarse cada vez que se calienta hasta los 60º. A falta de un criterio más preciso, calcularemos este periodo haciendo la media de todos los valores de periodo salvo el primero (el más corto) y el último (el más largo).

$$T_c = \frac{(2250 - 1194) + (3330 - 2250) + (4470 - 3330)}{3} = 1094s \quad (3.50)$$

Para calcular K_c , se toma $h = 10$, pues la ventana de histéresis es de ± 10 , $d = 50$, pues la salida varía entre 0 y 100 y $a = 60.45 - 39.95 = 20.5$. El valor resutante viene dado por la ecuación ??:

$$K_c = \frac{4 \cdot 50}{\pi \cdot \sqrt{20,5^2 - 20^2}} = 14,147 \quad (3.51)$$

Cone estos datos, se calculan los valores de las constantes K , T_d y T_i según las fórmulas de Ziegler-Nichols en cadena cerrada. Como el sistema es muy lento y presenta poca inercia térmica (y por tanto poca sobreoscilación) este criterio resulta suficientemente adecuado para el caso.

- $K = 0,6 \cdot K_c = 0,6 \cdot 14,147 = 8,49$
- $T_i = 0,5 \cdot T_c = 0,5 \cdot 1094 = 547$
- $T_d = 0,125 \cdot T_c = 0,125 \cdot 1094 = 136,75$

Estos valores son los utilizados en la fórmula del regulador PID siguiendo la expresión 3.52:

$$u(t) = K \cdot \left[e(t) + \frac{1}{T_i} \cdot \int_0^t e(t)dt + T_d \cdot \frac{d}{dt}e(t) \right] \quad (3.52)$$

donde $u(t)$ es la señal de control (salida del controlador PID) y $e(t)$ es la señal de error (entrada del controlador PID). En el regulador PID de Node-Red se ha de especificar los valores de ganancia K_p , K_i y K_d por separado, según la expresión 3.53:

$$u(t) = K_p \cdot e(t) + K_i \cdot \int_0^t e(t)dt + K_d \cdot \frac{d}{dt}e(t) \quad (3.53)$$

Estos valores se calculan como:

- $K_p = K = 8,49$
- $K_i = \frac{K}{T_i} = \frac{8,49}{547} = 0,02$
- $K_d = K \cdot T_d = 1160,76$

3.4. Anexos en función del ámbito de aplicación del trabajo

Se recogen aquí todos los códigos de programación desarrollados en el transcurso del proyecto

3.4.1. Código de Wemos D1

Se incluye a continuación el código de Arduino para utilizar la Wemos D1 como sistema de adquisición de la temperatura en la planta y realizar el control del circuito de potencia.

```
1 //Librerías para la comunicación WiFi
2 #include <ESP8266WiFi.h>
3 #include <PubSubClient.h>
4
5 //Parámetros para conexión a la red WiFi
6 const char* ssid = "udcdocencia";
7 const char* password = "SenFios.Especial";
8 const char* mqtt_server = "10.20.29.190";
9 WiFiClient espClient;
10 PubSubClient client(espClient);
11 unsigned long lastMsg = 0;
12 #define MSG_BUFFER_SIZE (50)
13 char msg[MSG_BUFFER_SIZE];
14 int value = 0;
15
16 //Variables del programa
17 const unsigned long tiempo[] =
18 {755200,707200,682480,664800,650560,638400,627680,617920,608960,
19 600640,592880,585520,578480,571760,565360,559200,553200,547360,
20 541760,536320,530960,525760,520640,515600,510720,505920,501120,
21 501120,491920,487360,482880,478400,474080,469680,465440,461200,
22 456960,452720,448560,444480,440320,436240,432160,428080,424080,
23 420080,416000,412000,408000,404000,400000,396000,392000,388000,
24 384000,379920,375920,371920,367840,363760,359680,355520,351440,
25 347280,343040,338800,334560,330320,325920,321600,317120,312640,
26 308080,303520,298880,294080,289280,284400,279360,274240,269040,
27 263680,258240,252640,246800,240800,234640,228240,221520,214480,
28 207120,199360,191040,182080,172320,161600,149440,135200,117520,
29 92800,4800};
30 const int t_muestreo = 3000;
31 unsigned long tiempo_actual = 0, retardo = 0;
```



```
32 int potencia;
33 float tension_entrada, media_entradas, lectura_ad,
    temperatura;
34 float entradas[] = {0,0,0,0,0,0};
35
36 void setup_wifi()
37 {
38     delay(10);
39     WiFi.mode(WIFI_STA);
40     WiFi.begin(ssid, password);
41     while (WiFi.status() != WL_CONNECTED)
42     {
43         delay(500);
44     }
45     randomSeed(micros());
46 }
47
48 void callback(char* topic, byte* payload, unsigned int length
    )
49 {
50     payload[length] = '\0';
51     potencia = atoi((char *)payload);
52 }
53
54 void reconnect()
55 {
56     while (!client.connected())
57     {
58         String clientId = "ESP8266Client-";
59         clientId += String(random(0xffff), HEX);
60         if (client.connect(clientId.c_str()))
61         {
62             client.subscribe("planta/senal_control");
63         }
64         else
65         {
66             delay(5000);
67         }
68     }
69 }
70
71 void setup()
72 {
73     Serial.begin(115200);
74     pinMode(D5, INPUT);
75     pinMode(A0, INPUT);
```

```
76  pinMode(D8, OUTPUT);
77  attachInterrupt(digitalPinToInterrupt(D5), PasoPorCero,
    CHANGE);
78  timer1_enable(TIM_DIV1, TIM_EDGE, TIM_SINGLE);
79  timer1_attachInterrupt(onTimer);
80  setup_wifi();
81  client.setServer(mqtt_server, 1883);
82  client.setCallback(callback);
83 }
84
85 void loop()
86 {
87   if (!client.connected())
88   {
89     reconnect();
90   }
91   client.loop();
92   tiempo_actual = millis();
93   lectura_ad = analogRead(A0);
94   tension_entrada = lectura_ad * (3.2 / 1023.0);
95   for (int i = 0; i <= 4; i++)
96   {
97     entradas[i] = entradas[i+1];
98   }
99   entradas[5] = tension_entrada;
100  media_entradas = (entradas[0]+entradas[1]+entradas[2]+
    entradas[3]+entradas[4]+entradas[5])/6;
101  temperatura = media_entradas * (100 / 3.2);
102  snprintf(msg, 75, "%.1f", temperatura);
103  client.publish("planta/temperatura", msg);
104  while(millis() < tiempo_actual + t_muestreo)
105  {
106    yield();
107  }
108 }
109
110 ICACHE_RAM_ATTR void onTimer()
111 {
112   digitalWrite(D8, HIGH);
113   retardo = micros();
114   while(micros() < retardo + 200){}
115   digitalWrite(D8, LOW);
116 }
117
118 ICACHE_RAM_ATTR void PasoPorCero()
119 {
```

```
120 timer1_write(tiempo[potencia]);  
121 }
```

3.4.2. Código controlador PID en Node-Red

Se incluye a continuación el código JSON del controlador PID implementado en Node-Red, con las opciones de visualización en dashboard, publicación y suscripción en servidor MQTT y envío a InfluxDB.

```
1  [  
2    {  
3      "id": "88ad40c4a9bed753",  
4      "type": "tab",  
5      "label": "Control",  
6      "disabled": false,  
7      "info": "Control de la temperatura de la planta mediante  
8        ↳ regulador PID",  
9      "env": []  
10   },  
11   {  
12     "id": "2ab52dc890b68128",  
13     "type": "mqtt_in",  
14     "z": "88ad40c4a9bed753",  
15     "name": "",  
16     "topic": "planta/temperatura",  
17     "qos": "0",  
18     "datatype": "auto-detect",  
19     "broker": "b4cbff7cc98e8a0c",  
20     "nl": false,  
21     "rap": true,  
22     "rh": 0,  
23     "inputs": 0,  
24     "x": 170,  
25     "y": 220,  
26     "wires": [  
27       [ "ff450af3556b18ec",  
28         "9f04f7bea47f19fc",  
29         "ad6476dd6442ad0a",  
30         "7bd35c954f6364b9"  
31     ]  
32   ],  
33   },  
34   {  
35     "id": "b7a99e607c34c973",  
36     "type": "ui_slider",  
37     "z": "88ad40c4a9bed753",  
38     "name": "",  
39     "label": "Consigna( C )",
```

```
40     "tooltip": "",
41     "group": "00f9fa0f51519116",
42     "order": 0,
43     "width": 0,
44     "height": 0,
45     "passthru": true,
46     "outs": "end",
47     "topic": "SV",
48     "topicType": "str",
49     "min": 0,
50     "max": "100",
51     "step": 1,
52     "className": "",
53     "x": 160,
54     "y": 280,
55     "wires": [
56         [
57             "dbdaf3ca7407ce2b"
58         ]
59     ],
60 },
61 {
62     "id": "ff450af3556b18ec",
63     "type": "ui_gauge",
64     "z": "88ad40c4a9bed753",
65     "name": "",
66     "group": "b5eab2e698a68696",
67     "order": 0,
68     "width": 0,
69     "height": 0,
70     "gtype": "gage",
71     "title": "Temperatura",
72     "label": " C ",
73     "format": "{{value}}",
74     "min": 0,
75     "max": "100",
76     "colors": [
77         "#ffff00",
78         "#ff7a00",
79         "#ff0000"
80     ],
81     "seg1": "",
82     "seg2": "",
83     "diff": false,
84     "className": "",
85     "x": 670,
86     "y": 40,
87     "wires": []
88 },
```

```
89     {
90         "id": "9c88b2368024860d",
91         "type": "ui_chart",
92         "z": "88ad40c4a9bed753",
93         "name": "",
94         "group": "6f058934711504e9",
95         "order": 0,
96         "width": "14",
97         "height": "4",
98         "label": "Temperatura_ ( C )",
99         "chartType": "line",
100        "legend": "false",
101        "xformat": "HH:mm:ss",
102        "interpolate": "linear",
103        "nodata": "",
104        "dot": false,
105        "ymin": "0",
106        "ymax": "100",
107        "removeOlder": "1",
108        "removeOlderPoints": "",
109        "removeOlderUnit": "3600",
110        "cutout": 0,
111        "useOneColor": false,
112        "useUTC": false,
113        "colors": [
114            "#0000ff",
115            "#e62e2e",
116            "#ffffff",
117            "#ffffff",
118            "#ffffff",
119            "#ffffff",
120            "#ffffff",
121            "#ffffff",
122            "#ffffff"
123        ],
124        "outputs": 1,
125        "useDifferentColor": false,
126        "className": "",
127        "x": 1110,
128        "y": 120,
129        "wires": [
130            []
131        ]
132    },
133    {
134        "id": "02be531eee083145",
135        "type": "ui_button",
136        "z": "88ad40c4a9bed753",
137        "name": "",
```

```
138     "group": "6f058934711504e9",
139     "order": 1,
140     "width": 0,
141     "height": 0,
142     "passthru": false,
143     "label": "Reiniciar gráficas",
144     "tooltip": "",
145     "color": "",
146     "bgcolor": "",
147     "className": "",
148     "icon": "",
149     "payload": "[]",
150     "payloadType": "json",
151     "topic": "topic",
152     "topicType": "msg",
153     "x": 690,
154     "y": 200,
155     "wires": [
156       [
157         "9c88b2368024860d",
158         "f1aeb54b6cd337e3"
159       ]
160     ],
161   },
162   {
163     "id": "f1aeb54b6cd337e3",
164     "type": "ui_chart",
165     "z": "88ad40c4a9bed753",
166     "name": "",
167     "group": "6f058934711504e9",
168     "order": 2,
169     "width": "14",
170     "height": "4",
171     "label": "Señal de control (0-100)",
172     "chartType": "line",
173     "legend": "false",
174     "xformat": "HH:mm:ss",
175     "interpolate": "linear",
176     "nodata": "",
177     "dot": false,
178     "ymin": "0",
179     "ymax": "100",
180     "removeOlder": "1",
181     "removeOlderPoints": "",
182     "removeOlderUnit": "3600",
183     "cutout": 0,
184     "useOneColor": false,
185     "useUTC": false,
186     "colors": [
```

```
187         "#00ff00",
188         "#ffffff",
189         "#ffffff",
190         "#ffffff",
191         "#ffffff",
192         "#ffffff",
193         "#ffffff",
194         "#ffffff",
195         "#ffffff"
196     ],
197     "outputs": 1,
198     "useDifferentColor": false,
199     "className": "",
200     "x": 1130,
201     "y": 280,
202     "wires": [
203         []
204     ]
205 },
206 {
207     "id": "8053493f2a1e9174",
208     "type": "change",
209     "z": "88ad40c4a9bed753",
210     "name": "",
211     "rules": [
212         {
213             "t": "set",
214             "p": "topic",
215             "pt": "msg",
216             "to": "Se al_de_control",
217             "tot": "str"
218         }
219     ],
220     "action": "",
221     "property": "",
222     "from": "",
223     "to": "",
224     "reg": false,
225     "x": 840,
226     "y": 280,
227     "wires": [
228         [
229             "f1aeb54b6cd337e3"
230         ]
231     ]
232 },
233 {
234     "id": "9f04f7bea47f19fc",
235     "type": "change",
```

```
236     "z": "88ad40c4a9bed753",
237     "name": "",
238     "rules": [
239         {
240             "t": "set",
241             "p": "topic",
242             "pt": "msg",
243             "to": "Temperatura",
244             "tot": "str"
245         }
246     ],
247     "action": "",
248     "property": "",
249     "from": "",
250     "to": "",
251     "reg": false,
252     "x": 700,
253     "y": 120,
254     "wires": [
255         [
256             "9c88b2368024860d"
257         ]
258     ]
259 },
260 {
261     "id": "dbdaf3ca7407ce2b",
262     "type": "easy-pid-controller",
263     "z": "88ad40c4a9bed753",
264     "name": "PID",
265     "k_p": "8.49",
266     "k_i": "0.02",
267     "k_d": "1160.76",
268     "sensor_type": "0-10V",
269     "dt": "3",
270     "range_min": 0,
271     "range_max": "100",
272     "x": 410,
273     "y": 280,
274     "wires": [
275         [
276             "6ee9e284e02f78fa"
277         ]
278     ]
279 },
280 {
281     "id": "ad6476dd6442ad0a",
282     "type": "change",
283     "z": "88ad40c4a9bed753",
284     "name": "",
```



```
285     "rules": [  
286         {  
287             "t": "set",  
288             "p": "topic",  
289             "pt": "msg",  
290             "to": "PV",  
291             "tot": "str"  
292         }  
293     ],  
294     "action": "",  
295     "property": "",  
296     "from": "",  
297     "to": "",  
298     "reg": false,  
299     "x": 620,  
300     "y": 240,  
301     "wires": [  
302         [  
303             "dbdaf3ca7407ce2b"  
304         ]  
305     ]  
306 },  
307 {  
308     "id": "ed7e9c57bc96c785",  
309     "type": "ui_switch",  
310     "z": "88ad40c4a9bed753",  
311     "name": "",  
312     "label": "Habilitar PID",  
313     "tooltip": "",  
314     "group": "00f9fa0f51519116",  
315     "order": 1,  
316     "width": 0,  
317     "height": 0,  
318     "passthru": true,  
319     "decouple": "false",  
320     "topic": "auto",  
321     "topicType": "msg",  
322     "style": "",  
323     "onvalue": "true",  
324     "onvalueType": "bool",  
325     "onicon": "",  
326     "oncolor": "",  
327     "offvalue": "false",  
328     "offvalueType": "bool",  
329     "officon": "",  
330     "offcolor": "",  
331     "animate": false,  
332     "className": "",  
333     "x": 150,
```

```
334         "y": 600,
335         "wires": [
336             [
337                 "49e2af73c8e92efa"
338             ]
339         ],
340     },
341     {
342         "id": "49e2af73c8e92efa",
343         "type": "trigger",
344         "z": "88ad40c4a9bed753",
345         "name": "",
346         "op1": "",
347         "op2": "0",
348         "op1type": "pay",
349         "op2type": "str",
350         "duration": "-3",
351         "extend": false,
352         "overrideDelay": false,
353         "units": "s",
354         "reset": "",
355         "bytopic": "all",
356         "topic": "topic",
357         "outputs": 1,
358         "x": 330,
359         "y": 600,
360         "wires": [
361             [
362                 "27f587cf518c8ceb"
363             ]
364         ],
365     },
366     {
367         "id": "27f587cf518c8ceb",
368         "type": "change",
369         "z": "88ad40c4a9bed753",
370         "name": "",
371         "rules": [
372             {
373                 "t": "set",
374                 "p": "topic",
375                 "pt": "msg",
376                 "to": "auto",
377                 "tot": "str"
378             }
379         ],
380         "action": "",
381         "property": "",
382         "from": "",
```

```

383         "to": "",
384         "reg": false,
385         "x": 540,
386         "y": 600,
387         "wires": [
388             [
389                 "dbdaf3ca7407ce2b"
390             ]
391         ]
392     },
393     {
394         "id": "b2ff8908a6f210f9",
395         "type": "mqtt_out",
396         "z": "88ad40c4a9bed753",
397         "name": "",
398         "topic": "planta/senal_control",
399         "qos": "0",
400         "retain": "",
401         "respTopic": "",
402         "contentType": "",
403         "userProps": "",
404         "correl": "",
405         "expiry": "",
406         "broker": "b4cbff7cc98e8a0c",
407         "x": 840,
408         "y": 320,
409         "wires": []
410     },
411     {
412         "id": "6ee9e284e02f78fa",
413         "type": "function",
414         "z": "88ad40c4a9bed753",
415         "name": "Salida",
416         "func": "var control=msg.payload.Output;\nif(control<
            ↳ >100)\n{\ncontrol=100;\n}\nif(control<
            ↳ 0)\n{\ncontrol=0;\n}\nmsg.payload=
            ↳ control;\nreturn msg;",
417         "outputs": 1,
418         "timeout": 0,
419         "noerr": 0,
420         "initialize": "",
421         "finalize": "",
422         "libs": [],
423         "x": 570,
424         "y": 280,
425         "wires": [
426             [
427                 "8053493f2a1e9174",
428                 "b2ff8908a6f210f9",

```

```
429         "26f1b4985c6ba4db"
430     ]
431 ]
432 },
433 {
434     "id": "7bd35c954f6364b9",
435     "type": "influxdb_out",
436     "z": "88ad40c4a9bed753",
437     "influxdb": "8451dcd15171604c",
438     "name": "Temperatura_a_InfluxDB",
439     "measurement": "Temperatura",
440     "precision": "",
441     "retentionPolicy": "",
442     "database": "database",
443     "precisionV18FluxV20": "ms",
444     "retentionPolicyV18Flux": "",
445     "org": "2324_GEIA_7",
446     "bucket": "Planta",
447     "x": 410,
448     "y": 40,
449     "wires": []
450 },
451 {
452     "id": "26f1b4985c6ba4db",
453     "type": "influxdb_out",
454     "z": "88ad40c4a9bed753",
455     "influxdb": "8451dcd15171604c",
456     "name": "Se_al_de_control_a_InfluxDB",
457     "measurement": "Se_al_de_control",
458     "precision": "",
459     "retentionPolicy": "",
460     "database": "database",
461     "precisionV18FluxV20": "ms",
462     "retentionPolicyV18Flux": "",
463     "org": "2324_GEIA_7",
464     "bucket": "Planta",
465     "x": 860,
466     "y": 360,
467     "wires": []
468 },
469 {
470     "id": "b4cbff7cc98e8a0c",
471     "type": "mqtt-broker",
472     "name": "Mosquitto_RPI_UDC",
473     "broker": "10.20.29.190",
474     "port": "1883",
475     "clientid": "",
476     "autoConnect": true,
477     "usetls": false,
```

```
478     "protocolVersion": "4",
479     "keepalive": "60",
480     "cleansession": true,
481     "autoUnsubscribe": true,
482     "birthTopic": "",
483     "birthQos": "0",
484     "birthRetain": "false",
485     "birthPayload": "",
486     "birthMsg": {},
487     "closeTopic": "",
488     "closeQos": "0",
489     "closeRetain": "false",
490     "closePayload": "",
491     "closeMsg": {},
492     "willTopic": "",
493     "willQos": "0",
494     "willRetain": "false",
495     "willPayload": "",
496     "willMsg": {},
497     "userProps": "",
498     "sessionExpiry": ""
499 },
500 {
501     "id": "00f9fa0f51519116",
502     "type": "ui_group",
503     "name": "Entradas",
504     "tab": "e979c0f74a28ed30",
505     "order": 3,
506     "disp": true,
507     "width": "6",
508     "collapse": false,
509     "className": ""
510 },
511 {
512     "id": "b5eab2e698a68696",
513     "type": "ui_group",
514     "name": "Indicadores",
515     "tab": "e979c0f74a28ed30",
516     "order": 1,
517     "disp": true,
518     "width": "6",
519     "collapse": false,
520     "className": "Indicadores"
521 },
522 {
523     "id": "6f058934711504e9",
524     "type": "ui_group",
525     "name": "Gráficas",
526     "tab": "e979c0f74a28ed30",
```

```

527     "order": 2,
528     "disp": true,
529     "width": "14",
530     "collapse": false,
531     "className": "Gráficas"
532   },
533   {
534     "id": "8451dcd15171604c",
535     "type": "influxdb",
536     "hostname": "127.0.0.1",
537     "port": "8086",
538     "protocol": "http",
539     "database": "database",
540     "name": "InfluxDB-UDC",
541     "usetls": false,
542     "tls": "",
543     "influxdbVersion": "2.0",
544     "url": "http://10.20.24.183:8086",
545     "timeout": "10",
546     "rejectUnauthorized": true
547   },
548   {
549     "id": "e979c0f74a28ed30",
550     "type": "ui_tab",
551     "name": "Control-Planta-de-temperatura",
552     "icon": "dashboard",
553     "disabled": false,
554     "hidden": false
555   }
556 ]

```

3.4.3. Código controlador Relay-Feedback en Node-Red

Se incluye a continuación el código JSON del controlador de tipo relé implementado en Node-Red, con las opciones de visualización en dashboard, publicación y suscripción en servidor MQTT y envío a InfluxDB.

```

1  [
2    {
3      "id": "7c9fce843c555673",
4      "type": "tab",
5      "label": "Relay-Feedback",
6      "disabled": false,
7      "info": "",
8      "env": []
9    },
10   {
11     "id": "4f7c56a73e1a5881",

```

```
12     "type": "mqtt_in",
13     "z": "7c9fce843c555673",
14     "name": "",
15     "topic": "planta/temperatura",
16     "qos": "0",
17     "datatype": "auto-detect",
18     "broker": "b4cbff7cc98e8a0c",
19     "nl": false,
20     "rap": true,
21     "rh": 0,
22     "inputs": 0,
23     "x": 130,
24     "y": 40,
25     "wires": [
26         [
27             "5df7e9a681ed18ad"
28         ]
29     ]
30 },
31 {
32     "id": "3adfedae180bcf2a",
33     "type": "ui_gauge",
34     "z": "7c9fce843c555673",
35     "name": "",
36     "group": "b5eab2e698a68696",
37     "order": 0,
38     "width": 0,
39     "height": 0,
40     "gtype": "gage",
41     "title": "Temperatura",
42     "label": " C ",
43     "format": "{{value}}",
44     "min": 0,
45     "max": "100",
46     "colors": [
47         "#ffff00",
48         "#ff7a00",
49         "#ff0000"
50     ],
51     "seg1": "",
52     "seg2": "",
53     "diff": false,
54     "className": "",
55     "x": 350,
56     "y": 100,
57     "wires": []
58 },
59 {
60     "id": "76e6a950d200ad3a",
```

```

61      "type": "ui_button",
62      "z": "7c9fce843c555673",
63      "name": "",
64      "group": "6f058934711504e9",
65      "order": 1,
66      "width": 0,
67      "height": 0,
68      "passthru": false,
69      "label": "Reiniciar gráficas",
70      "tooltip": "",
71      "color": "",
72      "bgcolor": "",
73      "className": "",
74      "icon": "",
75      "payload": "[]",
76      "payloadType": "json",
77      "topic": "topic",
78      "topicType": "msg",
79      "x": 370,
80      "y": 200,
81      "wires": [
82          [
83              "1f73f80bce9fb3e3"
84          ]
85      ]
86  },
87  {
88      "id": "1f73f80bce9fb3e3",
89      "type": "ui_chart",
90      "z": "7c9fce843c555673",
91      "name": "",
92      "group": "6f058934711504e9",
93      "order": 2,
94      "width": "14",
95      "height": "8",
96      "label": "Gráfica de planta",
97      "chartType": "line",
98      "legend": "true",
99      "xformat": "HH:mm:ss",
100     "interpolate": "linear",
101     "nodata": "",
102     "dot": false,
103     "ymin": "0",
104     "ymax": "100",
105     "removeOlder": "1",
106     "removeOlderPoints": "",
107     "removeOlderUnit": "3600",
108     "cutout": 0,
109     "useOneColor": false,

```



```
110     "useUTC": false,
111     "colors": [
112         "#0000ff",
113         "#ff0000",
114         "#ffffff",
115         "#ffffff",
116         "#ffffff",
117         "#ffffff",
118         "#ffffff",
119         "#ffffff",
120         "#ffffff"
121     ],
122     "outputs": 1,
123     "useDifferentColor": false,
124     "className": "",
125     "x": 710,
126     "y": 160,
127     "wires": [
128         []
129     ]
130 },
131 {
132     "id": "8c597a9035b13f48",
133     "type": "change",
134     "z": "7c9fce843c555673",
135     "name": "",
136     "rules": [
137         {
138             "t": "set",
139             "p": "topic",
140             "pt": "msg",
141             "to": "Se al_ude_control",
142             "tot": "str"
143         }
144     ],
145     "action": "",
146     "property": "",
147     "from": "",
148     "to": "",
149     "reg": false,
150     "x": 380,
151     "y": 240,
152     "wires": [
153         [
154             "1f73f80bce9fb3e3"
155         ]
156     ]
157 },
158 {
```

```
159     "id": "c0951ed81417bd0a",
160     "type": "change",
161     "z": "7c9fce843c555673",
162     "name": "",
163     "rules": [
164         {
165             "t": "set",
166             "p": "topic",
167             "pt": "msg",
168             "to": "Temperatura",
169             "tot": "str"
170         }
171     ],
172     "action": "",
173     "property": "",
174     "from": "",
175     "to": "",
176     "reg": false,
177     "x": 380,
178     "y": 160,
179     "wires": [
180         [
181             "1f73f80bce9fb3e3"
182         ]
183     ]
184 },
185 {
186     "id": "933289489e44578f",
187     "type": "mqtt_out",
188     "z": "7c9fce843c555673",
189     "name": "",
190     "topic": "planta/senal_control",
191     "qos": "0",
192     "retain": "",
193     "respTopic": "",
194     "contentType": "",
195     "userProps": "",
196     "correl": "",
197     "expiry": "",
198     "broker": "b4cbff7cc98e8a0c",
199     "x": 200,
200     "y": 420,
201     "wires": []
202 },
203 {
204     "id": "10c773272194ac0a",
205     "type": "hysteresis",
206     "z": "7c9fce843c555673",
207     "name": "Hist resis",
```

```
208     "ThresholdType": "fixed",
209     "ThresholdRising": "60",
210     "ThresholdFalling": "40",
211     "TopicThreshold": "",
212     "TopicCurrent": "",
213     "ThresholdDeltaRising": "",
214     "ThresholdDeltaFalling": "",
215     "InitialMessage": false,
216     "OutRisingType": "num",
217     "OutRisingValue": "0",
218     "OutFallingType": "num",
219     "OutFallingValue": "100",
220     "OutTopicType": "top",
221     "OutTopicValue": "",
222     "DynRaiseError": false,
223     "x": 160,
224     "y": 360,
225     "wires": [
226         [
227             "d3d88bb158a67038"
228         ]
229     ],
230 },
231 {
232     "id": "d3d88bb158a67038",
233     "type": "trigger",
234     "z": "7c9fce843c555673",
235     "name": "",
236     "op1": "",
237     "op2": "0",
238     "op1type": "pay",
239     "op2type": "str",
240     "duration": "-3",
241     "extend": false,
242     "overrideDelay": false,
243     "units": "s",
244     "reset": "",
245     "bytopic": "all",
246     "topic": "topic",
247     "outputs": 1,
248     "x": 350,
249     "y": 360,
250     "wires": [
251         [
252             "d2e8cfa0548d784d"
253         ]
254     ],
255 },
256 {
```

```
257     "id": "a927eda7515229d1",
258     "type": "influxdb_out",
259     "z": "7c9fce843c555673",
260     "influxdb": "8451dcd15171604c",
261     "name": "Temperatura_a_InfluxDB",
262     "measurement": "Temperatura",
263     "precision": "",
264     "retentionPolicy": "",
265     "database": "database",
266     "precisionV18FluxV20": "ms",
267     "retentionPolicyV18Flux": "",
268     "org": "2324_GEIA_7",
269     "bucket": "Planta",
270     "x": 210,
271     "y": 480,
272     "wires": []
273 },
274 {
275     "id": "5df7e9a681ed18ad",
276     "type": "link_out",
277     "z": "7c9fce843c555673",
278     "name": "Temperatura",
279     "mode": "link",
280     "links": [
281         "0de0fd5e4b96c982",
282         "73318e7ac549d00e",
283         "90214f99e219c9f1",
284         "b834987e99bcecf8"
285     ],
286     "x": 345,
287     "y": 40,
288     "wires": []
289 },
290 {
291     "id": "73318e7ac549d00e",
292     "type": "link_in",
293     "z": "7c9fce843c555673",
294     "name": "Temperatura",
295     "links": [
296         "5df7e9a681ed18ad"
297     ],
298     "x": 55,
299     "y": 100,
300     "wires": [
301         [
302             "3adfedae180bcf2a",
303             "c0951ed81417bd0a"
304         ]
305     ]
306 }
```

```
306     },
307     {
308         "id": "0de0fd5e4b96c982",
309         "type": "link_in",
310         "z": "7c9fce843c555673",
311         "name": "Temperatura",
312         "links": [
313             "5df7e9a681ed18ad"
314         ],
315         "x": 55,
316         "y": 360,
317         "wires": [
318             [
319                 "10c773272194ac0a"
320             ]
321         ]
322     },
323     {
324         "id": "d2e8cfa0548d784d",
325         "type": "link_out",
326         "z": "7c9fce843c555673",
327         "name": "Se al_de_control",
328         "mode": "link",
329         "links": [
330             "4fe1ea6e11fe47a8",
331             "61cf73d84c8ec19c",
332             "6fa4ae3107a21bbe",
333             "0c2a7fa17922d3c5"
334         ],
335         "x": 485,
336         "y": 360,
337         "wires": []
338     },
339     {
340         "id": "4fe1ea6e11fe47a8",
341         "type": "link_in",
342         "z": "7c9fce843c555673",
343         "name": "Se al_de_control",
344         "links": [
345             "d2e8cfa0548d784d"
346         ],
347         "x": 55,
348         "y": 240,
349         "wires": [
350             [
351                 "8c597a9035b13f48"
352             ]
353         ]
354     },
```

```
355 {
356     "id": "61cf73d84c8ec19c",
357     "type": "link_in",
358     "z": "7c9fce843c555673",
359     "name": "Se al_de_control",
360     "links": [
361         "d2e8cfa0548d784d"
362     ],
363     "x": 55,
364     "y": 420,
365     "wires": [
366         [
367             "933289489e44578f"
368         ]
369     ],
370 },
371 {
372     "id": "b834987e99bcecf8",
373     "type": "link_in",
374     "z": "7c9fce843c555673",
375     "name": "Temperatura",
376     "links": [
377         "5df7e9a681ed18ad"
378     ],
379     "x": 55,
380     "y": 480,
381     "wires": [
382         [
383             "a927eda7515229d1"
384         ]
385     ],
386 },
387 {
388     "id": "6fa4ae3107a21bbe",
389     "type": "link_in",
390     "z": "7c9fce843c555673",
391     "name": "Se al_de_control",
392     "links": [
393         "d2e8cfa0548d784d"
394     ],
395     "x": 55,
396     "y": 540,
397     "wires": [
398         [
399             "8042e7d4e4413e6e"
400         ]
401     ],
402 },
403 {
```

```
404     "id": "8042e7d4e4413e6e",
405     "type": "influxdb_out",
406     "z": "7c9fce843c555673",
407     "influxdb": "8451dcd15171604c",
408     "name": "Se al_de_control_a_InfluxDB",
409     "measurement": "Se al_de_control",
410     "precision": "",
411     "retentionPolicy": "",
412     "database": "database",
413     "precisionV18FluxV20": "ms",
414     "retentionPolicyV18Flux": "",
415     "org": "2324_GEIA_7",
416     "bucket": "Planta",
417     "x": 220,
418     "y": 540,
419     "wires": []
420 },
421 {
422     "id": "b4cbff7cc98e8a0c",
423     "type": "mqtt-broker",
424     "name": "Mosquitto_RPI_-_UDC",
425     "broker": "10.20.29.190",
426     "port": "1883",
427     "clientId": "",
428     "autoConnect": true,
429     "usetls": false,
430     "protocolVersion": "4",
431     "keepalive": "60",
432     "cleansession": true,
433     "autoUnsubscribe": true,
434     "birthTopic": "",
435     "birthQos": "0",
436     "birthRetain": "false",
437     "birthPayload": "",
438     "birthMsg": {},
439     "closeTopic": "",
440     "closeQos": "0",
441     "closeRetain": "false",
442     "closePayload": "",
443     "closeMsg": {},
444     "willTopic": "",
445     "willQos": "0",
446     "willRetain": "false",
447     "willPayload": "",
448     "willMsg": {},
449     "userProps": "",
450     "sessionExpiry": ""
451 },
452 {
```

```
453     "id": "b5eab2e698a68696",
454     "type": "ui_group",
455     "name": "Indicadores",
456     "tab": "e979c0f74a28ed30",
457     "order": 1,
458     "disp": true,
459     "width": "6",
460     "collapse": false,
461     "className": "Indicadores"
462   },
463   {
464     "id": "6f058934711504e9",
465     "type": "ui_group",
466     "name": "Gráficas",
467     "tab": "e979c0f74a28ed30",
468     "order": 2,
469     "disp": true,
470     "width": "14",
471     "collapse": false,
472     "className": "Gráficas"
473   },
474   {
475     "id": "8451dcd15171604c",
476     "type": "influxdb",
477     "hostname": "127.0.0.1",
478     "port": "8086",
479     "protocol": "http",
480     "database": "database",
481     "name": "InfluxDB-UDC",
482     "usetls": false,
483     "tls": "",
484     "influxdbVersion": "2.0",
485     "url": "http://10.20.24.183:8086",
486     "timeout": "10",
487     "rejectUnauthorized": true
488   },
489   {
490     "id": "e979c0f74a28ed30",
491     "type": "ui_tab",
492     "name": "Control-Planta de temperatura",
493     "icon": "dashboard",
494     "disabled": false,
495     "hidden": false
496   }
497 ]
```


Título Diseño de equipo de adquisición de datos inalámbrico
para plantas de control de temperatura

Estudios con entidad propia

Peticionario Escuela Politécnica de Ingeniería de Ferrol
Rúa Mendizábal, s/n, Campus de Esteiro,
15403, Ferrol

Fecha Septiembre 2024

Autor El Alumno

Fdo. Iván Hermida Mella

Índice general

4.1	Contenido	103
-----	---------------------	-----

4.1. Contenido

Dado que el presente proyecto se centra en la elaboración del hardware y el software de un prototipo de sistema de control, no se considera necesaria la inclusión de Estudios con entidad propia recogida en la norma UNE157001, pues no existen exigencias legales que requieran este tipo de estudios.

Título Diseño de equipo de adquisición de datos inalámbrico
para plantas de control de temperatura

Planos

Peticionario Escuela Politécnica de Ingeniería de Ferrol
Rúa Mendizábal, s/n, Campus de Esteiro,
15403, Ferrol

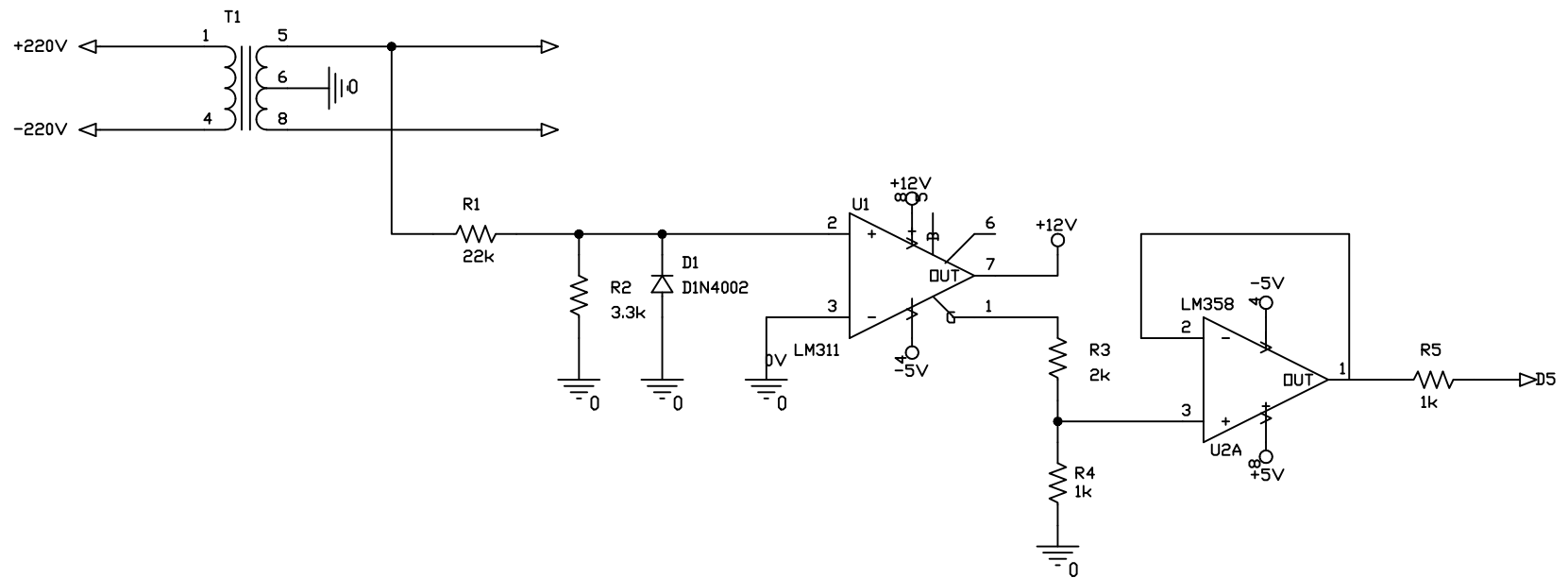
Fecha Septiembre 2024

Autor El Alumno

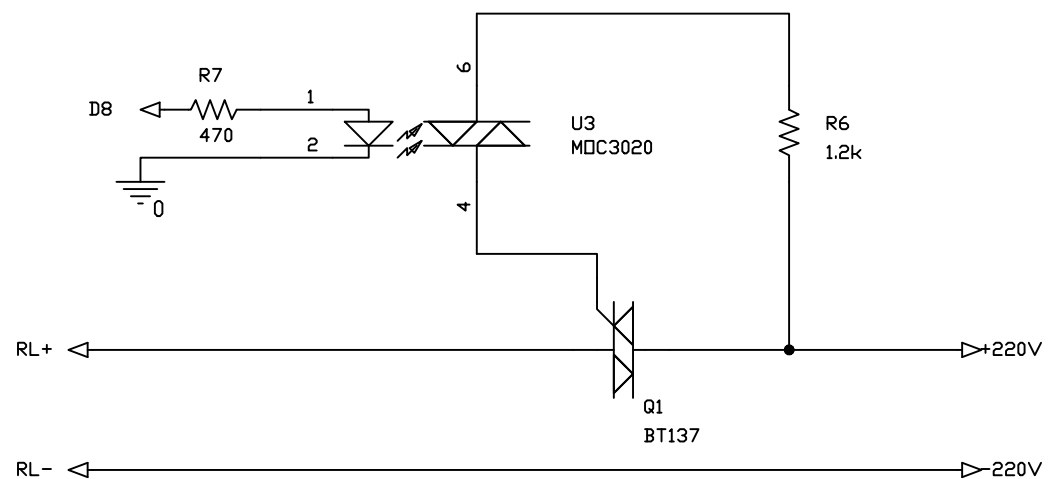
Fdo. Iván Hermida Mella

Índice general

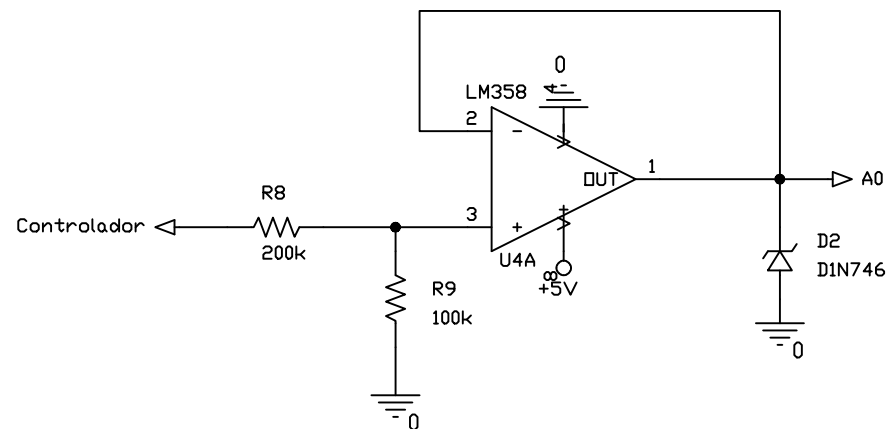
Circuito detector de paso por cero	107
Circuito de disparo	108
Circuito de acondicionamiento de la señal de control	109
Circuito de alimentaciones	110
Circuito de acondicionamiento del sensor	111
Conexiones con Wemos D1	112



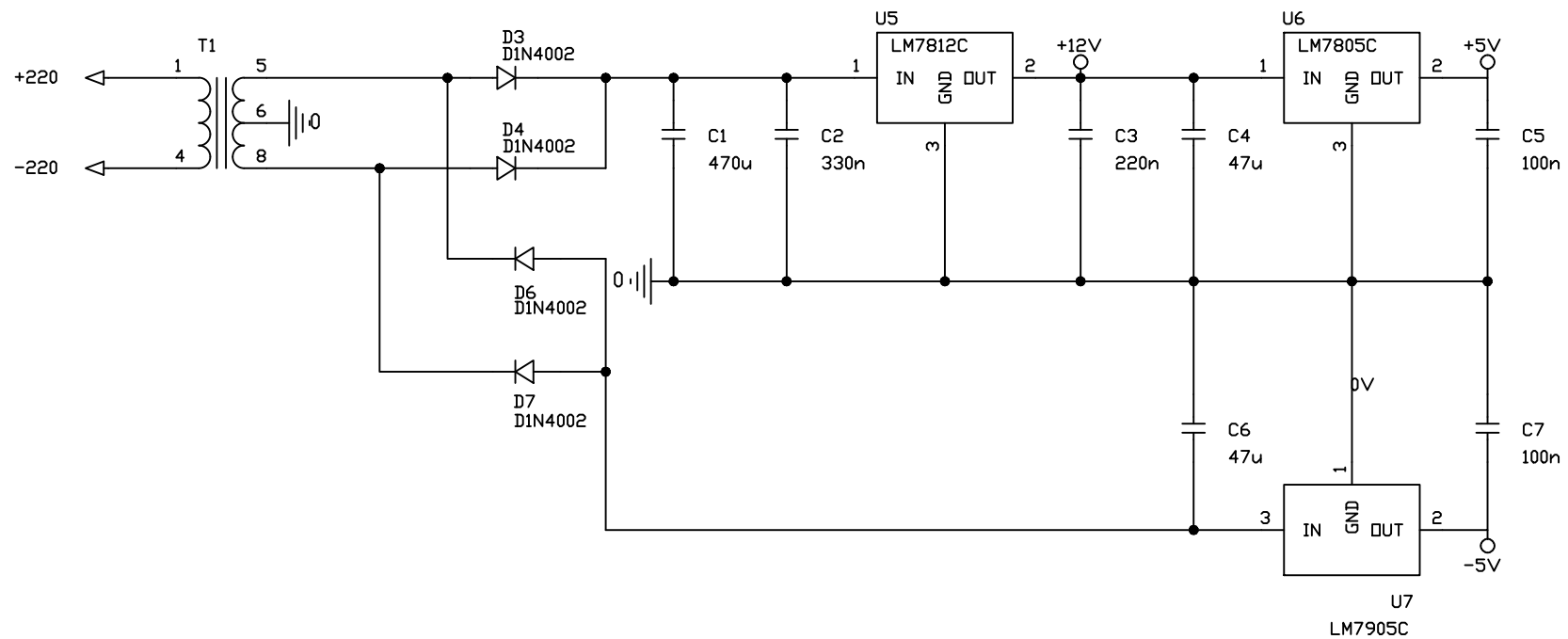
GRADO EN INGENIERÍA ELECTRÓNICA INDUSTRIAL Y AUTOMÁTICA		TFG Nº: 2324_GEIA_7
Escuela Politécnica de Ingeniería de Ferrol 		
TÍTULO DEL TFG:		
DISEÑO DE EQUIPO DE ADQUISICIÓN DE DATOS INALÁMBRICO PARA PLANTAS DE CONTROL DE TEMPERATURA		
TÍTULO DEL PLANO:		FECHA: Septiembre 2024
CIRCUITO DETECTOR DE PASO POR CERO		ESCALA: N/A
AUTOR:	FIRMA:	PLANO Nº: 01
IVÁN HERMIDA MELLA		



GRADO EN INGENIERÍA ELECTRÓNICA INDUSTRIAL Y AUTOMÁTICA		TFG Nº: 2324_GEIA_7
Escuela Politécnica de Ingeniería de Ferrol 		
TÍTULO DEL TFG:		
DISEÑO DE EQUIPO DE ADQUISICIÓN DE DATOS INALÁMBRICO PARA PLANTAS DE CONTROL DE TEMPERATURA		
TÍTULO DEL PLANO:		FECHA: Septiembre 2024
CIRCUITO DE DISPARO		ESCALA: N/A
AUTOR:	FIRMA:	PLANO Nº: 02
IVÁN HERMIDA MELLA		

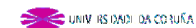


GRADO EN INGENIERÍA ELECTRÓNICA INDUSTRIAL Y AUTOMÁTICA		TFG Nº: 2324_GEIA_7
Escuela Politécnica de Ingeniería de Ferrol 		
TÍTULO DEL TFG:		
DISEÑO DE EQUIPO DE ADQUISICIÓN DE DATOS INALÁMBRICO PARA PLANTAS DE CONTROL DE TEMPERATURA		
TÍTULO DEL PLANO:		FECHA: Septiembre 2024
CIRCUITO DE ACONDICIONAMIENTO DE LA SEÑAL DE CONTROL		ESCALA: N/A
AUTOR:	FIRMA:	PLANO Nº: 03
IVÁN HERMIDA MELLA		



GRADO EN INGENIERÍA ELECTRÓNICA INDUSTRIAL Y AUTOMÁTICA

Escuela Politécnica de Ingeniería de Ferrol



TFG Nº: 2324_GEIA_7

TÍTULO DEL TFG:

DISEÑO DE EQUIPO DE ADQUISICIÓN DE DATOS INALÁMBRICO PARA PLANTAS DE CONTROL DE TEMPERATURA

TÍTULO DEL PLANO:

CIRCUITO DE ALIMENTACIONES

FECHA: Septiembre 2024

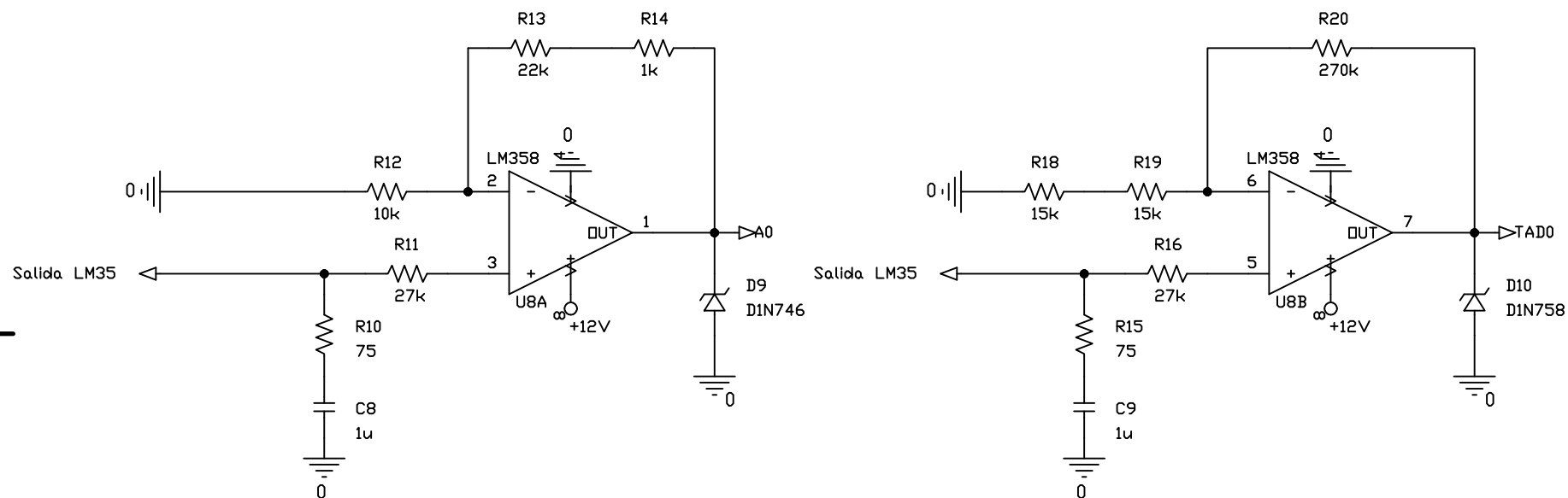
ESCALA: N/A

AUTOR:

IVÁN HERMIDA MELLA

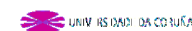
FIRMA:

PLANO Nº: 04



GRADO EN INGENIERÍA ELECTRÓNICA INDUSTRIAL Y AUTOMÁTICA

Escuela Politécnica de Ingeniería de Ferrol



TFG N°: 2324_GEIA_7

TÍTULO DEL TFG:

DISEÑO DE EQUIPO DE ADQUISICIÓN DE DATOS INALÁMBRICO PARA PLANTAS DE CONTROL DE TEMPERATURA

TÍTULO DEL PLANO:

CIRCUITO DE ACONDICIONAMIENTO DEL SENSOR

FECHA: Septiembre 2024

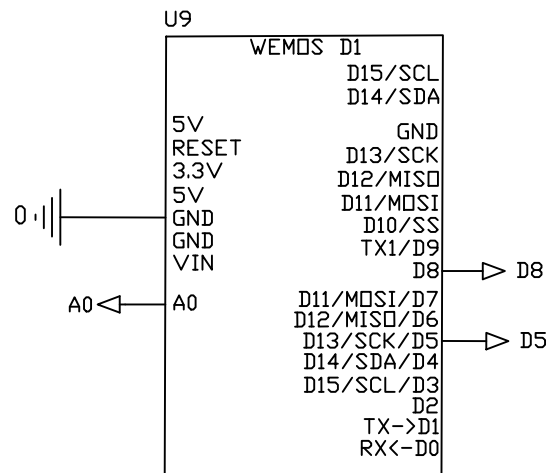
ESCALA: N/A

AUTOR:

IVÁN HERMIDA MELLA

FIRMA:

PLANO N°: 05



GRADO EN INGENIERÍA ELECTRÓNICA INDUSTRIAL Y AUTOMÁTICA		TFG Nº: 2324_GEIA_7
Escuela Politécnica de Ingeniería de Ferrol 		
TÍTULO DEL TFG:		
DISEÑO DE EQUIPO DE ADQUISICIÓN DE DATOS INALÁMBRICO PARA PLANTAS DE CONTROL DE TEMPERATURA		
TÍTULO DEL PLANO:		FECHA: Septiembre 2024
CONEXIONES CON WEMOS D1		ESCALA: N/A
AUTOR:	FIRMA:	PLANO Nº: 06
IVÁN HERMIDA MELLA		

Título Diseño de equipo de adquisición de datos inalámbrico
para plantas de control de temperatura

Pliego de condiciones

Peticionario Escuela Politécnica de Ingeniería de Ferrol
Rúa Mendizábal, s/n, Campus de Esteiro,
15403, Ferrol

Fecha Septiembre 2024

Autor El Alumno

Fdo. Iván Hermida Mella

Índice general

6.1	Contenido	115
-----	---------------------	-----

6.1. Contenido

Dado que el presente proyecto se centra en la elaboración del hardware y el software de un prototipo de sistema de control, no se considera necesaria la inclusión de Pliego de condiciones recogida en la norma UNE157001, pues el proyecto no ha sido ejecutado con la pretensión de que el objeto del trabajo pueda materializarse.

Título Diseño de equipo de adquisición de datos inalámbrico
para plantas de control de temperatura

Mediciones

Peticionario Escuela Politécnica de Ingeniería de Ferrol
Rúa Mendizábal, s/n, Campus de Esteiro,
15403, Ferrol

Fecha Septiembre 2024

Autor El Alumno

Fdo. Iván Hermida Mella

Índice general

7.1	Lista de materiales	119
7.1.0.1	Equipo electrónico	119
7.1.0.2	Conexionado del equipo electrónico	120
7.1.0.3	Equipo de control	120

7.1. Lista de materiales

7.1.0.1. Equipo electrónico

Descripción	Referencia	Cantidad (uds)
Amplificador operacional dual LM358	U2, U4, U8	3
Cable RJ45	-	1
Cable USB a microUSB	-	1
Comparador LM311	U1	1
Condensador cerámico 100nF, tensión 50V	C5, C7	2
Condensador cerámico 220nF, tensión 50V	C3	1
Condensador cerámico 330nF, tensión 50V	C2	1
Condensador electrolítico 1 μ F, tensión 63V	C8, C9	2
Condensador electrolítico 47 μ F, tensión 63V	C4, C6	2
Condensador electrolítico 470 μ F, tensión 63V	C1	1
Diodo D1N4002	D1, D3, D4, D6, D7	5
Diodo zener D1N746	D2, D9	2
Diodo zener D1N758	D10	1
Optoacoplador MOC3020	U3	1
Placa Wemos D1 R1	U9	1
Regulador de tensión LM7805	U6	1
Regulador de tensión LM7812	U5	1
Regulador de tensión LM7905	U7	1
Resistencia 75 Ω , tolerancia 5 %, potencia 1/4W	R10, R15	2
Resistencia 470 Ω , tolerancia 5 %, potencia 1/4W	R7	1
Resistencia 1k Ω , tolerancia 5 %, potencia 1/4W	R4, R5, R14	3
Resistencia 1.2k Ω , tolerancia 5 %, potencia 1/4W	R6	1
Resistencia 2k Ω , tolerancia 5 %, potencia 1/4W	R3	1
Resistencia 3.3k Ω , tolerancia 5 %, potencia 1/4W	R2	1

Resistencia 10k Ω , tolerancia 5 %, potencia 1/4W	R12	1
Resistencia 15k Ω , tolerancia 5 %, potencia 1/4W	R18, R19	2
Resistencia 22k Ω , tolerancia 5 %, potencia 1/4W	R1, R13	2
Resistencia 27k Ω , tolerancia 5 %, potencia 1/4W	R11, R16	2
Resistencia 100k Ω , tolerancia 5 %, potencia 1/4W	R9	1
Resistencia 200k Ω , tolerancia 5 %, potencia 1/4W	R8	1
Resistencia 270K Ω , tolerancia 5 %, potencia 1/4W	R20	1
Transformador 220V//12V con toma intermedia	T1	1
Triac BT137	Q1	1

7.1.0.2. Conexión del equipo electrónico

Descripción	Referencia	Cantidad (m)
Alambre de cobre 0.6mm, recubrimiento PVC	-	4

7.1.0.3. Equipo de control

Descripción	Referencia	Cantidad (uds)
Placa Raspberry Pi 4 Model B	-	1
Cable HDMI 1m	-	1

Título Diseño de equipo de adquisición de datos inalámbrico
para plantas de control de temperatura

Presupuesto

Peticionario Escuela Politécnica de Ingeniería de Ferrol
Rúa Mendizábal, s/n, Campus de Esteiro,
15403, Ferrol

Fecha Septiembre 2024

Autor El Alumno

Fdo. Iván Hermida Mella

Índice general

8.1	Cuadros de precios	123
8.1.1	Equipo electrónico	123
8.1.2	Conexionado del equipo electrónico	124
8.1.3	Equipo de control	124
8.2	Mano de obra	124
8.3	Coste total	125

8.1. Cuadros de precios

8.1.1. Equipo electrónico

Descripción	Cantidad	Precio ud. (€)	Precio total (€)
Amplificador operacional dual LM358	3	1,173	3,519
Cable RJ45	1	7,55	7,55
Cable USB a microUSB	1	3,24	3,24
Comparador LM311	1	1,307	1,307
Condensador cerámico 100nF, tensión 50V	2	0,44	0,88
Condensador cerámico 220nF, tensión 50V	1	0,44	0,44
Condensador cerámico 330nF, tensión 50V	1	0,44	0,44
Condensador electrolítico 1 μ F, tensión 63V	2	0,34	0,68
Condensador electrolítico 47 μ F, tensión 63V	2	1,17	2,34
Condensador electrolítico 470 μ F, tensión 63V	1	0,95	0,95
Diodo D1N4002	5	0,093	0,465
Diodo zener D1N746	2	0,094	0,188
Diodo zener D1N758	1	0,09	0,09
Optoacoplador MOC3020	1	0,654	0,654
Placa Wemos D1 R1	1	6,99	6,99
Regulador de tensión LM7805	1	0,835	0,835
Regulador de tensión LM7812	1	0,955	0,955
Regulador de tensión LM7905	1	0,865	0,865
Resistencia 75 Ω , tolerancia 5 %, potencia 1/4W	2	0,045	0,09
Resistencia 470 Ω , tolerancia 5 %, potencia 1/4W	1	0,045	0,045
Resistencia 1k Ω , tolerancia 5 %, potencia 1/4W	3	0,045	0,135
Resistencia 1.2k Ω , tolerancia 5 %, potencia 1/4W	1	0,045	0,045
Resistencia 2k Ω , tolerancia 5 %, potencia 1/4W	1	0,045	0,045
Resistencia 3.3k Ω , tolerancia 5 %, potencia 1/4W	1	0,045	0,045
Resistencia 10k Ω , tolerancia 5 %, potencia 1/4W	1	0,045	0,045

Resistencia 15k Ω , tolerancia 5 %, potencia 1/4W	2	0,045	0,09
Resistencia 22k Ω , tolerancia 5 %, potencia 1/4W	2	0,045	0,09
Resistencia 27k Ω , tolerancia 5 %, potencia 1/4W	2	0,045	0,09
Resistencia 100k Ω , tolerancia 5 %, potencia 1/4W	1	0,045	0,045
Resistencia 200k Ω , tolerancia 5 %, potencia 1/4W	1	0,045	0,045
Resistencia 270K Ω , tolerancia 5 %, potencia 1/4W	1	0,045	0,045
Transformador 220V//12V con toma intermedia	1		
Triac BT137	1	0,78	0,78
Total			34,12

8.1.2. Conexionado del equipo electrónico

Descripción	Cantidad (m)	Precio ud. (€)	Precio total (€)
Alambre de cobre 0.6mm, recubrimiento PVC	4	1,85	7,4
Total			7,4

8.1.3. Equipo de control

Descripción	Cantidad (uds)	Precio ud. (€)	Precio total (€)
Placa Raspberry Pi 4 Model B	1	49,95	49,95
Cable HDMI 1m	1	3,39	3,39
Total			53,34

8.2. Mano de obra

Actividad	Coste (€/hora)	Nº horas	Coste total
Diseño del equipo electrónico	40,00	35	1400,00
Desarrollo del software	40,00	30	1200,00
Montaje y conexionado	40,00	15	600,00
Prueba y depuración	40,00	3	120,00
Total			3320,00

8.3. Coste total

Operación		Importe (€)
Materiales	Equipo electrónico	34,12
	Conexionado del equipo electrónico	7,40
	Equipo de control	53,34
Mano de obra	Diseño del equipo electrónico	1400,00
	Desarrollo del software	1200,00
	Montaje y conexionado	600,00
	Prueba y depuración	120,00
Total		3414,86

El coste total asciende a la cantidad **de tres mil cuatrocientos catorce euros con ochenta y seis céntimos (3414,86 €)**.

